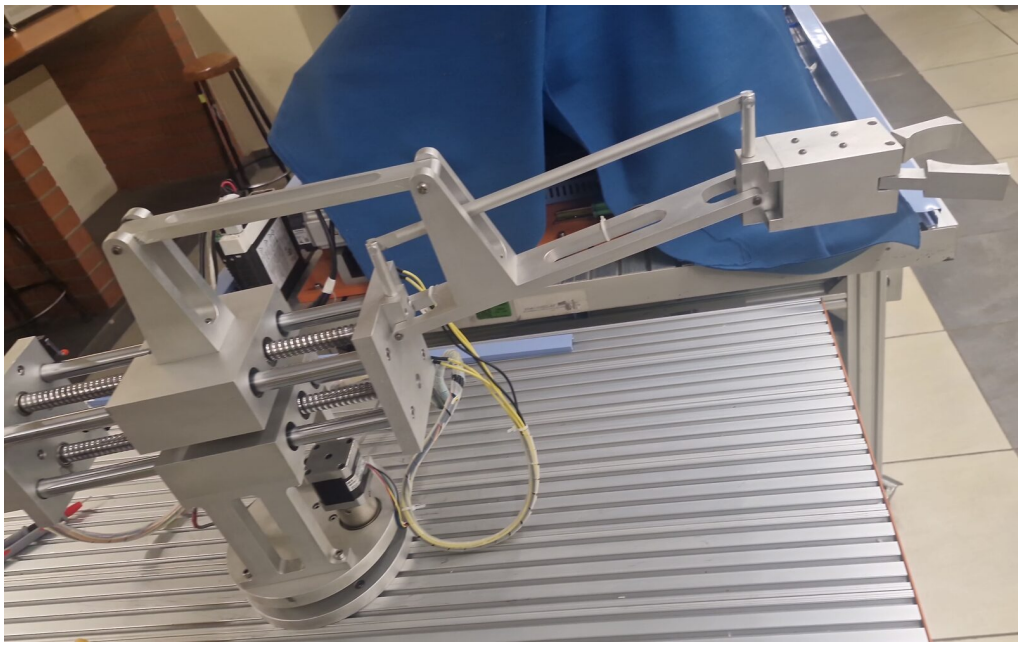
Universidad Mayor de San Andrés · Facultad de Ingeniería · Ingeniería Electrónica

Brazo robótico XK-JS1-1

Manual técnico del brazo y de su gemelo digital



Programas ladder · drivers y frecuencias · cinemática y jacobiano · protocolo PPI
aplicaciones · base de datos · estructura del proyecto

ETN-902 Sistemas de Control II · ETN-1034 Aplicación de Técnicas de Control · ETN-1000 Bases de Datos

Gestión II/2026 · La Paz, 10 de noviembre de 2026

Índice general

1. Objetivos y cómo usar este manual **7**
- 1.1. Marco: el laboratorio integrado . . . 7
- 1.2. Qué objetivo cubre cada capítulo . . . 7
- 1.3. Convenciones . . . 8
- 1.4. Documentos relacionados . . . 8

2. La planta **9**
- 2.1. Qué es y qué hace . . . 9
- 2.2. Ficha técnica del hardware . . . 9
- 2.3. Diagrama de bloques . . . 12
- 2.4. Mapa de entradas y salidas verificado . . . 12
- 2.5. Variables del proceso (PV, MV, DV) . . . 13
- 2.6. Lazo abierto con supervisión discreta . . . 13
- 2.7. Tags ISA-5.1 propuestos . . . 14
- 2.8. Discrepancias Kaizen: versión 2.0 . . . 15

3. Esquema eléctrico y cableado **16**
- 3.1. Plano histórico . . . 16
- 3.2. Etapa de potencia . . . 17
- 3.3. Etapa de mando . . . 17
- 3.4. Por qué los relés K1 y K2 . . . 18
- 3.5. Tensiones medidas . . . 18

4. Motores, drivers, transmisión y frecuencias **19**
- 4.1. Cómo se mueve un motor paso a paso . . . 19
- 4.2. Configuración de los drivers . . . 19
  - 4.2.1. Análisis por eje . . . 20
- 4.3. Transmisión y resolución . . . 20
- 4.4. Frecuencias, velocidades y tiempos . . . 21
- 4.5. Arranque sin rampa y pérdida de pasos . . . 21
- 4.6. Recomendaciones . . . 22

5. El PLC S7-200: lo necesario para leer los programas **23**
- 5.1. El ciclo del PLC . . . 23
- 5.2. Áreas de memoria . . . 23
- 5.3. Temporizadores TON . . . 23
- 5.4. Salidas de pulsos (PTO) . . . 24
- 5.5. Instrucciones que aparecen en los programas . . . 24
- 5.6. Cómo se lee un diagrama ladder . . . 25
- 5.7. Máquinas de pasos . . . 25

6. Programa original de secuencias **26**
- 6.1. Origen y estructura . . . 26
- 6.2. Memoria que usa . . . 26
- 6.3. Bloque principal OB1 . . . 27

6.4. Modo manual (SBR20) . . . 28
6.5. Homing (SBR0) . . . 29
6.6. Secuencias automáticas (SBR21) . . . 29
6.7. Limitaciones . . . 30

7. Programa del gemelo: `brazo_gemelo` 31
7.1. Por qué un programa nuevo . . . 31
7.2. Mapa de memoria . . . 31
7.3. Bloque principal OB1 . . . 32
7.4. Subrutinas de pulsos (SBR1 y SBR2) . . . 33
7.5. Modo manual (SBR3) . . . 33
7.6. Modo remoto (SBR4) . . . 34
7.6.1. Velocidad, watchdog y buzón (segmentos 1–19) . . . 34
7.6.2. Ejecución de un movimiento (segmentos 20–51) . . . 35
7.6.3. Homing (segmentos 52–64) . . . 36
7.7. Red de Petri del homing . . . 36
7.8. Comparación de los dos programas . . . 37

8. Modelo cinemático, jacobiano y modelo 3D 38
8.1. Levantamiento de medidas . . . 38
8.2. El mecanismo . . . 39
8.3. Cinemática directa: de los carros a la garra . . . 40
8.4. Cinemática inversa: de la garra a los carros . . . 41
8.5. Jacobiano y linealización . . . 41
8.6. Espacio de trabajo . . . 43
8.7. Modelo 3D . . . 43
8.8. Scripts del modelado . . . 44

9. Protocolo de comunicación PLC–PC: PPI 45
9.1. Qué es PPI y por qué se usó . . . 45
9.2. Capa física . . . 46
9.3. Tramas . . . 46
9.4. Intercambio y tiempos medidos . . . 47
9.5. Mensajes S7: ejemplos byte a byte . . . 47
9.5.1. Lectura del estado . . . 47
9.5.2. Escritura de una orden . . . 47
9.6. Lo aprendido en la planta . . . 48
9.7. Limitaciones . . . 49

10. Las aplicaciones 50
10.1. Panorama . . . 50
10.2. Arquitectura de tres capas . . . 50
10.3. La app del gemelo . . . 51
10.3.1. Módulos . . . 51
10.3.2. Hilos de ejecución . . . 51
10.3.3. Perfiles: la app se adapta al programa del PLC . . . 51
10.3.4. El estimador (programa original) . . . 52
10.3.5. El PLC simulado . . . 52
10.3.6. Modos y menú . . . 52
10.3.7. API HTTP . . . 53

10.3.8. Seguridad y control de acceso por roles . . . 53
10.4. La página publicada . . . 54

11. Base de datos **55**
11.1. Requerimientos . . . 55
11.2. Del modelo preliminar al modelo normalizado . . . 55
11.3. Modelo lógico . . . 56
11.4. Por qué está en tercera forma normal . . . 57
11.5. El script SQL . . . 57
11.6. De la app a las tablas . . . 57

12. Estructura del proyecto y control de versiones **59**
12.1. Repositorios . . . 59
12.2. Árbol del repositorio . . . 59
12.3. Archivos fuera del repositorio . . . 60
12.4. Flujo de trabajo con Git . . . 60
12.5. Convenciones . . . 61

13. Variables medidas y resultados **62**
13.1. Datos medidos o verificados en la planta . . . 62
13.2. Pruebas realizadas . . . 62
13.3. Validación del gemelo . . . 63

14. Cumplimiento de los objetivos de las guías **64**
14.1. Siguientes pasos . . . 64

A. Programa original de secuencias (AWL completo) **66**

B. Programa `brazo_gemelo` (AWL completo) **75**

C. Script de la base de datos **84**

Índice de figuras

1.1. Fases del semestre según la guía 1. 7

2.1. El brazo XK-JS1-1 en el laboratorio. 9
2.2. Componentes de la planta (fotos de `global/fotos-planta/`). 11
2.3. Diagrama de bloques de la planta. 12
2.4. Diagrama de bloques fenomenológico del equipo (`diagramas/`): no hay realimentación continua de posición. 13

3.1. Plano eléctrico heredado (`planos/electrico/brazo_electrico_historico.dwg`). El informe 1 encontró pines sin graficar, DIR+ de X e Y sin conectar y el trazo de la garra equivocado (sección 2.8). 16
3.2. Distribución de energía. 17
3.3. Esquema funcional de mando. Q0.1 llega al driver X solo con K1 cerrado y al driver Y solo con K2 cerrado. 17

4.1. Cadena de mando de un eje. 19
4.2. Carros X e Y: 200 pulsos por milímetro. 20
4.3. Tren de pulsos del PTO: ancho al 50 %, ciclo T y cantidad N. 21

5.1. Ciclo de barrido. Las peticiones de la PC se atienden en la etapa de comunicación, entre ciclos. 23
5.2. Byte de control `16#85` que usan los dos programas: PTO de un solo segmento, base de tiempo en µs, con actualización de ciclo y cantidad. 24

6.1. Estructura del programa original. 26
6.2. OB1, segmento 1 (parte): inicio por I0.7 *o* V104.0, sin emergencia. 28
6.3. OB1, segmento 9: en automático, el carro X se detiene si llega al final del lado hacia el que avanza. 28
6.4. SBR20, segmento 5: jog de X adelante. 28
6.5. Máquina de pasos del homing (VW322). 29
6.6. SBR0, segmento 3: arranca el tren hacia atrás y lo corta al llegar a I0.1. 29

7.1. Estructura de `brazo_gemelo`. SBR1 y SBR2 son los únicos lugares donde se arrancan trenes de pulsos: así toda la cuenta pasa por un solo sitio. 31
7.2. OB1, segmento 4: el final de carrera corta el tren y fija la posición exacta. 33
7.3. OB1, segmento 9: suma del tramo terminado a la posición de X y/o de Y. 33
7.4. SBR4, segmento 8: una orden solo se toma cuando cambia el número de secuencia. 34
7.5. SBR4, segmentos 5 y 7: watchdog. 34
7.6. Validación de una orden (segmentos 9–19). 35
7.7. Máquina de pasos de un movimiento. 36
7.8. Red de Petri del homing de `brazo_gemelo`. Es una red de estado: un solo token recorre la secuencia, lo que garantiza que dos ejes nunca se buscan a la vez. 36

8.1. Esquema de medidas en vista lateral (`levantamiento/esquema_medidas_lateral.png`). 38
8.2. Esquema cinemático del brazo (`modelo-3d/informe/figuras/`). 39
8.3. Esquema simplificado: lazo A–B–C y brazo A–E. 40

8.4. Relación entre el carro Y y el ángulo del brazo: de −32,6ř ($s_Y = 0$) a +47,0ř ($s_Y =$ 170 mm), no lineal. 41
8.5. Espacio de trabajo: una franja curva en el plano vertical que gira con la base; alcance radial de 411 a 677 mm. 43
8.6. Modelo 3D del brazo. 43

9.1. Capas del protocolo. 45
9.2. Conector, cable y señal diferencial RS-485 (esquemática). 46
9.3. Formato de cada carácter: 8 bits, paridad par, 1 bit de parada (8E1). 46
9.4. Formatos de trama PPI. 46
9.5. Lectura de 40 bytes a 9,6 kbit/s con los tiempos medidos en la planta. 47
9.6. Petición «leer 40 bytes desde VB1120». 47
9.7. Tiempo de una lectura antes y después del silencio entre tramas (naranja: petición; azul: sondeo; verde: respuesta). 48

10.1. Arquitectura de tres capas del gemelo. La base de datos (línea punteada) está diseñada pero no conectada. 50
10.2. Hilos de la app. Las páginas nunca esperan al PLC: leen el último estado guardado. 51
10.3. Menú de la app. 52
10.4. Publicación automática de la página en cada `push` a `main`. 54
10.5. Página del gemelo dentro de la app (modo simulación). 54

11.1. Modelo E-R preliminar del equipo (`base-de-datos/modelo/`). 55
11.2. Modelo lógico normalizado (PostgreSQL). PK: clave primaria; FK: clave foránea; UQ: único; 1–N: cardinalidad. 56

12.1. Ramas de trabajo que se integran a `main` con revisión. 60

Índice de tablas

4.1. Corriente de trabajo (SW1–SW3). 20
4.2. Micropasos (SW5–SW8). 20

Capítulo 1

Objetivos y cómo usar este manual

Este manual reúne, en un solo documento, todo el conocimiento técnico del brazo robótico XK-JS1-1 del laboratorio y del gemelo digital desarrollado sobre él durante la gestión II/2026: el hardware y su cableado, la configuración de los drivers y las frecuencias de trabajo, los dos programas ladder del PLC explicados red por red, el modelo cinemático con su jacobiano, el protocolo de comunicación entre la computadora y el PLC, las aplicaciones de software, la base de datos y la estructura del repositorio.

Está escrito para que un estudiante que llegue al laboratorio sin conocer el brazo pueda entenderlo por completo, reproducir los resultados y continuar el trabajo.

1.1. Marco: el laboratorio integrado

Las guías de laboratorio del semestre (carpeta `semestres/2026_II/guias/` del repositorio) organizan el trabajo de tres asignaturas sobre las mismas plantas, siguiendo el Ciclo de Vida del Desarrollo de Sistemas (SDLC) y la mejora continua (Kaizen):

Asignatura	Responsabilidad sobre el brazo
ETN-902 Control II	caracterización, modelado matemático, linealización, identificación de variables y lazos
ETN-1034 Aplicación de Técnicas	programación del PLC (PTO), cableado de drivers y sensores, control avanzado, comunicaciones industriales
ETN-1000 Bases de Datos	modelo de datos de telemetría, interfaces (HMI y web), conectividad entre la planta y el servidor

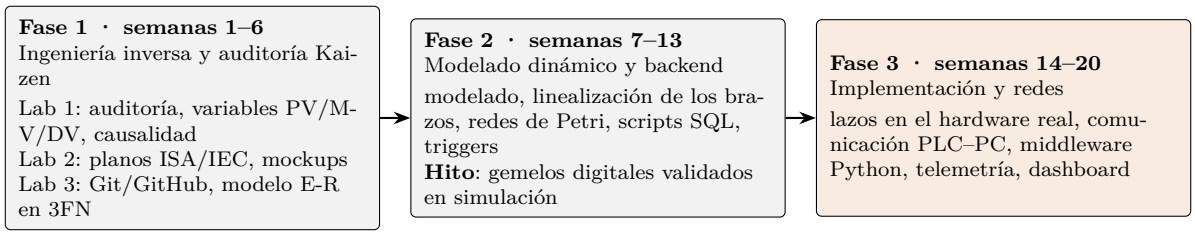


Figura 1.1: Fases del semestre según la guía 1.

1.2. Qué objetivo cubre cada capítulo

Objetivo de las guías	Dónde se cubre
Identificar y clasificar variables PV, MV, DV; ruptura de causalidad (Lab 1)	Capítulo 2: variables, tags ISA, lazo abierto con supervisión discreta
Ficha técnica del hardware, ingeniería inversa (Lab 1)	Capítulos 2 y 4
Planos eléctricos, potencia y mando, bornes al PLC (Lab 2)	Capítulo 3
Programación del PLC: PTO, temporizadores, secuencias (ETN-1034)	Capítulos 5, 6 y 7

Objetivo de las guías	Dónde se cubre
Redes de Petri para comportamiento secuencial (Fase 2)	Sección 7.7
Modelado y linealización de los brazos (Fase 2)	Capítulo 8: cinemática, jacobiano, linealización
Gemelo digital validado (hito de la Fase 2)	Capítulos 8, 10 y 13
Comunicación PLC–PC, tramas binarias (Fase 3)	Capítulo 9: protocolo PPI byte a byte
Middleware en Python, arquitectura de 3 capas (Lab 1, Fase 3)	Capítulo 10
Modelo E-R normalizado en 3FN, scripts SQL, triggers (Lab 3, Fase 2)	Capítulo 11
Control de acceso por roles (Fase 3)	Secciones 10.3.8 y 11.3
Estructura del repositorio, Git y GitHub (Lab 3)	Capítulo 12
Variables medidas y resultados	Capítulo 13
Balance de cumplimiento	Capítulo 14

1.3. Convenciones

- **SUPUESTO**: dato que no se midió en la planta y se tomó de una estimación o del fabricante. Debe verificarse.
- **Ejes**: la *base* gira un ángulo θ y su motor se llama *eje Z* (así lo nombran el informe 1 y la app); los *carros X e Y* son lineales. La garra es el cuarto grado de libertad (binario).
- Direcciones del PLC en notación Siemens: `I0.5` entrada, `Q0.1` salida, `M5.0` marca, `VB1100` byte de la memoria V, `VW` palabra (2 bytes), `VD` doble palabra (4 bytes), `SM` memoria especial, `T` temporizador.
- Los fragmentos de código están copiados de los archivos reales del repositorio; los programas del PLC completos están en los apéndices.

1.4. Documentos relacionados

Documento	Contenido
`manual_linux.pdf`, `manual_windows.pdf`	instalación y uso de la app del gemelo
`arquitectura_gemelo_digital.pdf`	por qué se diseñó así: páginas, puertas de acceso
`guia_completa_brazo.pdf`	versión resumida de este manual
`informes/informe_1_auditoria_y_requerimientos.pdf`	auditoría del equipo (Lab 1)
`modelo-3d/informe/informe_cinematica_modelo_3d.pdf`	modelo cinemático y 3D
`global/datasheets/`	hojas de datos de PLC, drivers, sensores y relés

Capítulo 2

La planta

2.1. Qué es y qué hace

El XK-JS1-1 es un brazo didáctico de **4 grados de libertad** que toma piezas pequeñas y las traslada. Tres movimientos son continuos y accionados por motores paso a paso; el cuarto es la garra, de dos estados.

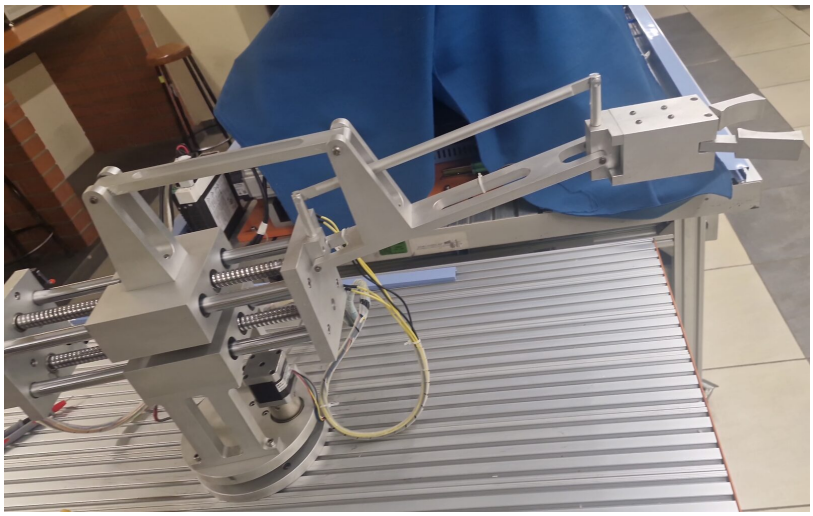


Figura 2.1: El brazo XK-JS1-1 en el laboratorio.

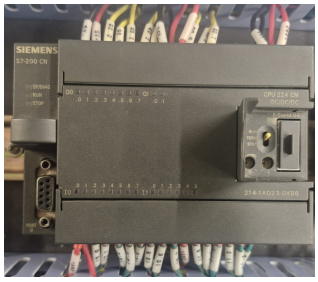
Grado de libertad	**Movimiento**	**Accionamiento**
Base (θ, motor Z)	giro alrededor del eje vertical	NEMA 17 + driver 2M530 + engranaje
Carro X (s_X)	desplaza todo el brazo: cambia el alcance	NEMA 23 + driver 2M542 + husillo inferior
Carro Y (s_Y)	mueve un bloque que, por una biela, levanta el brazo: cambia la altura	NEMA 23 + driver 2M542 + husillo superior
Garra	abre o cierra las mordazas	solenoide de 24 V por el relé K3 (salida Q0.7)

2.2. Ficha técnica del hardware

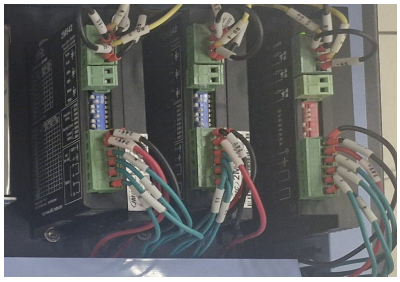
Datos de la auditoría del informe 1, de las hojas de datos de `global/datasheets/` y de las verificaciones hechas en la planta en octubre de 2026.

Componente	**Modelo**	**Datos clave**
PLC	Siemens S7-200 CPU 224 CN (6ES7 214-1AD23)	14 entradas y 10 salidas digitales de 24 V DC (transistor); 2 salidas de pulsos rápidas (Q0.0, Q0.1) hasta 20 kHz; un puerto RS-485 (puerto 0) con protocolo PPI

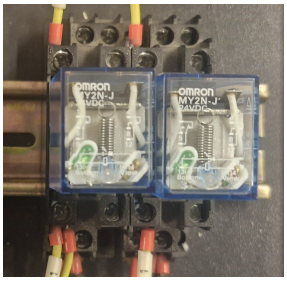
Componente	Modelo	Datos clave
Drivers X e Y	JMC 2M542-N (compatible DM542)	20–50 V DC, 1,0–4,2 A pico; entradas PUL, DIR y ENA optoacopladas; micropasos por interruptores DIP
Driver base	JMC 2M530	18–50 V DC, 0,5–3,0 A (informe 1); tabla DIP por confirmar en su placa
Motores X e Y	NEMA 23, 57J1854-828	1,8° por paso (200 pasos/vuelta), 2,8 A por fase
Motor base	NEMA 17, 42J1848-604	1,8° por paso, 1,2 A por fase
Husillos X e Y	de bolas, tipo SFU1605 (SUPUESTO)	paso 5 mm/vuelta (SUPUESTO)
Relés K1, K2, K3	Omron MY2N-J (24 V DC)	2 contactos conmutados (DPDT), 5 A; K1 y K2 reparten el tren de pulsos, K3 acciona el solenoide
Finales de carrera X e Y	microinterruptores (informe 1: M102-011; hoja de datos: V-156-1C25)	4 unidades, contacto NA/NC
Sensor de la base	LJ12A3-4-Z/BX	inductivo NPN normalmente abierto, 6–36 V DC, alcance 4 mm, 100 Hz
Garra	solenoide de 24 V DC	todo o nada; Q0.7 = 1 cerrada
Fuente	conmutada 24 V / 5 V DC	alimenta PLC, sensores y la lógica de los drivers
Protección	disyuntor diferencial DZ47LE-63	16 A, 30 mA, curva C
Panel	botonera	pulsadores Start, Stop, Reset, S1–S7 y selector Manual/Automático



(a) PLC S7-200 CPU 224



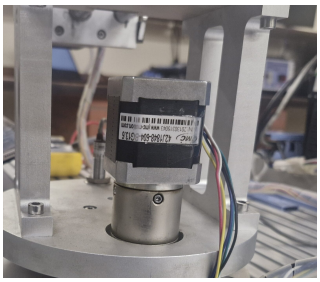
(b) drivers 2M542 y 2M530



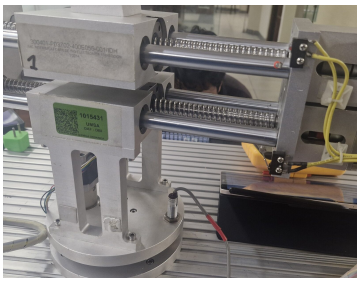
(c) relés Omron MY2N-J



(d) motores NEMA 23 (X, Y)



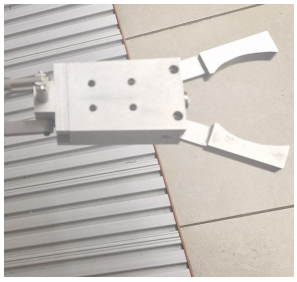
(e) motor NEMA 17 (base)



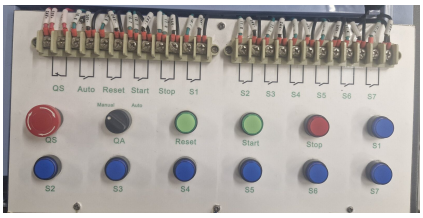
(f) sensor inductivo de la base



(g) finales de carrera X e Y



(h) garra



(i) botonera

Figura 2.2: Componentes de la planta (fotos de `global/fotos-planta/`).

2.3. Diagrama de bloques

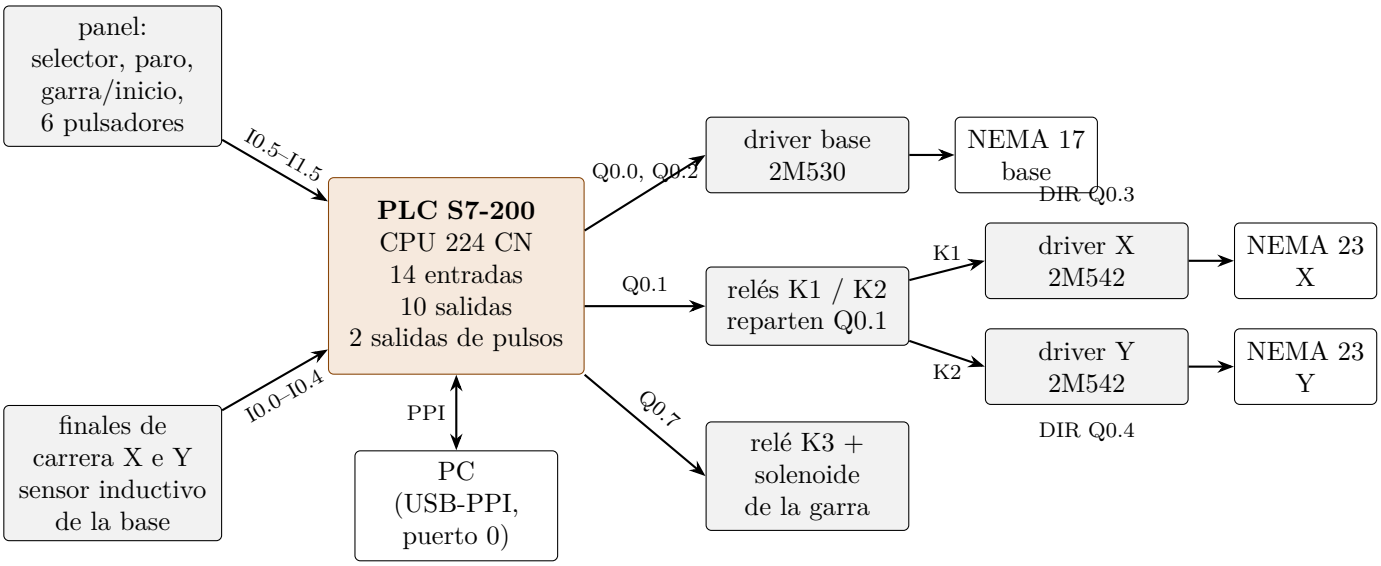


Figura 2.3: Diagrama de bloques de la planta.

2.4. Mapa de entradas y salidas verificado

Cada sensor se accionó a mano mientras un monitor por PPI registraba los cambios (26/10/2026). Esto corrigió el mapa del informe 1 (sección 2.8).

Entrada	Función	Salida	Función
I0.0	sensor inductivo de la base ($\theta = 0$)	Q0.0	pulsos de la base (PTO0)
I0.1	final X atrás (origen)	Q0.1	pulsos de X e Y (PTO1), compartidos
I0.2	final X adelante	Q0.2	DIR base (1 = hacia el inductivo)
I0.3	final Y abajo (origen)	Q0.3	DIR X (0 = adelante)
I0.4	final Y arriba	Q0.4	DIR Y (0 = arriba)
I0.5	selector: 0 manual, 1 automático/remoto	Q0.5	relé K1: Q0.1 al driver X
I0.6	paro	Q0.6	relé K2: Q0.1 al driver Y
I0.7	garra / inicio	Q0.7	relé K3: solenoide (1 = cerrada)
I1.0	jog base + / −		
/			
I1.1			
I1.2	jog X adelante / atrás		
/			
I1.3			
I1.4	jog Y arriba / abajo		
/			
I1.5			

La correspondencia entre los rótulos de la botonera (S1–S7, Start, Stop) y las entradas I0.6–I1.5 no se registró pulsador por pulsador (SUPUESTO: Stop = I0.6, Start = I0.7).

2.5. Variables del proceso (PV, MV, DV)

El brazo no tiene sensores de posición continua (encoders): la posición de cada eje se *infiere* contando pulsos. Clasificación corregida respecto del informe 1:

Tipo	Variable	Señal física y rango
MV	tren de pulsos de la base (N_θ, f_θ)	Q0.0 (PTO0), 0–20 kHz; se usa 2,5–4 kHz
MV	tren de pulsos de X e Y (N_{XY}, f_{XY})	Q0.1 (PTO1), uno solo para los dos carros
MV	selección de carro	Q0.5 (K1, carro X), Q0.6 (K2, carro Y)
MV	dirección de cada eje	Q0.2 (base), Q0.3 (X), Q0.4 (Y)
MV	garra	Q0.7, 24 V al relé K3
PV	posición de la base θ	sin sensor continuo; $\theta = n_\theta/190{,}3$ [°]
PV	posición de los carros s_X, s_Y	sin sensor continuo; $s = n/200$ [mm], 0–170 mm
PV	posición de la garra (X, Y, Z)	calculada con la cinemática directa (capítulo 8)
PV discreta	referencias de posición	I0.0 (inductivo), I0.1–I0.4 (finales): solo dicen si se alcanzó un punto
DV	fricción en guías y husillos	no medible eléctricamente
DV	carga en la garra	peso de la pieza: cambia el par que necesita el carro Y
DV	pérdida de pasos	arranques sin rampa o sobrecarga: la posición contada deja de ser la real
DV	holguras y deformaciones	error de posición de la garra no detectable

2.6. Lazo abierto con supervisión discreta

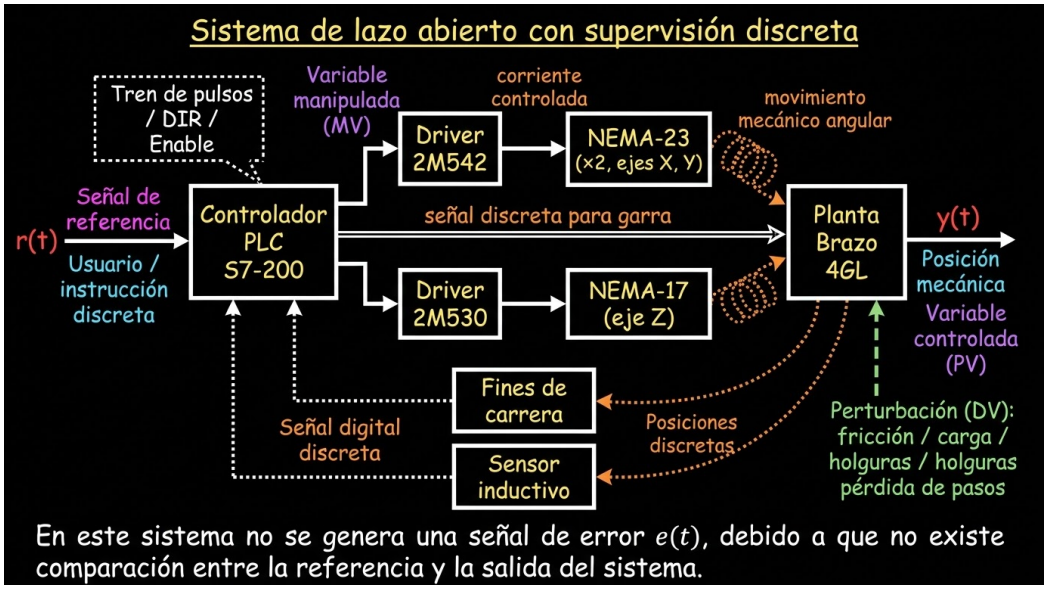


Figura 2.4: Diagrama de bloques fenomenológico del equipo (`diagramas/`): no hay realimentación continua de posición.

El PLC estima la posición como

$$q_{est} = q_0 + N_p\,\Delta q,$$

donde q_0 es la posición de referencia (fijada por el homing), N_p los pulsos enviados y Δq el avance por pulso (0,005 mm en los carros). Si el motor pierde pasos, $q_{real} \neq q_{est}$ y el PLC no lo sabe: la acción de control actúa sobre la planta ($u \to y$) pero la salida no vuelve al controlador ($y \not\to u$). Es la *ruptura de causalidad* del informe 1. Los finales de carrera solo corrigen la estimación en puntos aislados. El programa del gemelo aprovecha esto: cada vez que se pisa un final, fija la posición exacta (capítulo 7).

2.7. Tags ISA-5.1 propuestos

Para que los planos, el programa y la base de datos usen los mismos nombres (pedido de las guías 2 y 3), se propone esta identificación. La letra Z de ISA-5.1 designa *posición*; S = interruptor; L/H = bajo/alto; Y = función de cálculo o relé; H = manual; I = indicación; X = sin clasificar.

Tag	Señal	Descripción
Lazo 100: general		
`HS-100`	`I0.5`	selector manual / automático
`HS-100P`	`I0.6`	pulsador de paro
`ZY-100`	`Q0.1`	tren de pulsos compartido X/Y (PTO1)
Lazo 101: base (θ)		
`ZS-101`	`I0.0`	interruptor de posición inductivo ($\theta = 0$)
`HS-101A / B`	`I1.0 / I1.1`	jog base + / −
`ZY-101`	`Q0.0`	tren de pulsos de la base (PTO0)
`ZY-101D`	`Q0.2`	dirección de la base
`ZI-101`	`VD1124`	posición de la base (pulsos contados)
Lazo 102: carro X		
`ZSL-102 / ZSH-102`	`I0.1 / I0.2`	final de carrera bajo (atrás) / alto (adelante)
`HS-102A / B`	`I1.2 / I1.3`	jog X adelante / atrás
`ZY-102D`	`Q0.3`	dirección X
`ZY-102K`	`Q0.5`	relé K1
`ZI-102`	`VD1128`	posición X
Lazo 103: carro Y		
`ZSL-103 / ZSH-103`	`I0.3 / I0.4`	final de carrera bajo (abajo) / alto (arriba)
`HS-103A / B`	`I1.4 / I1.5`	jog Y arriba / abajo
`ZY-103D`	`Q0.4`	dirección Y
`ZY-103K`	`Q0.6`	relé K2
`ZI-103`	`VD1132`	posición Y
Lazo 104: garra		
`HS-104`	`I0.7`	garra / inicio
`XY-104`	`Q0.7`	relé K3 del solenoide

Los mismos tags están en la base de datos (tablas `punto_io` y `variable`, capítulo 11).

2.8. Discrepancias Kaizen: versión 2.0

El informe 1 registró discrepancias entre el plano histórico y la planta. La puesta en marcha del gemelo permitió verificarlas con el PLC en marcha y agregar otras:

N°	Elemento	Antes	Verificado (oct. 2026)
1	finales de carrera	I0.0–I0.3 (informe 1, con desfase de 1 bit)	**I0.1**–**I0.4**; I0.0 es el inductivo de la base
2	tren de pulsos de X e Y	Q0.5 (X) y Q0.6 (Y)	**Q0.1 compartido**; Q0.5 y Q0.6 son los relés K1 y K2 que lo reparten
3	final de homing del carro Y	I0.4 (tabla anterior del proyecto)	**I0.3** (Y abajo)
4	sentido de X	no documentado	Q0.3 = 0 adelante (se separa de I0.1)
5	dirección PPI del PLC	7 (proyecto de Miguel)	**6** a 19,2 kbit/s con el programa original; **2** a 9,6 kbit/s tras cargar `brazo_gemelo` con su bloque de sistema
6	reducción de la base	5:1 (SUPUESTO del modelo)	**190,3 pulsos/°** medidos: unos 68,5 vueltas del motor por vuelta de la base (con 1000 pulsos/vuelta, SUPUESTO)
7	driver de la base	mismo modelo que X/Y	2M530 (otra tabla de corriente): revisar su configuración (capítulo 4)
8	pines DIR+ de X e Y	sin conexión en el plano	conectados (informe 1); el plano debe corregirse
9	garra	trazo incorrecto en el plano	salida Q0.7 → relé K3 → solenoide

Pendiente. Pasar estas correcciones al plano eléctrico y al P&ID en AutoCAD Electrical (versión 2.0 oficial, guía 2). Los archivos actuales están en `semestres/2026_II/planos/`.

Capítulo 3

Esquema eléctrico y cableado

3.1. Plano histórico

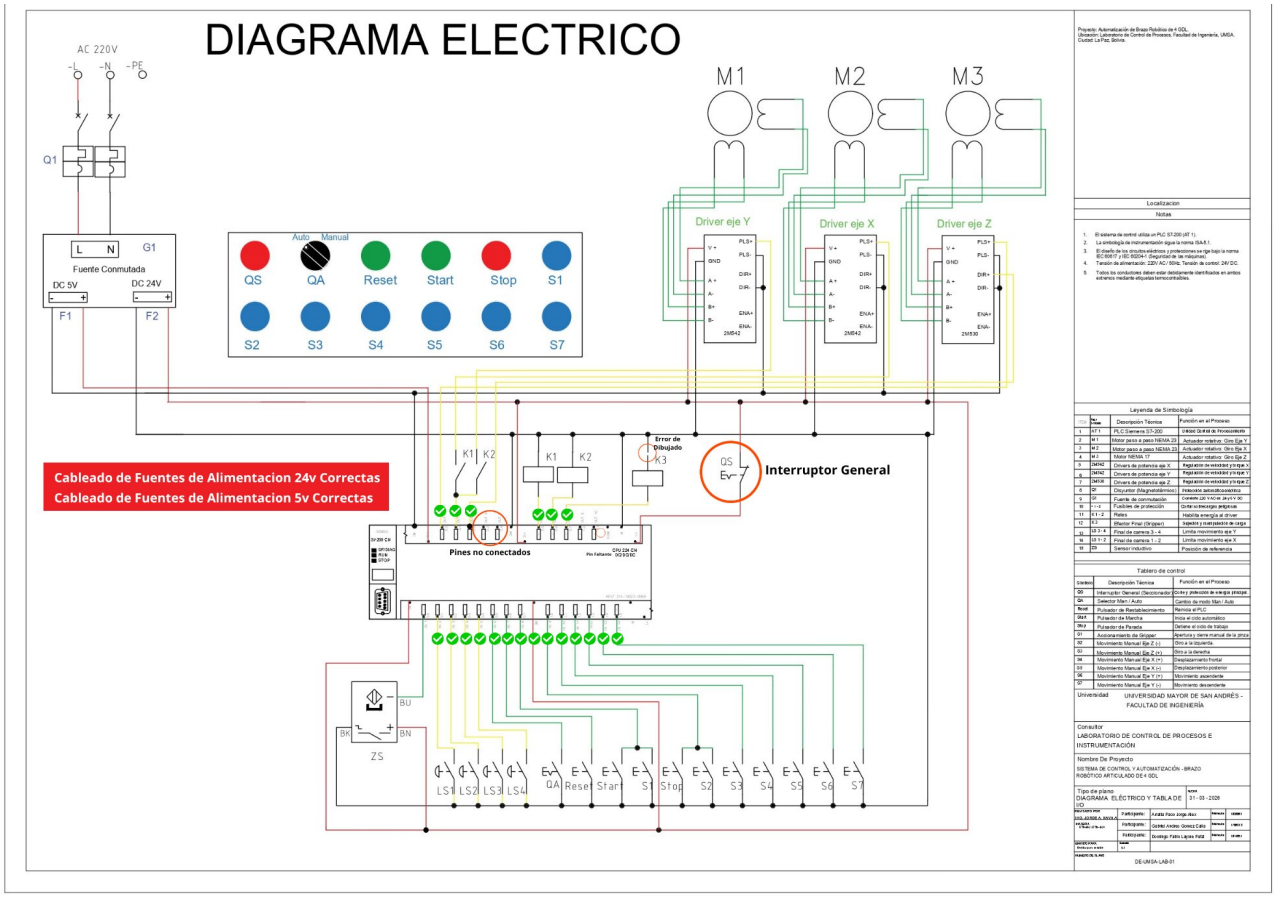


Figura 3.1: Plano eléctrico heredado (`planos/electrico/brazo_electrico_historico.dwg`). El informe 1 encontró pines sin graficar, DIR+ de X e Y sin conectar y el trazo de la garra equivocado (sección 2.8).

3.2. Etapa de potencia

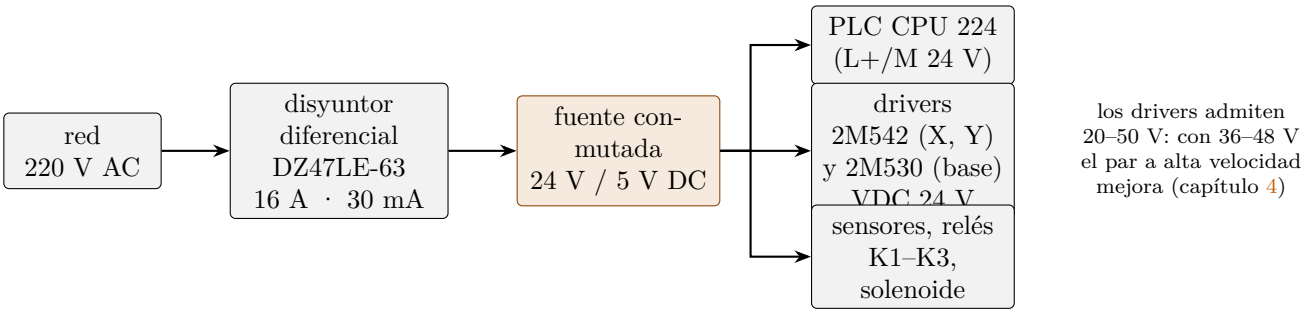


Figura 3.2: Distribución de energía.

3.3. Etapa de mando

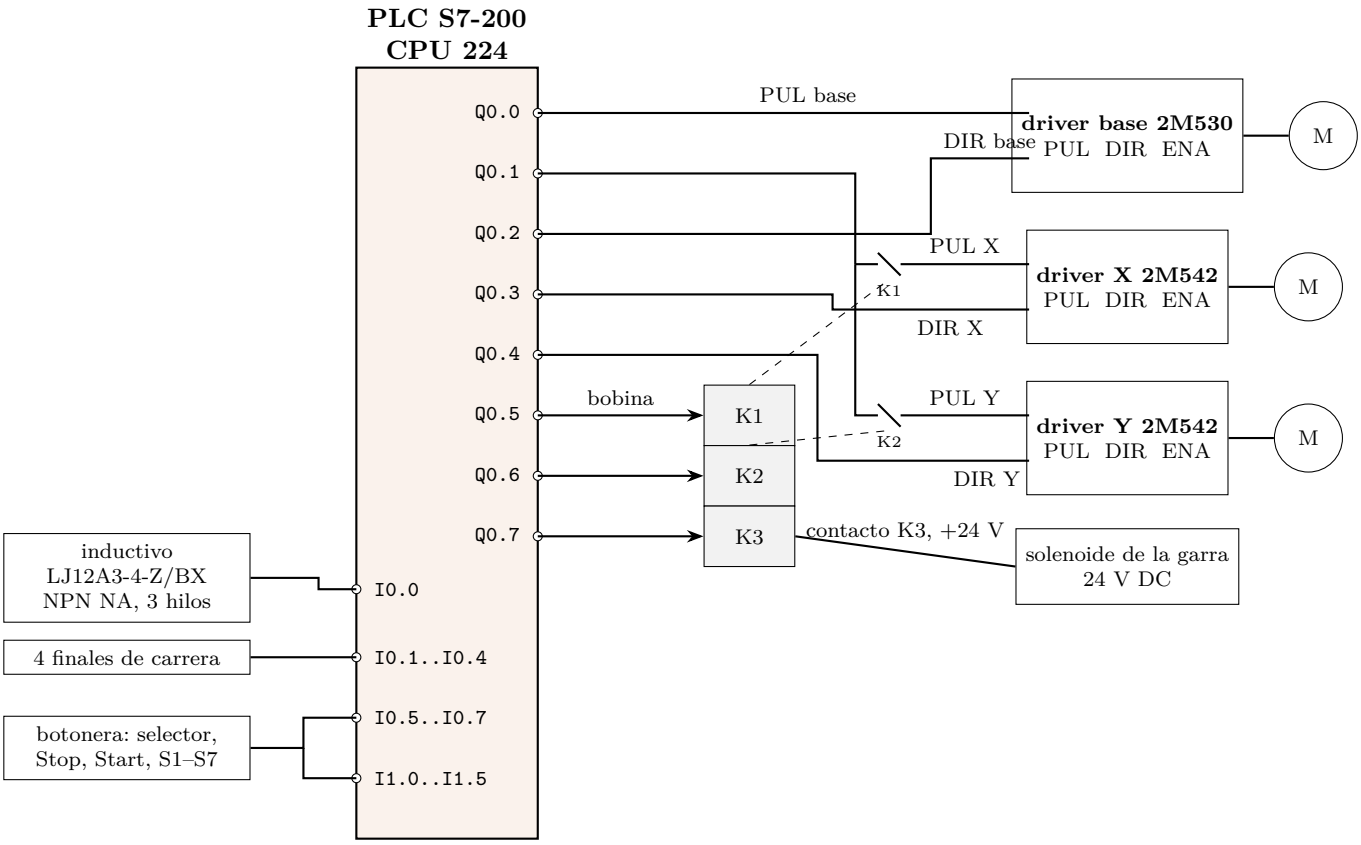


Figura 3.3: Esquema funcional de mando. Q0.1 llega al driver X solo con K1 cerrado y al driver Y solo con K2 cerrado.

Este esquema es funcional: muestra qué señal va a qué componente, verificado con el PLC en marcha. Los bornes exactos, los comunes (1M, 2M, 1L+), el nivel de las entradas PUL/DIR de los drivers (entradas optoacopladas de 5 V, alimentadas desde el riel de 5 V o con resistencias en serie) y la entrada ENA deben tomarse del plano versión 2.0 cuando se termine (guía 2).

3.4. Por qué los relés K1 y K2

La CPU 224 tiene solo dos generadores de pulsos rápidos, en Q0.0 y Q0.1. Con tres motores, quien diseñó la planta dedicó Q0.0 a la base y repartió Q0.1 entre los dos carros mediante los contactos de K1 y K2. Consecuencias de diseño:

- X e Y pueden moverse a la vez, pero recibiendo exactamente los mismos pulsos, a la misma frecuencia.
- Un movimiento con recorridos distintos se divide en un tramo común (K1 y K2 cerrados) y un resto (solo el relé del eje más largo).
- Un relé tarda unos milisegundos en cerrar: los programas esperan 30–300 ms entre cerrar el relé y dar pulsos, para no perder los primeros.
- Abrir un relé con el tren en marcha cortaría pulsos a mitad del tramo: los programas solo conmutan los relés con el PTO detenido (SM76.7 = 1).

3.5. Tensiones medidas

Mediciones pasivas con multímetro del informe 1 (Lab 1):

Punto	**Accionado**	**En reposo**
entradas de la botonera (I0.4–I1.5)	21,6 V	23,1 V
salidas de pulsos (multímetro, valor medio)	0,0–1,2 V	0,1–0,2 V
salidas de dirección	0,1–5,1 V	0,1–5,1 V
Q0.7 (garra)	23,2 V	0,2 V
rieles del PLC (1M–1L+, 2M–2L+)	5,1–23,4 V	5,2–23,7 V

Un multímetro muestra el valor medio de un tren de pulsos, no su forma: para ver la frecuencia hace falta un osciloscopio o un contador (por ejemplo, el contador rápido HSC del propio PLC).

Capítulo 4

Motores, drivers, transmisión y frecuencias

4.1. Cómo se mueve un motor paso a paso

Un motor paso a paso híbrido de 1,8° tiene 200 posiciones estables por vuelta. El driver alimenta sus dos bobinas (A y B) con corrientes que van girando el campo magnético; con *micropasos*, divide cada paso en partes más pequeñas haciendo las corrientes casi senoidales, lo que da un movimiento más suave.

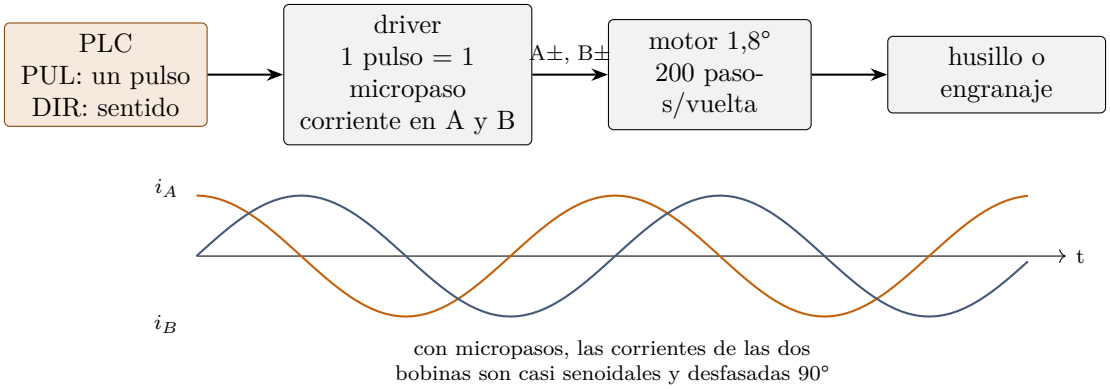
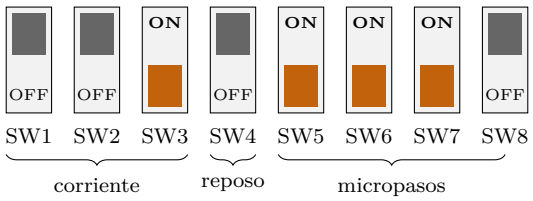


Figura 4.1: Cadena de mando de un eje.

4.2. Configuración de los drivers

Los drivers JMC 2M542-N son compatibles con el Leadshine DM542, cuya tabla está en `global/datasheets/driv`
Los tres drivers del tablero tienen los interruptores en la misma posición (foto del 26/10/2026):



SW1	SW2	SW3	pico	RMS
ON	ON	ON	1,00 A	0,71 A
OFF	ON	ON	1,46 A	1,04 A
ON	OFF	ON	1,91 A	1,36 A
OFF	OFF	ON	**2,37 A**	**1,69 A**
ON	ON	OFF	2,84 A	2,03 A
OFF	ON	OFF	3,31 A	2,36 A
ON	OFF	OFF	3,76 A	2,69 A
OFF	OFF	OFF	4,20 A	3,00 A

Tabla 4.1: Corriente de trabajo (SW1–SW3).

pulsos/vuelta	SW5	SW6	SW7	SW8
400	OFF	ON	ON	ON
800	ON	OFF	ON	ON
1600	OFF	OFF	ON	ON
3200	ON	ON	OFF	ON
6400	OFF	ON	OFF	ON
12800	ON	OFF	OFF	ON
25600	OFF	OFF	OFF	ON
1000	ON	ON	ON	OFF
2000	OFF	ON	ON	OFF
4000	ON	OFF	ON	OFF
5000	OFF	OFF	ON	OFF
8000	ON	ON	OFF	OFF
10000	OFF	ON	OFF	OFF
20000	ON	OFF	OFF	OFF
25000	OFF	OFF	OFF	OFF

Tabla 4.2: Micropasos (SW5–SW8).

- **SW4 = OFF**: un segundo después del último pulso, la corriente baja a la mitad (el calor baja al 25 %, porque $P = I^2R$). Con SW4 = ON se mantiene toda la corriente.
- **X e Y deben tener siempre los mismos micropasos**, porque reciben el mismo tren de pulsos: si no, avanzarían distinto con los mismos pulsos.

4.2.1. Análisis por eje

Eje	Motor	Corriente ajustada	Comentario
X, Y	NEMA 23, 2,8 A/fase	1,69 A RMS (2,37 A pico)	60 % de la nominal: el motor trabaja frío pero con menos par (el par es casi proporcional a la corriente)
Base	NEMA 17, 1,2 A/fase	desconocida	el 2M530 tiene otra tabla (0,5–3,0 A). Si la posición OFF OFF ON diera más de 1,2 A RMS, el motor se calentaría: **leer la placa del driver y verificar**

4.3. Transmisión y resolución

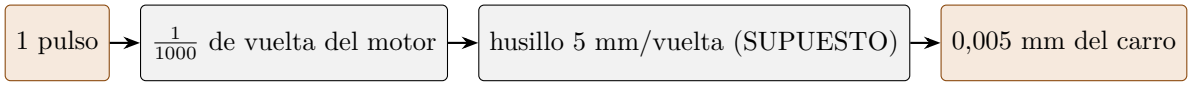


Figura 4.2: Carros X e Y: 200 pulsos por milímetro.

- **Carros**: $k_{XY} = \dfrac{1000 \text{ pulsos/vuelta}}{5 \text{ mm/vuelta}} = 200$ pulsos/mm; resolución 5 µm; recorrido de 170 mm = 34 000 pulsos.
- **Base**: calibrada en la planta midiendo un giro con transportador: $k_\theta = 190{,}3$ pulsos/grado, o sea 68 508 pulsos por vuelta de la base. Si el driver de la base también da 1000 pulsos por vuelta (SUPUESTO), la reducción mecánica es de unas **68,5 vueltas del motor por vuelta de la base**. Resolución: 0,0053° por pulso.

4.4. Frecuencias, velocidades y tiempos

La velocidad de cada eje sale directamente de la frecuencia del tren de pulsos f (o de su ciclo $T = 1/f$, que es lo que se programa en el PLC):

$$v_{carro} = \frac{f}{k_{XY}}\ [\text{mm/s}], \qquad \omega_{base} = \frac{f}{k_\theta}\ [°/\text{s}], \qquad n_{motor} = \frac{60\,f}{1000}\ [\text{rpm}].$$

Dónde se usa	ciclo	f	motor	eje
jog X/Y, homing X/Y, secuencias, movimientos del gemelo (por defecto)	`250 µs`	4 kHz	240 rpm	20 mm/s
jog de la base (programa del gemelo), homing de la base, movimientos de la base (por defecto)	`400 µs`	2,5 kHz	150 rpm	13,1 °/s
jog y secuencias de la base (programa original)	`250 µs`	4 kHz	240 rpm	21,0 °/s
mínimo admitido por el programa del gemelo (X/Y)	`2000 µs`	500 Hz	30 rpm	2,5 mm/s
máximo admitido por el programa del gemelo (X/Y)	`200 µs`	5 kHz	300 rpm	25 mm/s
límite del hardware (PTO de la CPU 224)	`50 µs`	20 kHz	1200 rpm	100 mm/s

Tiempos típicos a las frecuencias por defecto: recorrer los 170 mm de un carro toma $170/20 = 8{,}5$ s; girar la base 90° toma $90 \times 190{,}3/2500 = 6{,}9$ s.

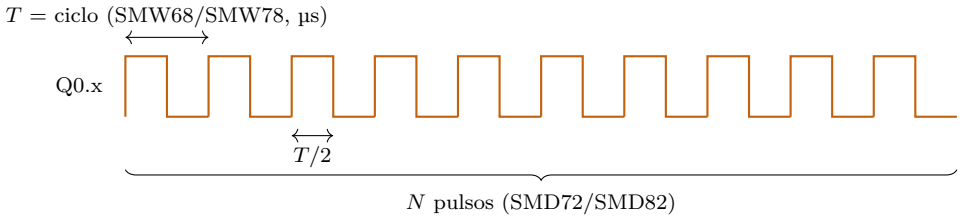


Figura 4.3: Tren de pulsos del PTO: ancho al 50 %, ciclo T y cantidad N.

4.5. Arranque sin rampa y pérdida de pasos

Un motor paso a paso solo puede arrancar *de golpe* hasta una frecuencia límite (de arranque o *pull-in*), que baja con la carga y la inercia. Por encima de ella, el rotor no alcanza a seguir al campo y pierde pasos: el eje queda atrás de la cuenta del PLC.

Los dos programas del brazo arrancan los trenes a frecuencia constante, sin rampa. Por eso:

- se trabaja a 2,5–4 kHz, frecuencias que se probaron sin pérdidas visibles;
- la base del programa del gemelo usa 2,5 kHz (en lugar de 4 kHz) porque arranca más suave;
- la app limita la velocidad a 500–5000 Hz y recomienda no pasar de 4000 Hz.

4.6. Recomendaciones

1. **Corriente de X e Y**: subir a 2,84 A pico (ON ON OFF) para aprovechar más el par del NEMA 23 de 2,8 A, vigilando la temperatura.
2. **SW4 del carro Y**: el husillo de bolas es reversible y el peso del brazo empuja el carro Y. Si el brazo baja solo al quedar quieto, poner SW4 = ON (retención con toda la corriente).
3. **Rampas**: implementar perfiles de varios segmentos en el PTO (arranque a 1–1,5 kHz, máximo 10–15 kHz, rampa de 0,5–1 s). Así se podría trabajar más rápido sin perder pasos.
4. **Tensión**: los drivers admiten hasta 50 V; con 36–48 V el par a alta velocidad mejora mucho.
5. **Base**: verificar el driver 2M530 y el motor de 1,2 A.

Capítulo 5

El PLC S7-200: lo necesario para leer los programas

5.1. El ciclo del PLC

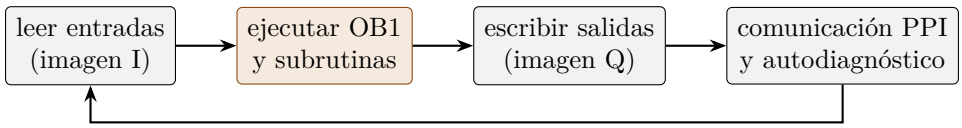


Figura 5.1: Ciclo de barrido. Las peticiones de la PC se atienden en la etapa de comunicación, entre ciclos.

El programa se ejecuta de arriba abajo en cada ciclo (unos milisegundos). Las salidas de pulsos (PTO) son la excepción: las genera un circuito de hardware que sigue funcionando aunque el ciclo siga adelante; el programa solo las configura, las arranca y consulta si terminaron.

5.2. Áreas de memoria

Área	Nombre	Uso en el brazo
`I`	imagen de entradas	sensores y botonera (I0.0–I1.5)
`Q`	imagen de salidas	pulsos, direcciones, relés (Q0.0–Q0.7)
`M`	marcas	banderas internas del programa original (M0.0–M10.0)
`V`	memoria de variables	datos y buzón con la PC (VB100, VB104, VB1100–VB1205)
`SM`	memoria especial	configuración y estado del PTO (SMB66–SMD82), SM0.0 (siempre 1), SM0.1 (primer ciclo)
`T`	temporizadores	retardos y vigilancia de tiempos

Tamaños: `VB` = byte, `VW` = palabra de 2 bytes (`VW1102` ocupa VB1102 y VB1103), `VD` = doble palabra de 4 bytes. El S7-200 guarda los números con el byte más significativo primero (*big-endian*): la app debe leerlos igual.

5.3. Temporizadores TON

La resolución depende del número del temporizador. Esto es clave para leer los programas: `TON T35, +3` son 30 ms, pero `TON T38, +2` son 200 ms.

Temporizadores TON	Resolución	Ejemplo
T32, T96	1 ms	–
T33–T36, T97–T100	10 ms	`TON T35, +3` = 30 ms
T37–T63, T101–T255	100 ms	`TON T58, +120` = 12 s

5.4. Salidas de pulsos (PTO)

PTO0 (Q0.0)	PTO1 (Q0.1)	Función
SMB66	SMB76	estado (SM66.7 / SM76.7 = 1: en reposo)
SMB67	SMB77	byte de control
SMW68	SMW78	ciclo en µs o ms
SMD72	SMD82	cantidad de pulsos

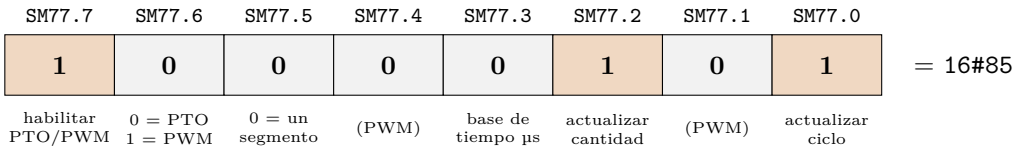


Figura 5.2: Byte de control `16#85` que usan los dos programas: PTO de un solo segmento, base de tiempo en µs, con actualización de ciclo y cantidad.

Para arrancar un tren se escriben el control, el ciclo y la cantidad, y se ejecuta `PLS`. Para detenerlo antes de tiempo, `SMB77 = 0` y `PLS 1`. Con un solo segmento, el ciclo es constante durante todo el tren (sin rampa).

5.5. Instrucciones que aparecen en los programas

Instrucción (AWL)	Qué hace
`LD / LDN`	empieza un camino con un contacto abierto / cerrado
`A / AN`	contacto en serie (Y) abierto / cerrado
`O / ON`	contacto en paralelo (O) abierto / cerrado
`EU / ED`	detecta el flanco ascendente / descendente: deja pasar la corriente un solo ciclo
`=`	bobina: copia el resultado lógico en un bit
`S bit, n / R bit, n`	pone a 1 / a 0 *n* bits a partir de *bit*, y quedan así
`LPS / LRD / LPP`	guardan, leen y sacan el resultado lógico de una pila: así AWL representa las ramificaciones de un segmento ladder
`ALD / OLD`	combinan en serie / en paralelo dos bloques de contactos
`LDW= , AW<> , AD>` ...	contactos de comparación de palabras (W), dobles palabras (D) o bytes (B)
`MOVB / MOVW / MOVD`	copian un byte, una palabra o una doble palabra
`+D , -D`	suma y resta de dobles palabras con signo
`FILL`	llena un bloque de palabras con un valor
`TON T, PT`	temporizador con retardo a la conexión: se activa cuando la entrada lleva PT unidades en 1
`PLS 0 / PLS 1`	aplica la configuración y arranca (o detiene) el PTO0 / PTO1
`CALL SBRn / CRET`	llama a una subrutina / vuelve de ella

5.6. Cómo se lee un diagrama ladder



contacto abierto: pasa si el bit es 1



contacto cerrado: pasa si el bit es 0



flanco positivo: pasa un solo ciclo



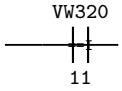
bobina: escribe el bit



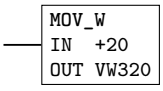
pone a 1 (set) y queda



pone a 0 (reset)



comparación: pasa si VW320 = 11



cuadro: copia, suma, temporiza...

5.7. Máquinas de pasos

Las secuencias de los dos programas se escriben como **máquinas de pasos**: una palabra guarda el número de paso (VW320 o VW322 en el programa original, VW1136 en el del gemelo). Cada segmento pregunta «¿estoy en el paso N?», hace su tarea y, cuando se cumple la condición de salida, escribe el número del paso siguiente. Se dibujan como diagramas de estados, y su comportamiento puede modelarse con redes de Petri (sección 7.7).

Capítulo 6

Programa original de secuencias

6.1. Origen y estructura

Es el programa que tenía cargado el PLC al empezar el proyecto: una versión del programa de Miguel («BRAZO MIGUEL ROBADO»), con un pequeño buzón en VB100/VB104 para recibir órdenes de la PC. Se subió desde el PLC el 26/10/2026 y está en el repositorio como `programas-plc/brazo_secuencias_original` (y `.mwp`); el listado completo está en el apéndice A.

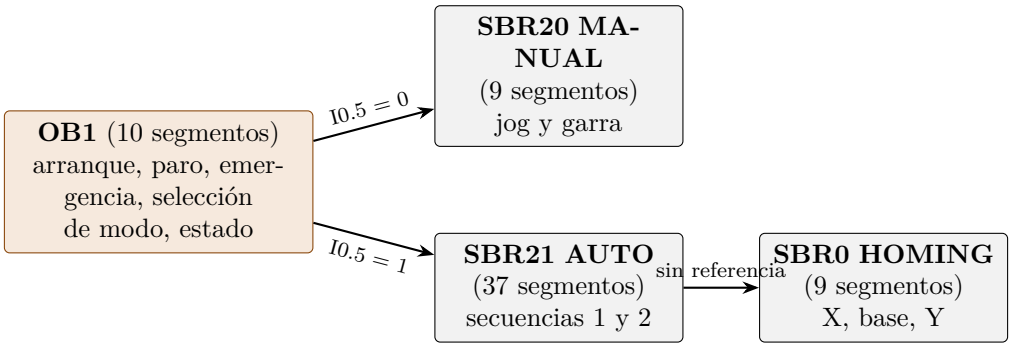


Figura 6.1: Estructura del programa original.

6.2. Memoria que usa

Dirección	Significado
Órdenes de la PC (las escribe la app con el perfil *cargado*)	
`V104.0`	iniciar (equivale a I0.7)
`V104.1`	paro en automático
`V104.2`	emergencia (queda enclavada)
`V104.3 /`	forzar modo (con VB104 = 0, el modo lo decide I0.5)
`V104.4`	
`V104.5 /`	paro / emergencia adicionales
`V104.6`	
`V104.7`	latido: la PC lo alterna; si no cambia en 5 s, el PLC borra VB104
`V100.0 /`	iniciar / paro / rearmar por flanco
`V100.2 /`	
`V100.3`	
Marcas internas	
`M4.0`	inicio pedido (I0.7 o V104.0, sin emergencia)
`M5.0`	flanco del inicio: dura un ciclo
`M4.3`	modo automático (I0.5 = 1)
`M4.5`	paro (I0.6 o V104.5)
`M4.6 / M4.7`	emergencia / emergencia enclavada
`M5.1`	flanco de rearme (V100.3)
`M5.3`	flanco del latido
`M5.5 / M5.6`	flancos de I0.2 / I0.4 (final opuesto pisado durante el homing)
`M0.0 / M0.5`	homing permitido / homing pedido

Dirección	Significado
M0.3	**referenciado**
M5.2	falla de homing
M1.0	inicio pedido desde la PC (V100.0) en automático
M1.1	final de carrera cortó el carro X en automático
M10.0	auxiliar para alternar la garra
Estado para la PC	
V200.0 / .1 / .3 / .7	listo / ocupado / falla de homing / referenciado
V301.0 - V301.6	homing en curso, listo, secuencia 1 en curso y terminada, secuencia 2 en curso y terminada, Y arriba en la secuencia 2
V302.0	la próxima secuencia es la 2
VW320 / VW322	paso de la secuencia automática / del homing
VB201 - VB203	copia de los pasos y de IB0
Constantes (se cargan en el primer ciclo)	
VD330 / VD334	avance de X en las secuencias: 20 000 / 14 000 pulsos
VD340 / VD344 / VD348	pulsos máximos de búsqueda del homing: X 30 000, Y 25 000, base 45 000

La app reconoce este programa por la firma VD340 = 30 000 y VD344 = 25 000.

6.3. Bloque principal OB1

Seg.	Qué hace
1	Señales de mando: borra VB104 en el primer ciclo; vigila el latido V104.7 con T61 (5 s); calcula inicio (M4.0) y su flanco (M5.0), modo automático (M4.3), paro (M4.5), emergencia (M4.6) y la enclava (M4.7); flancos de rearme (M5.1) y de los finales opuestos (M5.5, M5.6).
2	Emergencia: corta los dos PTO (SMB67/77 = 0 y PLS), apaga Q0.0–Q0.6, borra los pasos y las banderas. Rearme: con la emergencia ya liberada, un inicio (M5.0) o V100.3 quitan el enclavamiento.
3	Primer ciclo (SM0.1): apaga las salidas, borra marcas y memoria de mando y estado, y carga las constantes VD330–VD348.
4	Paro en automático: flanco de I0.6, V300.2, **I1.0** o V104.1, con el selector en automático. Corta los trenes, apaga salidas y vuelve las secuencias al inicio.
5	Al pasar a manual: corta todo y reinicia; en manual y sin emergencia llama a SBR20.
6	Al pasar a automático: corta todo y permite el homing; en automático y sin emergencia llama a SBR21.
7	Flanco de V100.0 en automático: inicio desde la PC (M1.0). Flanco de V100.2: paro desde la PC (V300.2).
8	Publica el estado en VB200–VB203.
9	Protección en automático: si X avanza y pisa I0.2, o retrocede y pisa I0.1, corta el PTO1 (M1.1).
10	Copia VD210/VD220 en VD304/VD308 (posiciones; en este programa siempre son 0).

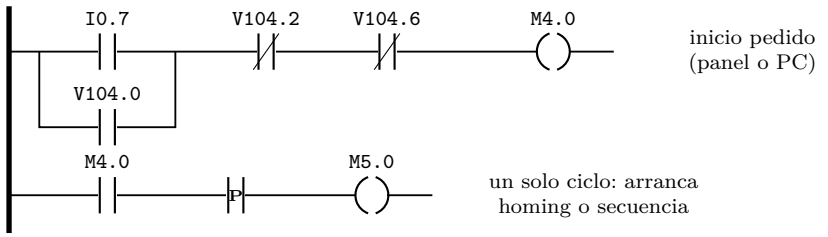


Figura 6.2: OB1, segmento 1 (parte): inicio por I0.7 *o* V104.0, sin emergencia.

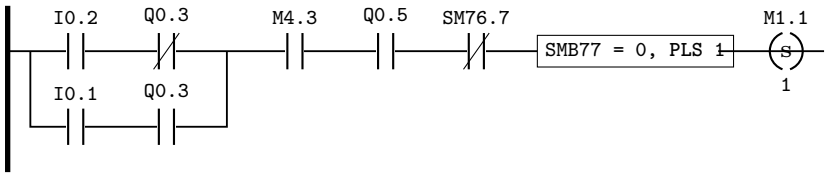


Figura 6.3: OB1, segmento 9: en automático, el carro X se detiene si llega al final del lado hacia el que avanza.

6.4. Modo manual (SBR20)

Seg.	Qué hace
1	Con cada flanco de inicio (I0.7) alterna la garra Q0.7 (abre si estaba cerrada, cierra si estaba abierta).
2, 3	I1.0 (o I1.1), con el PTO0 libre: DIR base Q0.2 = 0 (o 1) y paquete de 100 pulsos a 4 kHz.
4	Al soltar los dos pulsadores de la base: corta el PTO0.
5, 6	I1.2 / I1.3 sin el pulsador contrario, sin el final del lado de avance y con K2 abierto: cierra K1, fija DIR X y, tras 30 ms (T35/T36), da paquetes de 100 pulsos a 4 kHz.
7, 8	Igual para Y con I1.4 / I1.5, K2 y T97/T98 (exige K1 abierto).
9	Sin ningún pulsador de X/Y: corta el PTO1 y, 30 ms después (T99), abre K1 y K2.

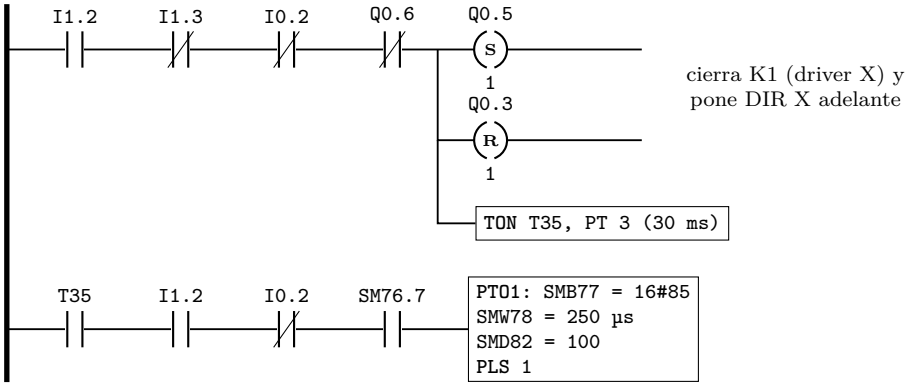


Figura 6.4: SBR20, segmento 5: jog de X adelante.

Como el paquete se corta al soltar el pulsador (segmento 9), no se sabe cuántos de los 100 pulsos salieron: con este programa la posición no se puede contar con exactitud. El programa del gemelo corrige esto dejando terminar cada paquete.

6.5. Homing (SBR0)

Seg.	Qué hace
1	Homing permitido (M0.0), pedido (M0.5), sin falla y en reposo: marca «homing en curso» y va al paso 10.
2	Paso 10: abre K2, cierra K1, DIR X atrás (Q0.3 = 1); a los 50 ms (T34) pasa al 11.
3	Paso 11: arranca el PTO1 con VD340 = 30 000 pulsos a 4 kHz; al llegar a I0.1 lo corta, abre K1 y pasa al 20.
4	Paso 20: DIR base hacia el inductivo (Q0.2 = 1) y PTO0 con VD348 = 45 000 pulsos a 2,5 kHz; al llegar a I0.0 lo corta y pasa al 30.
5	Paso 30: abre K1, cierra K2, DIR Y abajo (Q0.4 = 1); a los 50 ms (T33) pasa al 31.
6	Paso 31: PTO1 con VD344 = 25 000 pulsos; al llegar a I0.3 lo corta, abre K2 y pasa al 40.
7	Paso 40: abre la garra; a los 400 ms (T37) pasa al 50.
8	Paso 50: pone las posiciones en cero, marca **referenciado** (M0.3) y homing terminado.
9	Fallas: tiempo excedido (T58 = 12 s en X, T59 = 25 s en la base, T60 = 12 s en Y), tren terminado sin sensor, o final opuesto pisado (M5.5, M5.6). Corta todo y marca M5.2.

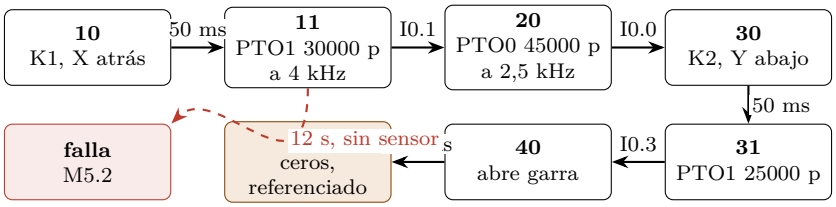


Figura 6.5: Máquina de pasos del homing (VW322).

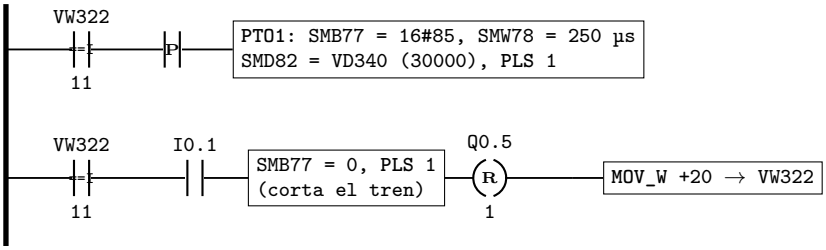


Figura 6.6: SBR0, segmento 3: arranca el tren hacia atrás y lo corta al llegar a I0.1.

6.6. Secuencias automáticas (SBR21)

Sin referencia, el primer inicio ejecuta el homing (segmento 1). Con referencia, el primer inicio ejecuta la **secuencia 1** y los siguientes la **secuencia 2** (V302.0 queda en 1).

Pasos	Seg.	Movimiento	Pulsos / tiempo
Secuencia 1			
10–12	3–5	Y sube (K2, Q0.4 = 0)	12 000 (60 mm), relé 200 ms
20–21	6–7	base gira (Q0.2 = 0)	17 000 (≈89°)

Pasos	Seg.	Movimiento	Pulsos / tiempo
30–32	8–10	X avanza (K1, Q0.3 = 0)	VD330 = 20 000 (100 mm)
40	11	abre la garra; fin	–
Secuencia 2			
100	13	cierra la garra (toma la pieza)	400 ms
110–112	14–16	Y sube	5 000 (25 mm)
120–122	17–19	X retrocede (Q0.3 = 1)	VD334 = 14 000 (70 mm)
130–131	20–21	base gira (Q0.2 = 0)	18 000 (≈95°)
140–142	22–24	Y baja (Q0.4 = 1)	14 000 (70 mm)
150	25	abre la garra (deja la pieza)	400 ms
160–162	26–28	Y sube	14 000
170–171	29–30	base vuelve (Q0.2 = 1)	18 000
180–182	31–33	X avanza	14 000
190–192	34–36	Y baja	5 000
200	37	abre la garra; fin	–

Patrón de cada movimiento: (1) cerrar el relé y fijar la dirección, esperar 200 ms (T38, T40...); (2) arrancar el tren de pulsos a 4 kHz; (3) esperar 100 ms y luego a que SM76.7 (o SM66.7) indique que terminó.

6.7. Limitaciones

- No conoce la posición: solo sabe si hizo el homing.
- Solo hace dos rutinas fijas; para cambiarlas hay que reprogramar el PLC.
- El jog corta los paquetes al soltar: no se puede contar con exactitud.
- Los finales de carrera solo se vigilan en el homing, en el jog y (para X) en automático.
- Con este programa, la app *estima* la posición (sección 10.3.4).

Capítulo 7

Programa del gemelo: `brazo_gemelo`

7.1. Por qué un programa nuevo

El programa original no podía ser la «mitad física» de un gemelo digital: no guarda la posición y solo sabe hacer dos rutinas. El programa `gemelo-digital/plc/brazo_gemelo.awl` (proyecto `.mwp` al lado; listado completo en el apéndice B) se escribió conservando el modo manual y el homing del programa de Miguel, y agregando:

1. **cuenta de pulsos** de cada eje, con signo: la posición es exacta mientras no se pierdan pasos;
2. **finales de carrera que cortan** cualquier tren de pulsos y fijan la posición;
3. un **buzón de órdenes** en la memoria V para que la PC pida homing, movimientos libres, garra, paro y velocidad;
4. un **watchdog**: si la PC deja de dar señales de vida durante un movimiento, el PLC se detiene.

Compiló sin errores en STEP 7-Micro/WIN (CPU 224 CN) y está cargado en el PLC desde el 27/10/2026. Su bloque de sistema dejó el puerto en la dirección PPI 2 a 9,6 kbit/s.

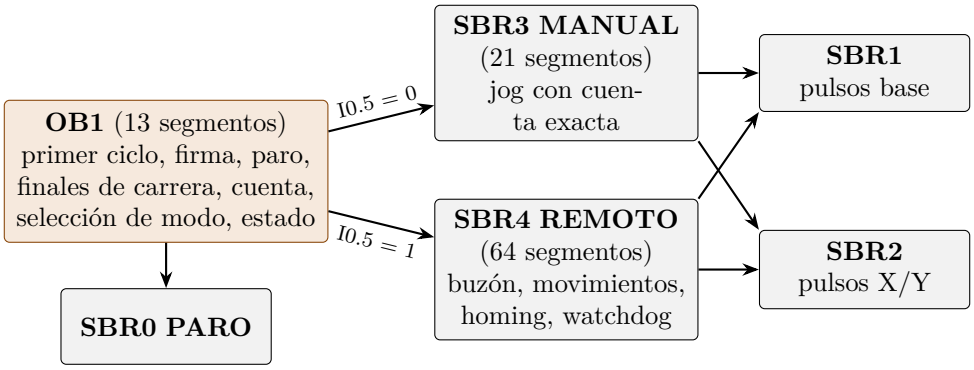


Figura 7.1: Estructura de `brazo_gemelo`. SBR1 y SBR2 son los únicos lugares donde se arrancan trenes de pulsos: así toda la cuenta pasa por un solo sitio.

7.2. Mapa de memoria

Dirección	Quién	Contenido
Buzón de órdenes (lo escribe la PC en una sola escritura de 20 bytes)		
`VB1100`	PC	orden: 1 homing, 2 mover, 3 paro, 4 cerrar garra, 5 abrir garra
`VB1101`	PC	secuencia: el PLC ejecuta cuando cambia
`VW1102`	PC	latido (watchdog de 2 s)
`VD1104 / 1108 / 1112`	PC	pulsos relativos de base / X / Y, con signo
`VW1116 / VW1118`	PC	ciclo del tren X/Y (200–2000 µs) y de la base (250–4000 µs)
Estado (lo lee la PC: 40 bytes desde VB1120)		
`VB1120`	PLC	estado: 0 reposo, 1 homing, 2 moviendo
`VB1121`	PLC	eco de la secuencia

Dirección	Quién	Contenido
VB1122	PLC	.0 referenciado, .1 remoto (I0.5), .2 PTO0 ocupado, .3 PTO1 ocupado, .4 posición incierta, .5 watchdog
VB1123	PLC	error: 1 ocupado, 2 fuera de recorrido, 3 watchdog, 4 sin homing, 5 orden desconocida, 6 paro
VD1124 / 1128 / 1132	PLC	posición de base / X / Y en pulsos
VW1136	PLC	paso de la máquina de estados
VB1138 / 1139 / 1140	PLC	copia de IB0, IB1 y QB0
VB1141	PLC	firma 16#47
VD1144 / 1148 / 1152	PLC	tramo en curso de base / X / Y
VW1156 / VW1158	PLC	ciclo en curso X/Y / base
Uso interno		
VW1160		último latido visto
VB1162		orden que se está ejecutando
VD1164 / 1168 / 1172		destino de X / Y / base
VD1176 / 1180 / 1184		$\lvert\Delta X\rvert$, $\lvert\Delta Y\rvert$, $\lvert\Delta\theta\rvert$
VD1188 / VD1192		tramo común / resto
VW1196 / VW1198, VD1202		parámetros para SBR2 / SBR1: ciclo y pulsos
V1200.0 / .1		tramo de la base / de X-Y pendiente de sumar
V1200.2 / .3		posición incierta / referenciado
V1200.5 / .6 / .7		watchdog / orden de la PC en curso / orden nueva por validar
V1201.0		tramo de X-Y recién arrancado

7.3. Bloque principal OB1

Seg.	Qué hace
1	Primer ciclo: apaga Q0.0–Q0.7 y los PTO, llena VB1100–VB1209 con ceros y carga ciclos por defecto (250 µs X/Y, 400 µs base).
2	Escribe la firma 16#47 en VB1141 en cada ciclo.
3	Paro: flanco de I0.6 o cualquier cambio del selector I0.5; error 6 y llamada a PARO.
4	X yendo hacia atrás (K1, DIR = 1) con el tren en marcha y llega a I0.1: corta el PTO1, anula el tramo y fija X = 0. Si K2 también estaba cerrado, marca posición incierta (Y pudo quedar a medio tramo).
5	Igual con I0.2: X = 34 000.
6	Igual con I0.3: Y = 0.
7	Igual con I0.4: Y = 34 000.
8	Al terminar un tramo de la base (flanco de SM66.7): suma el tramo a VD1124.
9	Al terminar un tramo de Q0.1 (flanco de SM76.7): suma los tramos a X (VD1128) y a Y (VD1132).
10	Selector en manual: llama a MANUAL (SBR3).
11	Selector en automático: llama a REMOTO (SBR4).
12	Copia IB0, IB1 y QB0 en VB1138–VB1140 para la PC.

Seg.	Qué hace
13	Banderas de estado en VB1122.

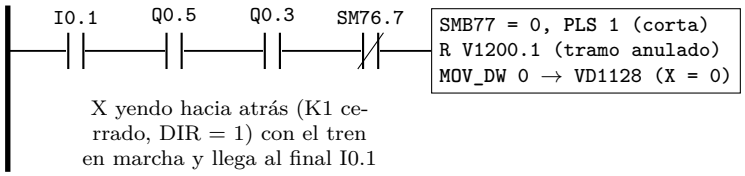


Figura 7.2: OB1, segmento 4: el final de carrera corta el tren y fija la posición exacta.

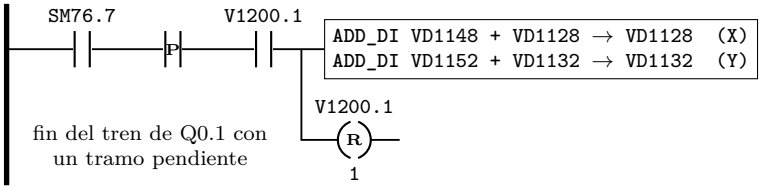


Figura 7.3: OB1, segmento 9: suma del tramo terminado a la posición de X y/o de Y.

7.4. Subrutinas de pulsos (SBR1 y SBR2)

Bloque	Qué hace
SBR1, seg. 1–2	Arranca el PTO0 con VD1202 pulsos y ciclo VW1198; guarda el tramo en VD1144 con signo (− si Q0.2 = 1) y marca V1200.0.
SBR2, seg. 1	Arranca el PTO1 con VD1202 pulsos y ciclo VW1196; borra los tramos de X e Y y marca V1200.1.
SBR2, seg. 2–5	El tramo va a X si K1 está cerrado (con signo − si Q0.3 = 1) y a Y si K2 está cerrado (− si Q0.4 = 1). Con los dos relés cerrados, el mismo tramo se suma a los dos.
SBR0 PARO	Corta los dos trenes; si había un tramo en curso marca la posición como incierta y quita el «referenciado»; abre los relés y vuelve al reposo.

7.5. Modo manual (SBR3)

Seg.	Qué hace
1	Flanco de I0.7: alterna la garra.
2–4	I1.0 con el PTO0 libre: Q0.2 = 0; a los 100 ms (T106), paquete de 100 pulsos a 2,5 kHz por SBR1.
5–7	I1.1: igual con Q0.2 = 1 (T107).
8–10	I1.2: cierra K1, Q0.3 = 0; a los 200 ms (T101), paquetes de 100 pulsos a 4 kHz por SBR2, mientras no se pise I0.2 y K2 esté abierto.

Seg.	Qué hace
11–13	I1.3: igual hacia atrás, hasta I0.1 (T102).
14–16	I1.4: cierra K2, Q0.4 = 0, hasta I0.4 (T103).
17–19	I1.5: igual hacia abajo, hasta I0.3 (T104).
20–21	Sin pulsadores de X/Y y con Q0.1 quieto: a los 300 ms (T105) abre K1 y K2.

Diferencias con el original: cada paquete de 100 pulsos se termina siempre (un nuevo paquete solo arranca con el PTO en reposo), así la cuenta es exacta; la dirección se fija antes de dar pulsos; la base va a 2,5 kHz.

7.6. Modo remoto (SBR4)

7.6.1. Velocidad, watchdog y buzón (segmentos 1–19)

Seg.	Qué hace
1–4	Limita los ciclos pedidos por la PC: X/Y entre 200 y 2000 µs; base entre 250 y 4000 µs.
5–7	Watchdog: con una orden de la PC en curso y una máquina de pasos activa, si el latido VW1102 no cambia en 2 s (T110), marca watchdog, error 3 y llama a PARO.
8	Orden nueva: si VB1101 ≠ VB1121, copia la secuencia como eco, toma la orden y marca V1200.7.
9	Orden 3 (paro): siempre se acepta; error 6 y PARO.
10	Cualquier otra orden con un movimiento en curso: error 1.
11, 12	Órdenes 4 y 5: cierra o abre la garra.
13	Orden 1: homing (paso 100).
14	Flanco de I0.7 en reposo: homing también desde el panel.
15	Orden 2 sin referencia: error 4.
16	Orden 2: destino = posición actual + pulsos relativos.
17	Destino fuera de 0–34 000 (X, Y) o de ±50 000 (base): error 2.
18	Orden 2 aceptada: paso 10.
19	Orden desconocida: error 5.

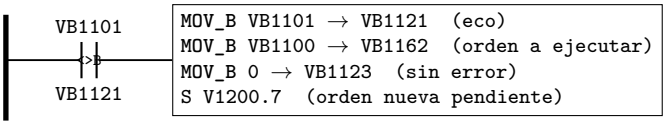


Figura 7.4: SBR4, segmento 8: una orden solo se toma cuando cambia el número de secuencia.

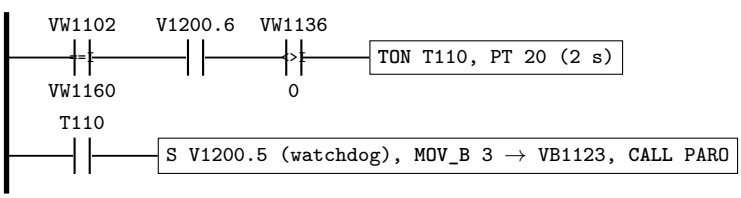


Figura 7.5: SBR4, segmentos 5 y 7: watchdog.

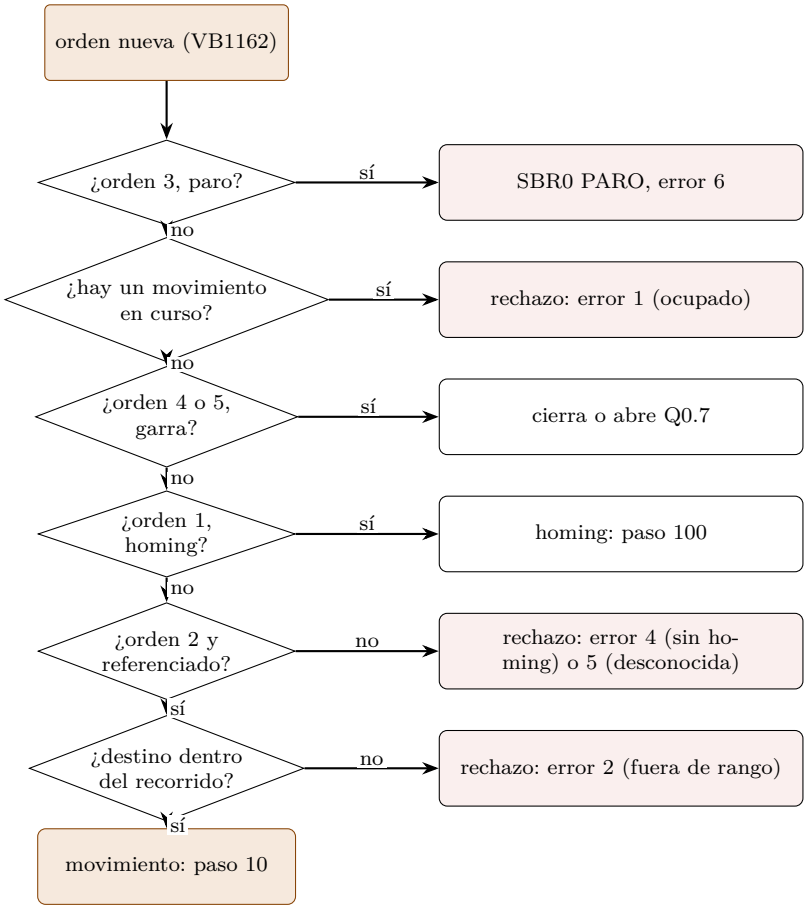


Figura 7.6: Validación de una orden (segmentos 9–19).

7.6.2. Ejecución de un movimiento (segmentos 20–51)

Seg.	Paso	Qué hace
20–23	10	Valores absolutos de los tramos: $\|\Delta X\|$, $\|\Delta Y\|$, $\|\Delta\theta\|$.
24–27	10	Tramo común = el menor de $\|\Delta X\|$ y $\|\Delta Y\|$; resto = el mayor menos el común.
28–35	10	Cierra K1 si $\Delta X \neq 0$ y K2 si $\Delta Y \neq 0$; fija las tres direcciones según el signo.
36–37	10	Espera 300 ms (T111) a que cierren los relés; pasa al 11.
38	11	Arranca la base en paralelo (SBR1) si $\Delta\theta \neq 0$.
39–41	11	Arranca el tramo común con K1 y K2 cerrados (SBR2) y pasa al 12; sin tramo común, va al 13.
42–43	12	Al terminar el común, abre el relé del eje que ya llegó; pasa al 13.
44–47	13	A los 300 ms (T113), da el resto al eje que falta y pasa al 14; sin resto, al 15.
48–49	14	Espera el fin del resto.
50–51	15	Espera a la base, abre los relés, estado 0 (reposo).

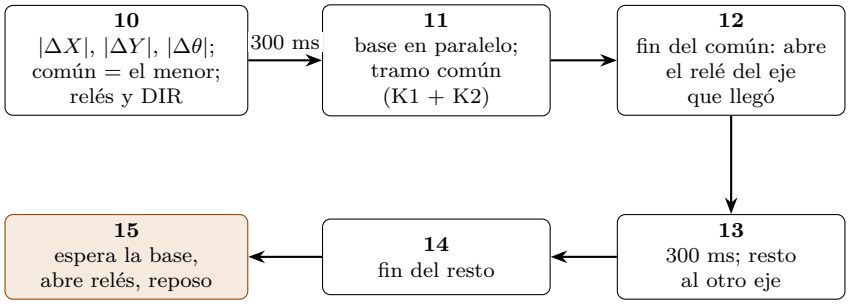


Figura 7.7: Máquina de pasos de un movimiento.

Ejemplo numérico: desde X = 10 000, Y = 4 000 se pide X = 30 000, Y = 16 000. Entonces $\Delta X = +20\,000$, $\Delta Y = +12\,000$; común = 12 000 (X e Y avanzan juntos 60 mm en 3 s a 4 kHz); resto = 8 000 para X (40 mm más, en 2 s). Total 5,6 s contando las esperas de los relés.

7.6.3. Homing (segmentos 52–64)

Seg.	Paso	Qué hace
52–54	100	Abre K1, cierra K2, Y hacia abajo; a los 300 ms (T120), si ya está en I0.3 salta al 103; si no, tren de búsqueda de 500 000 pulsos a 4 kHz (paso 102).
55	102	Llega a I0.3: corta y fija Y = 0.
56–58	103	Abre K2, cierra K1, X hacia atrás; T121; tren de búsqueda (paso 105).
59	105	Llega a I0.1: corta y fija X = 0.
60–62	106	Abre K1, abre la garra, base hacia el inductivo (Q0.2 = 1); a los 400 ms (T122), tren de búsqueda a 2,5 kHz (paso 108).
63	108	Llega a I0.0: corta el PTO0.
64	109	Pone las tres posiciones en cero, marca **referenciado**, reposo.

7.7. Red de Petri del homing

La guía 1 pide modelar el comportamiento secuencial con redes de Petri. En una red de Petri, los *lugares* (círculos) son estados o recursos, las *transiciones* (barras) son eventos, y una *marca* (punto) indica el estado actual; una transición se dispara cuando sus lugares de entrada tienen marca y se cumple su condición.

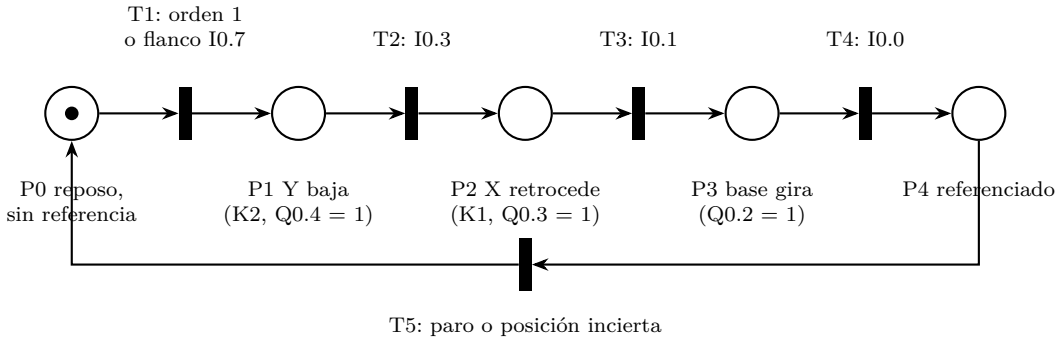


Figura 7.8: Red de Petri del homing de `brazo_gemelo`. Es una red de estado: un solo token recorre la secuencia, lo que garantiza que dos ejes nunca se buscan a la vez.

Lo mismo vale para la orden de movimiento: el lugar «máquina de pasos en reposo» (VW1136 = 0) es un recurso que la orden consume y devuelve al final. Por eso una orden que llega con el recurso ocupado se rechaza con error 1.

7.8. Comparación de los dos programas

	Original	**brazo_gemelo**
Posición	no la conoce	contada al pulso
Movimientos	2 rutinas fijas	a cualquier punto del recorrido
Finales de carrera	homing, jog y X en automático	cortan cualquier movimiento y fijan la posición
Mando desde la PC	iniciar, paro, emergencia, rearme	homing, mover, garra, paro, velocidad
Seguridad ante la PC	borra órdenes a los 5 s	watchdog de 2 s durante el movimiento
Velocidad	fija (4 kHz)	elegible (500–5000 Hz)
Orden del homing	X, base, Y	Y, X, garra, base (como el de Miguel)
Puerto PPI	dirección 6, 19,2 kbit/s	dirección 2, 9,6 kbit/s

Capítulo 8

Modelo cinemático, jacobiano y modelo 3D

8.1. Levantamiento de medidas

El modelo parte de un levantamiento de la máquina: 20 fotos acotadas (`modelo-3d/levantamiento/fotos/`), bocetos de las piezas, una planilla de medición imprimible y el manual del fabricante. Cada cota quedó clasificada: *leída* (medida en la pieza o el boceto), *estimada* (de fotos o del manual, ajustada para que el modelo reproduzca las fotos con menos de 5 mm de diferencia) o *SUPUESTO*.

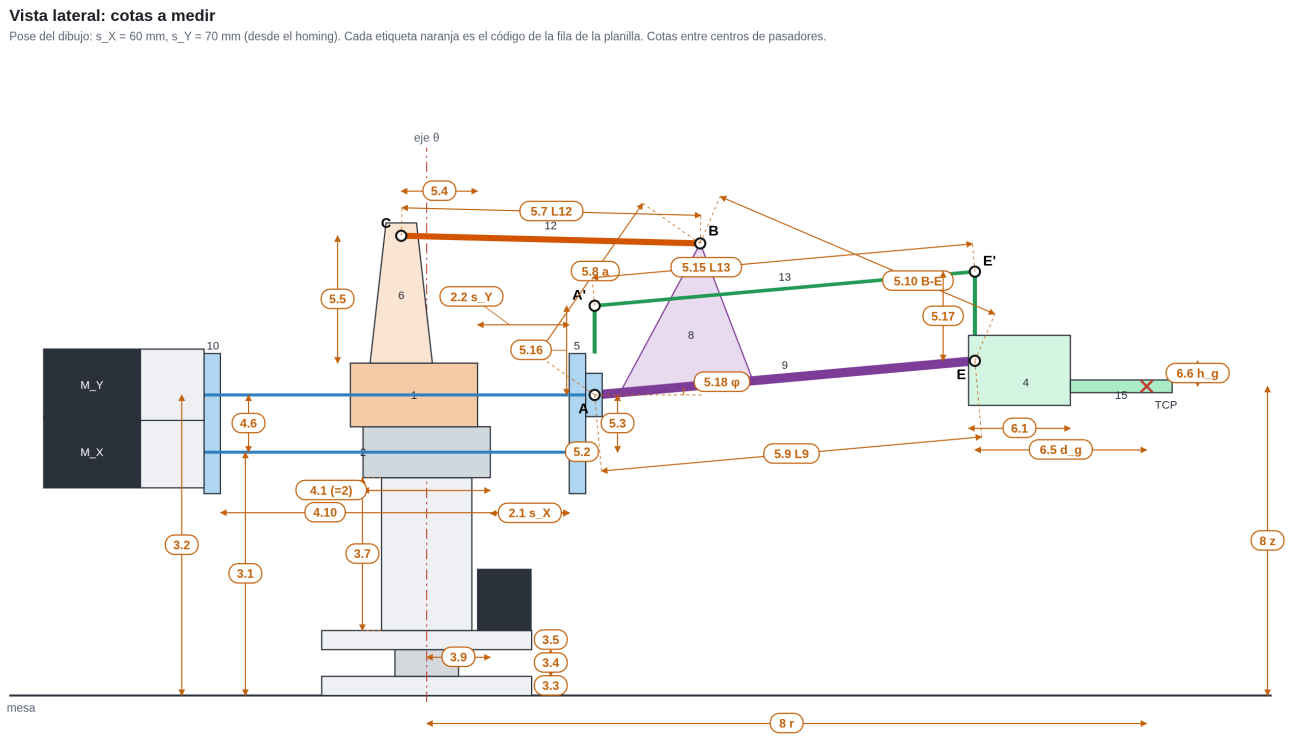


Figura 8.1: Esquema de medidas en vista lateral (`levantamiento/esquema_medidas_lateral.png`).

Parámetro	Valor	Origen	Significado
`largoGuias`	300 mm	leída	guías del bastidor
`espesorPlaca`	13 mm	leída	placas extremas del bastidor
`largoBloque`	100 mm	leída	bloques 1 y 2
`holgura`	2 mm	SUPUESTO	distancia bloque–placa en el final de carrera
`recorridoX`, `recorridoY`	170 mm	dato del equipo	$300 - 2 \cdot 13 - 100 - 2 \cdot 2$
`sepNiveles`	45 mm	leída	separación entre los dos husillos
`a`	145 mm	estimada	balancín A–B
`beta` (β)	50°	estimada	ángulo entre A–B y el brazo
`L12`	235 mm	estimada	biela B–C
`L9`	300 mm	estimada	brazo A–E (igual a la varilla 13: paralelogramo)
`dg`, `hg`	135, 20 mm	estimada	del pivote E al centro de las mordazas (TCP)
`uA`, `dhA`	20, 45 mm	estimada	posición del pivote A en el bastidor

Parámetro	Valor	Origen	Significado
`hC, dhC, uCBloque`	110, −10, −60 mm	leída / SUPUESTO / estimada	posición del pivote C en el bloque del carro Y
`thetaMin, thetaMax`	±175°	SUPUESTO	límite de giro por los cables
`pasoHusillo`	5 mm	SUPUESTO	husillo SFU1605
`micropasosXY`	5 (1000 p/vuelta)	leída	DIP de los drivers

Todos los parámetros viven en un solo archivo, `web/js/cinematica.js`; el script `pruebas_cinematica.js` los exporta a `modelo-3d/cinematica/parametros.json` para MATLAB y Python. De ellos se derivan, entre otros: altura del pivote A $h_A = 236$ mm, posición de la placa delantera con $s_X = 0$, $u_0 = 52$ mm, y la posición del pivote C respecto de A, $(c_{u0}, c_h) = (-252, 125)$ mm.

8.2. El mecanismo

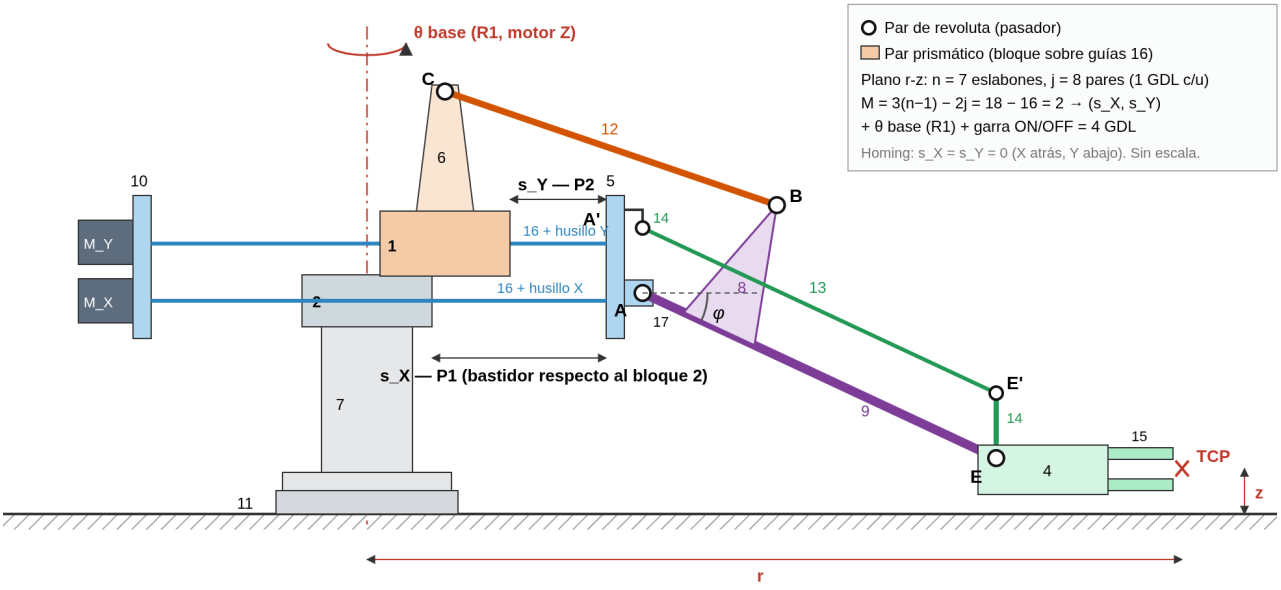


Figura 8.2: Esquema cinemático del brazo (`modelo-3d/informe/figuras/`).

El brazo trabaja en un plano vertical (r, z) que gira con la base:

- el **carro X** desplaza todo el bastidor, y con él el pivote A del brazo: cambia el alcance;
- el **carro Y** mueve el bloque 1 sobre el bastidor; la biela L_{12} une el pivote C del bloque con el pivote B del balancín A–B, solidario al brazo: al moverse el bloque, el brazo gira un ángulo φ;
- la varilla 13 forma un paralelogramo que mantiene la garra horizontal.

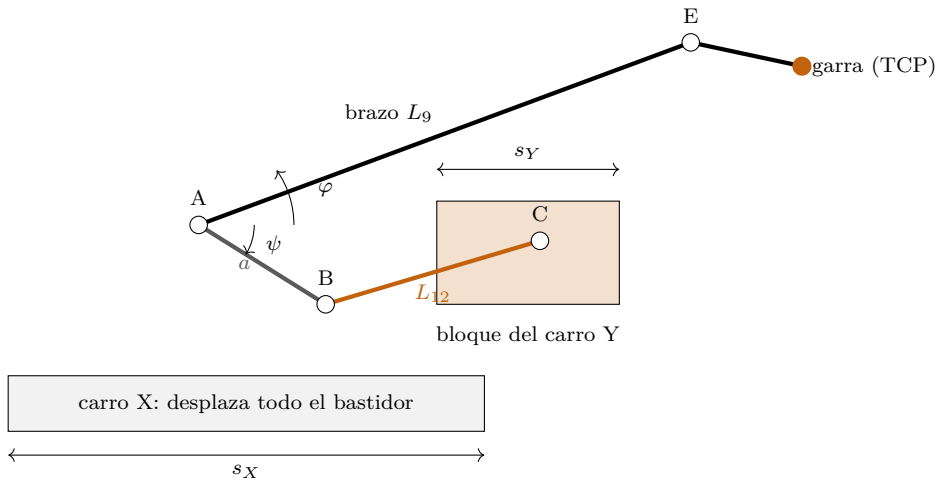


Figura 8.3: Esquema simplificado: lazo A–B–C y brazo A–E.

8.3. Cinemática directa: de los carros a la garra

Entrada $q = (\theta, s_X, s_Y)$; salida (X, Y, Z) de la garra.

Paso 1: lazo de bielas. La posición geométrica del bloque desde su tope trasero es $g = 170 - s_Y$ (el homing de Y deja el bloque adelante). El pivote C queda, respecto de A, en $C = (g + c_{u0},\ c_h) = (g - 252,\ 125)$. El pivote B está a distancia a de A y a distancia L_{12} de C: es la intersección de dos circunferencias. Con $d = |C|$,

$$\psi = \operatorname{atan2}(c_h,\ g + c_{u0}) - \arccos\left(\frac{a^2 + d^2 - L_{12}^2}{2\,a\,d}\right),$$

eligiendo el signo del montaje real (B por debajo de la recta A–C). El ángulo del brazo es $\varphi = \psi - \beta$.

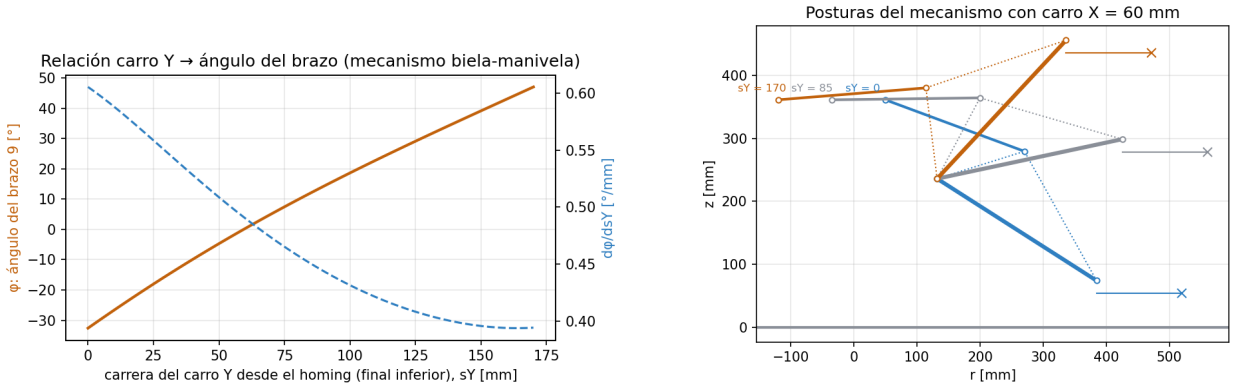
Paso 2: posición de la garra en el plano.

$$r = u_0 + s_X + u_A + L_9 \cos\varphi + d_g = 207 + s_X + 300\cos\varphi \quad [\text{mm}]$$

$$z = h_A + L_9 \sin\varphi - h_g = 216 + 300\sin\varphi \quad [\text{mm}]$$

Paso 3: giro de la base. $X = r\cos\theta,\ \ Y = r\sin\theta,\ \ Z = z.$

Ejemplo resuelto: el homing $q = (0, 0, 0)$. $g = 170$; $C = (-82, 125)$, $d = 149{,}5$ mm; $\arccos\frac{145^2+149{,}5^2-235^2}{2\cdot145\cdot149{,}5} = \arccos(-0{,}273) = 105{,}9$ř; $\operatorname{atan2}(125, -82) = 123{,}3$ř; $\psi = 17{,}4$ř, $\varphi = -32{,}6$ř. Luego $r = 207 + 0 + 300\cos(-32{,}6\text{ř}) = 459{,}7$ mm y $z = 216 + 300\sin(-32{,}6\text{ř}) = 54{,}4$ mm: la garra queda baja y cerca de la columna.



(a) ángulo del brazo φ y su derivada frente a s_Y (b) posturas del mecanismo con $s_X = 60$ mm

Figura 8.4: Relación entre el carro Y y el ángulo del brazo: de $-32{,}6$ř ($s_Y = 0$) a $+47{,}0$ř ($s_Y = 170$ mm), no lineal.

8.4. Cinemática inversa: de la garra a los carros

Entrada (X, Y, Z); salida q.

1. $\theta = \operatorname{atan2}(Y, X)$, $r = \sqrt{X^2 + Y^2}$. Si hay varias soluciones (θ y $\theta \pm 360$ř), se elige la más cercana a la actual.
2. De la altura: $\sin\varphi = \dfrac{Z + h_g - h_A}{L_9}$. Si $|\sin\varphi| > 1$, la altura es inalcanzable.
3. Del alcance: $s_X = r - d_g - L_9\cos\varphi - u_0 - u_A$.
4. Del lazo: con $\psi = \varphi + \beta$, $B = a(\cos\psi, \sin\psi)$; la biela exige $|B - C| = L_{12}$ con $C = (g - 252, 125)$:

$$g = B_x + 252 \pm \sqrt{L_{12}^2 - (125 - B_z)^2}, \qquad s_Y = 170 - g,$$

   eligiendo la solución del montaje real.
5. Se descartan soluciones fuera de $0 \le s_X, s_Y \le 170$ o con la pinza chocando con la mesa.

Ejemplo resuelto: garra en $(500, 200, 250)$ mm. $\theta = \operatorname{atan2}(200, 500) = 21{,}80$ř; $r = 538{,}52$ mm; $\sin\varphi = (250 + 20 - 236)/300 = 0{,}1133 \Rightarrow \varphi = 6{,}51$ř; $s_X = 538{,}52 - 135 - 300\cos 6{,}51$ř $- 52 - 20 = 33{,}45$ mm; $\psi = 56{,}51$ř, $B = (80{,}0;\ 120{,}9)$; $g = 80{,}0 + 252 - \sqrt{235^2 - 4{,}1^2} = 97{,}05$ mm; $s_Y = 72{,}95$ mm. Comprobación con la directa: vuelve exactamente a $(500{,}000;\ 200{,}000;\ 250{,}000)$.

8.5. Jacobiano y linealización

El jacobiano relaciona pequeños movimientos de los ejes con pequeños movimientos de la garra:

$$\Delta\mathbf{p} \approx J(q)\,\Delta q, \qquad J = \frac{\partial(X, Y, Z)}{\partial(\theta, s_X, s_Y)} = \begin{bmatrix} -r\sin\theta & \cos\theta & \cos\theta\,\dfrac{\partial r}{\partial s_Y} \\ r\cos\theta & \sin\theta & \sin\theta\,\dfrac{\partial r}{\partial s_Y} \\ 0 & 0 & \dfrac{\partial z}{\partial s_Y} \end{bmatrix},$$

con $\dfrac{\partial r}{\partial s_Y} = -L_9 \sin\varphi \dfrac{d\varphi}{ds_Y}$ y $\dfrac{\partial z}{\partial s_Y} = L_9 \cos\varphi \dfrac{d\varphi}{ds_Y}$. La derivada $d\varphi/ds_Y$ sale de derivar la ecuación del lazo $|a\,e(\psi) - C(g)|^2 = L_{12}^2$. Esta es la **linealización** del brazo alrededor de un punto de operación que pide la guía de laboratorio para la Fase 2: cerca de q_0, el brazo se comporta como el sistema lineal $\Delta\mathbf{p} = J(q_0)\Delta q$.

Punto q	J **(**θ **en rad, mm/mm)**	$\det J$	$d\varphi/ds_Y$	μ
(0ř, 0, 0) homing	$\begin{bmatrix} 0 & 1 & 1{,}708 \\ 459{,}7 & 0 & 0 \\ 0 & 0 & 2{,}671 \end{bmatrix}$	1228	0,606 °/mm	37,7°
(0ř, 85, 85) centro	$\begin{bmatrix} 0 & 1 & -0{,}492 \\ 585{,}4 & 0 & 0 \\ 0 & 0 & 2{,}307 \end{bmatrix}$	1350	0,451 °/mm	61,3°
(30ř, 120, 40)	$\begin{bmatrix} -311{,}3 & 0{,}866 & 0{,}411 \\ 539{,}1 & 0{,}500 & 0{,}237 \\ 0 & 0 & 2{,}725 \end{bmatrix}$	1696	0,528 °/mm	47,8°
(0ř, 170, 170) esquina	$\begin{bmatrix} 0 & 1 & -1{,}510 \\ 581{,}5 & 0 & 0 \\ 0 & 0 & 1{,}407 \end{bmatrix}$	818	0,394 °/mm	92,4°

Cómo se lee. En el centro (0ř, 85, 85): un milímetro del carro Y sube la garra 2,31 mm y la acerca 0,49 mm; un milímetro del carro X la aleja exactamente 1 mm; un grado de base la desplaza $585{,}4 \cdot \pi/180 = 10{,}2$ mm de lado. Es decir, el carro Y *amplifica* su movimiento en altura por más de 2.

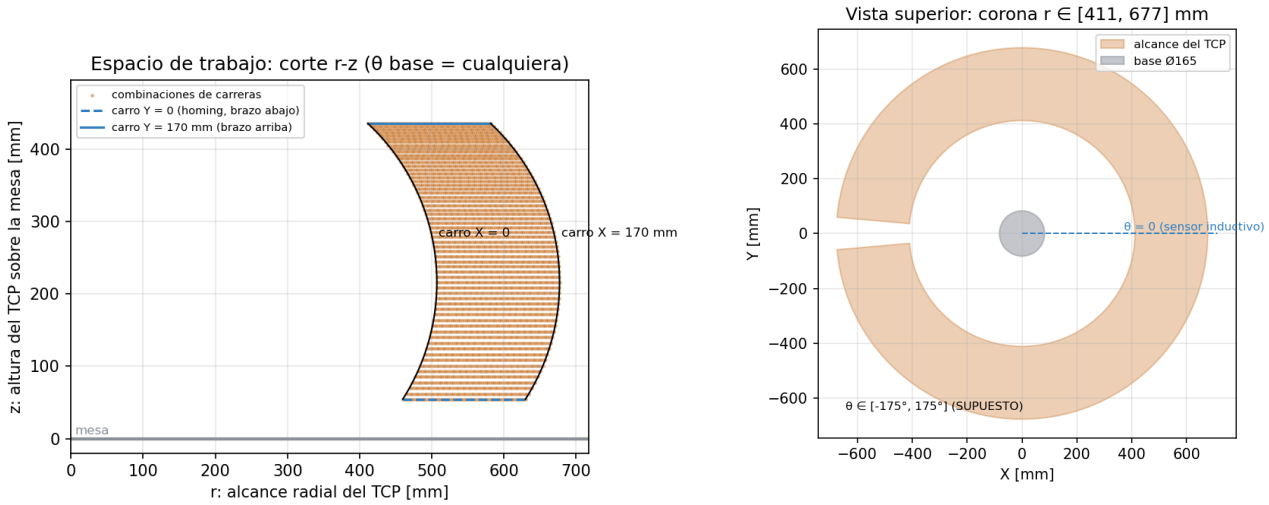
Qué tan buena es la linealización. Comparación con la cinemática exacta desde el centro:

$\Delta q = (\Delta\theta, \Delta s_X, \Delta s_Y)$	exacto $\Delta(X, Y, Z)$ [mm]	lineal $J\Delta q$ [mm]	error
(0ř, 2, 2)	(0,98; 0; 4,59)	(1,02; 0; 4,61)	0,04 mm
(2ř, 0, 0)	(−0,36; 20,43; 0)	(0; 20,43; 0)	0,36 mm
(0ř, 10, 10)	(4,28; 0; 22,52)	(5,08; 0; 23,07)	0,97 mm
(5ř, 0, −10)	(1,82; 51,38; −23,61)	(4,92; 51,09; −23,07)	3,2 mm

La aproximación es excelente para movimientos de pocos milímetros y empeora con desplazamientos grandes, sobre todo de la base (el arco no es una recta). Por eso la página recalcula la inversa en cada punto de una trayectoria en lugar de usar solo el jacobiano.

Singularidades y transmisión. Como $\det J = r\,\partial z/\partial s_Y$ y en todo el recorrido $r > 400$ mm y $\partial z/\partial s_Y > 1{,}4$, el determinante nunca se anula: **no hay singularidades dentro del recorrido**. El *ángulo de transmisión* μ entre la biela y el balancín (ideal 90°) mide qué tan bien la biela empuja al brazo: vale 37,7° en el homing (transmisión pobre, el carro Y necesita más fuerza) y 92,4° con el brazo arriba.

8.6. Espacio de trabajo



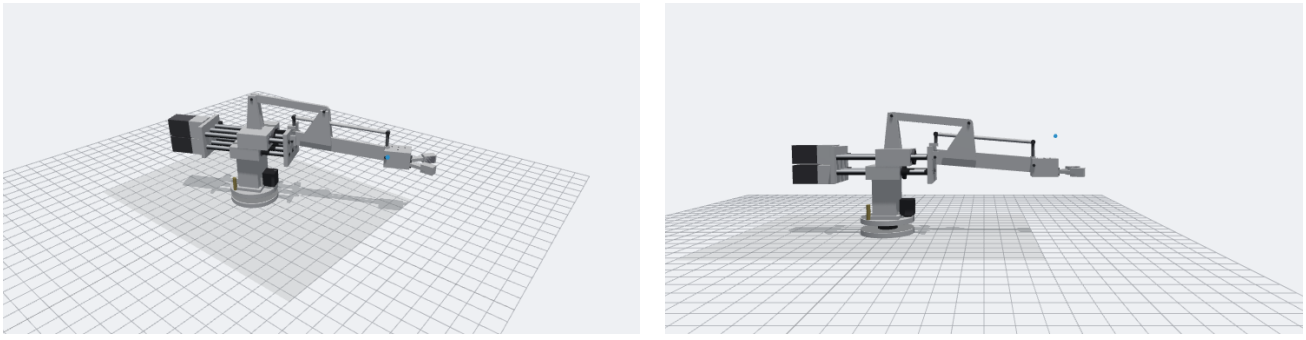
(a) corte vertical (r, z) (b) vista superior

Figura 8.5: Espacio de trabajo: una franja curva en el plano vertical que gira con la base; alcance radial de 411 a 677 mm.

Desde el homing (esquina inferior izquierda de la franja), un movimiento vertical de la garra sale de la zona alcanzable: por eso muchos puntos escritos a mano resultan inalcanzables, y la app propone el punto alcanzable más cercano.

8.7. Modelo 3D

Las piezas se modelaron con las mismas cotas y se ensamblaron en `global/modelo-3d/brazo_xk-js1-1.glb`, generado por el script `generar_modelo.js` (sin programas de diseño). El visor (three.js) coloca cada pieza según la cinemática directa en cada cuadro.



(a) isométrica (b) lateral

Figura 8.6: Modelo 3D del brazo.

8.8. Scripts del modelado

Archivo	Qué hace
`web/js/cinematica.js`	modelo: parámetros, directa, inversa, jacobiano, espacio de trabajo, pulsos, trayectorias
`modelo-3d/cinematica/pruebas_cinematica.js`	pruebas: la inversa de la directa devuelve el mismo q; exporta `parametros.json` y `tabla_cinematica.csv`
`modelo-3d/cinematica/cinematica_brazo.m`	la misma cinemática en MATLAB
`modelo-3d/cinematica/graficas_cinematica.py`	gráficas del informe (espacio de trabajo, $\varphi(s_Y)$, posturas)
`modelo-3d/levantamiento/generar_esquemas.py`	esquemas de los pasos del levantamiento
`global/modelo-3d/generar_modelo.js`	genera el modelo 3D `.glb`

Listing 8.1: Cinemática inversa (fragmento de `web/js/cinematica.js`).

```
const s = (obj.Z + p.hg - d.hA) / p.L9;           // seno del ángulo del brazo
if (Math.abs(s) > 1) return { valida: false, motivo: 'Altura inalcanzable' };
for (const [rama, phi] of [['adelante', Math.asin(s)], ['atrás', Math.PI - Math.asin(s)]]) {
  const sX = r - p.dg - p.L9 * Math.cos(phi) - d.u5_0 - p.uA;   // alcance -> carro X
  const psi = phi + beta;                                        // balancín A-B
  const bx = p.a * Math.cos(psi), bz = p.a * Math.sin(psi);
  const disc = p.L12 * p.L12 - (d.ch - bz) * (d.ch - bz);        // biela L12 hasta C
  if (disc < 0) continue;
  // dos posiciones posibles del bloque; se queda la del montaje real
}
```

Capítulo 9

Protocolo de comunicación PLC–PC: PPI

9.1. Qué es PPI y por qué se usó

PPI (*Point-to-Point Interface*) es el protocolo con el que STEP 7-Micro/WIN programa y monitorea los S7-200. Es un protocolo **maestro–esclavo** sobre RS-485, derivado de PROFIBUS-FDL: la PC es el maestro (dirección 0) y el PLC el esclavo (hoy, dirección 2). Dentro de las tramas PPI viajan mensajes **S7** que leen o escriben zonas de memoria.

	PPI (elegido)	**Modbus RTU** (guía, Fase 3)
Hardware	el cable USB-PPI que ya tenía el laboratorio	el mismo puerto RS-485
Programa del PLC	no hay que programar la comunicación: el sistema operativo del PLC responde solo	hay que programarla en modo *Freeport* o con la librería Modbus de Micro/WIN, y el puerto deja de servir para programar mientras está activa
Acceso	cualquier zona de memoria (I, Q, M, V, SM)	solo los registros que el programa mapea
Herramientas	poco documentado: se programó desde cero en Python	muy documentado, muchas herramientas

PPI permitió leer el programa original *sin modificarlo* (las primeras pruebas fueron de solo lectura) y seguir usando Micro/WIN con el mismo cable. El cliente está en `app/ppi.py` y `app/serie.py`, sin bibliotecas externas.

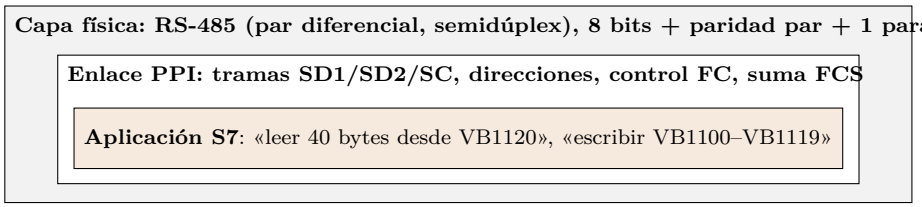


Figura 9.1: Capas del protocolo.

9.2. Capa física

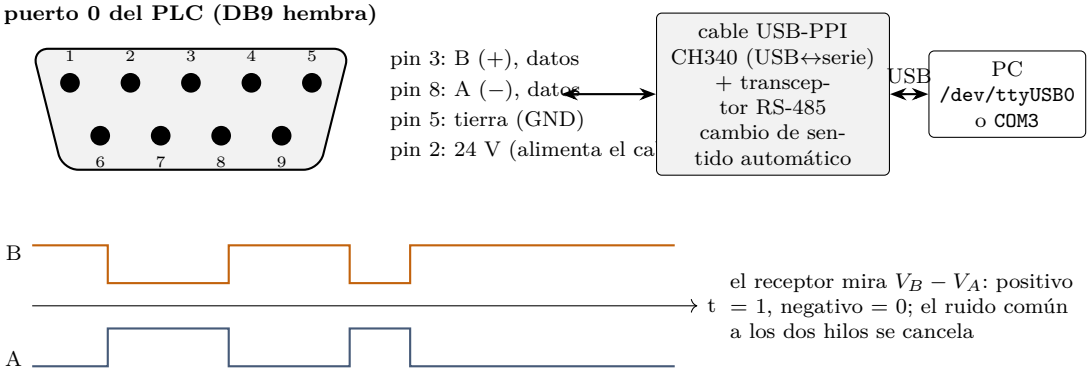


Figura 9.2: Conector, cable y señal diferencial RS-485 (esquemática).

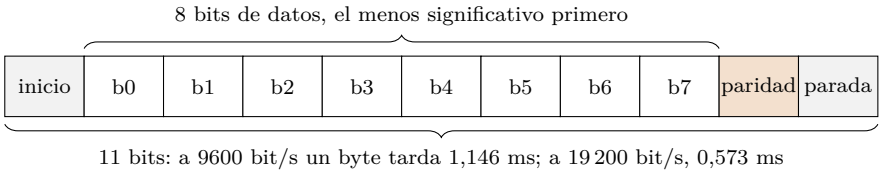


Figura 9.3: Formato de cada carácter: 8 bits, paridad par, 1 bit de parada (8E1).

9.3. Tramas

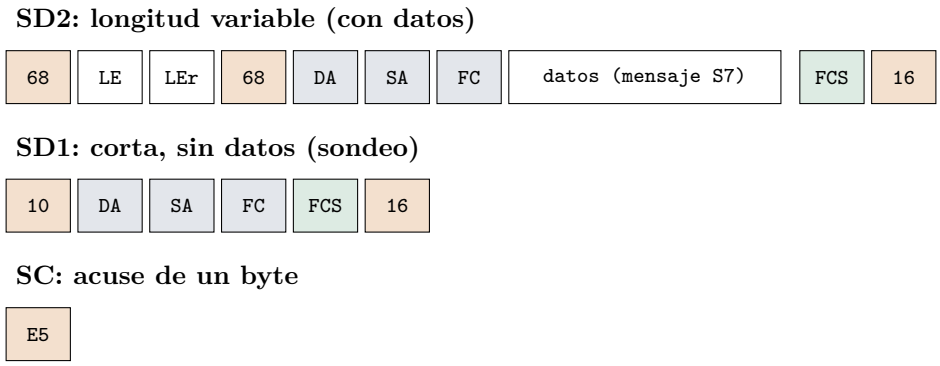


Figura 9.4: Formatos de trama PPI.

Campo	Significado
`68 / 10`	delimitador de inicio de SD2 / SD1
`LE, LEr`	longitud de DA + SA + FC + datos, repetida
`DA, SA`	dirección de destino y de origen (PC = 0, PLC = 2)
`FC`	control: `6C`/`7C` petición (el bit FCB alterna para distinguir una trama nueva de una repetida); `5C`/`7C` sondeo; `08` respuesta con datos
`FCS`	suma de DA, SA, FC y los datos, módulo 256
`16`	delimitador de fin
`E5`	acuse: «recibí tu trama» o «todavía no tengo la respuesta»

9.4. Intercambio y tiempos medidos

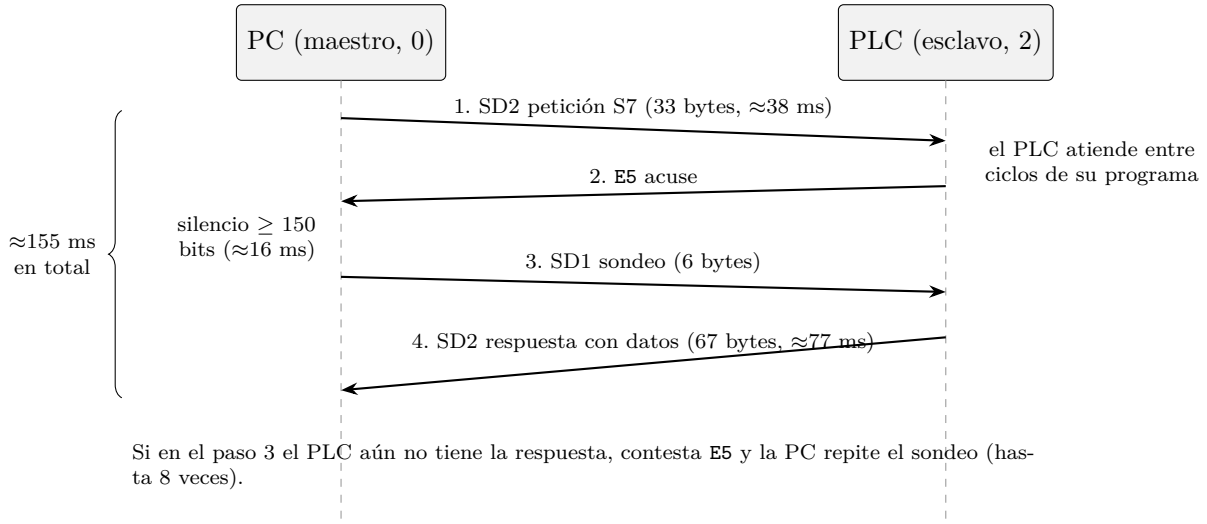


Figura 9.5: Lectura de 40 bytes a 9,6 kbit/s con los tiempos medidos en la planta.

9.5. Mensajes S7: ejemplos byte a byte

9.5.1. Lectura del estado

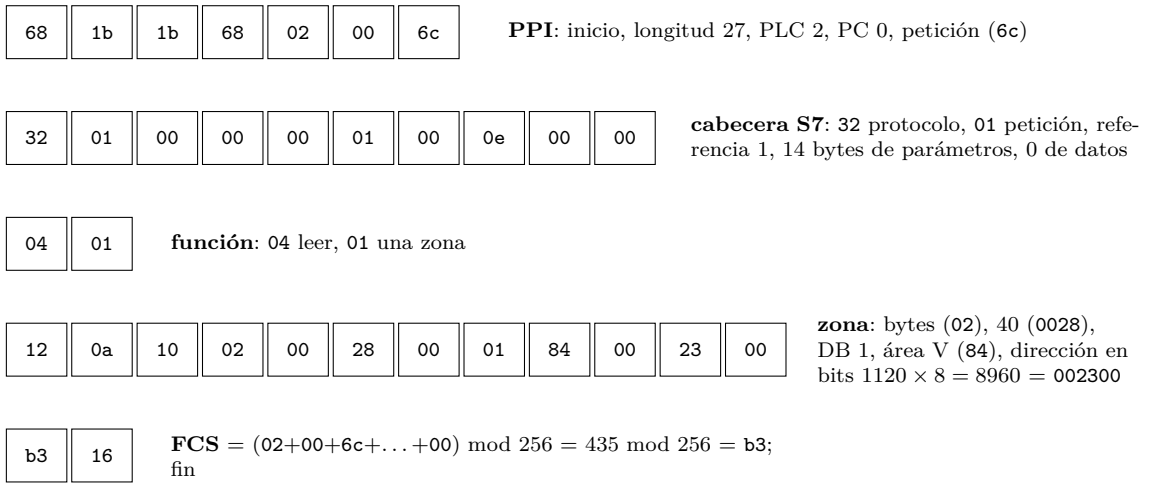


Figura 9.6: Petición «leer 40 bytes desde VB1120».

La respuesta (67 bytes) trae: cabecera PPI con las direcciones invertidas y FC `08`; cabecera S7 de tipo `03` (respuesta) con código de error 0; función `04 01`; `FF` (lectura correcta), `04` (bytes), longitud en bits `0140` (320 bits) y los 40 bytes de estado.

9.5.2. Escritura de una orden

Orden «mover X +4000 pulsos (20 mm) e Y −2000 pulsos (−10 mm)», secuencia 5, latido 7, ciclos 250 y 400 µs. La app empaqueta los 20 bytes del buzón con `struct.pack('>BBHiiiHH', ...)` (big-endian, como el PLC):

Bytes	Campo	Hex	Valor
VB1100	orden	`02`	mover
VB1101	secuencia	`05`	5
VW1102	latido	`00 07`	7
VD1104	pulsos base	`00 00 00 00`	0
VD1108	pulsos X	`00 00 0f a0`	+4000
VD1112	pulsos Y	`ff ff f8 30`	−2000 (complemento a 2)
VW1116	ciclo X/Y	`00 fa`	250 µs
VW1118	ciclo base	`01 90`	400 µs

Trama completa (57 bytes):

```
68 33 33 68 02 00 7c                          PPI: longitud 51, PLC 2, PC 0, petición
32 01 00 00 00 01 00 0e 00 18                 S7: petición, 14 bytes de parámetros, 24 de datos
05 01 12 0a 10 02 00 14 00 01 84 00 22 60     escribir 1 zona: 20 bytes en V, bit 8800 (VB1100)
00 04 00 a0                                   datos: tipo bytes, 160 bits
02 05 00 07 00 00 00 00 00 00 0f a0 ff ff f8 30 00 fa 01 90
39 16                                         FCS = 2105 mod 256 = 0x39; fin
```

La app solo puede escribir en las zonas autorizadas para el programa reconocido (VB1100–VB1119 con `brazo_gemelo`); cualquier otra dirección se rechaza en `ppi.py` antes de salir por el cable.

9.6. Lo aprendido en la planta

- **Silencio entre tramas.** A 9,6 kbit/s el PLC ignoraba la trama que llegaba pegada a la anterior: el primer sondeo se perdía y se esperaban 0,4 s. Con un silencio de unos 150 bits antes de transmitir, las lecturas bajaron de 568 ms a 155 ms y dejaron de fallar.
- **Búsqueda automática.** Si cambia la dirección o la velocidad del PLC (al cargar un bloque de sistema), la app prueba primero 2, 6 y 7 a 9,6 y 19,2 kbit/s, y luego las 31 direcciones posibles.
- **Un solo maestro.** El cable lo usa la app o Micro/WIN, nunca los dos a la vez.

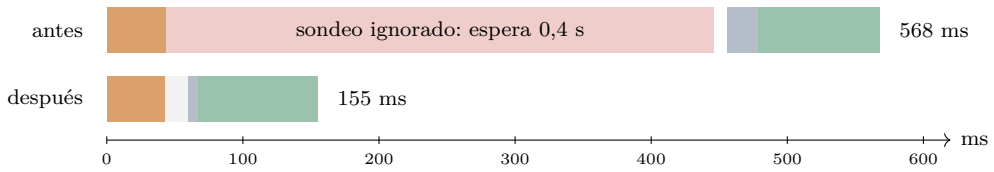


Figura 9.7: Tiempo de una lectura antes y después del silencio entre tramas (naranja: petición; azul: sondeo; verde: respuesta).

Listing 9.1: Núcleo del intercambio (`app/ppi.py`).

```
def _intercambio(self, pdu):
    self.p.vaciar_entrada()
    self._pausa()                              # silencio de ~150 bits: el PLC lo necesita
    self.p.escribir(self._sd2(0x7C if self._fcb else 0x6C, pdu))
    self._fcb = not self._fcb
    if not self._leer(0.4).startswith(b'\xe5'):
        raise ErrorPPI('el␣PLC␣no␣acusó␣la␣petición')
    for intento in range(8):                   # sondeo hasta que llegue la respuesta
        self._pausa()
        self.p.escribir(self._sd1(0x5C if intento % 2 == 0 else 0x7C))
        r = self._leer(0.4)
        if r.startswith(b'\x68'):
            return r[7:4 + r[1]]               # mensaje S7 (tras verificar la suma)
```

9.7. Limitaciones

A 9,6 kbit/s caben unas 6 lecturas por segundo: suficiente para ver y mandar el brazo, no para un lazo de control rápido desde la PC. Por eso los movimientos los ejecuta el PLC y la PC solo envía destinos. Volver a 19,2 kbit/s en el bloque de sistema duplicaría la frecuencia de lectura.

Capítulo 10

Las aplicaciones

10.1. Panorama

Aplicación	Dónde corre	Para qué
STEP 7-Micro/WIN	máquina virtual Windows XP (VirtualBox) en la PC de Linux; o Windows directo	editar, compilar y cargar los programas del PLC; el USB del cable se pasa a la VM desde *Dispositivos* → *USB*
App del gemelo (`app/`)	PC del laboratorio, Linux o Windows, Python 3	conectar con el PLC, ver y mandar el brazo; menú de tres modos; acceso por red
Página publicada	internet (GitHub Pages y Cloudflare Pages)	documentación del proyecto, simulación del brazo, catálogo de archivos
Scripts de modelado	cualquier PC (Node.js, Python, MATLAB)	cinemática, gráficas, modelo 3D

10.2. Arquitectura de tres capas

La guía 1 pide pensar el sistema en tres capas. Así quedaron:

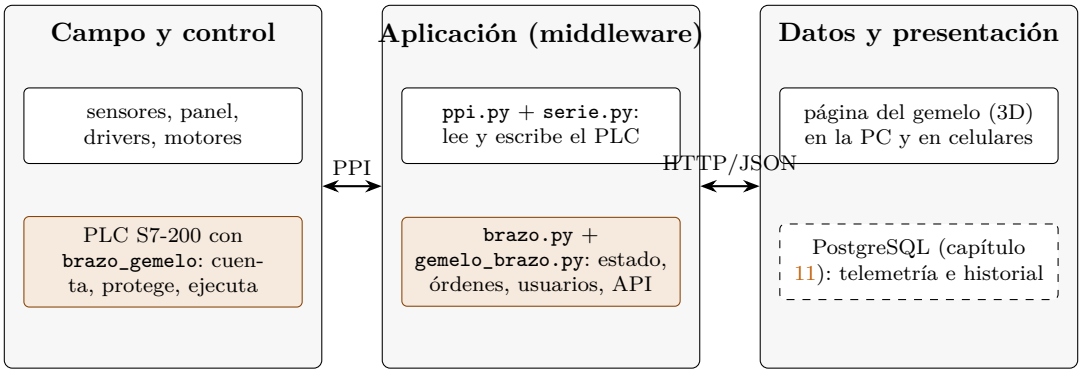


Figura 10.1: Arquitectura de tres capas del gemelo. La base de datos (línea punteada) está diseñada pero no conectada.

10.3. La app del gemelo

10.3.1. Módulos

Archivo	Responsabilidad
`gemelo_brazo.py`	programa principal: menú de modos, cambio de modo en caliente, servidor local (127.0.0.1:8050) y de red (0.0.0.0:8443 con HTTPS), sesiones, filtros de órdenes, administración de usuarios
`brazo.py`	clase `Brazo` (bucle de lectura, mando, registro), *perfiles* por programa del PLC, estimador, PLC simulado
`ppi.py`	cliente PPI (capítulo 9)
`serie.py`	puerto serie: termios en Linux, API Win32 por `ctypes` en Windows; lista de puertos
`acceso.py`, `usuarios.py`	contraseñas PBKDF2-SHA256 con 200 000 iteraciones y sal propia; sesiones de 8 h; bloqueo de 60 s tras 5 fallos
`certificado_tls.py`	genera el certificado HTTPS autofirmado en Python puro (RSA 2048, SHA-256)
`ventana.py`	ventana propia: GTK + WebKit (Linux) o Edge en modo aplicación (Windows)
`probar_plc.py`	diagnóstico de solo lectura
`menu.html`, `ui/`	interfaz: menú y página del gemelo (copia de `web/js` hecha por `empaquetar.py`)

10.3.2. Hilos de ejecución

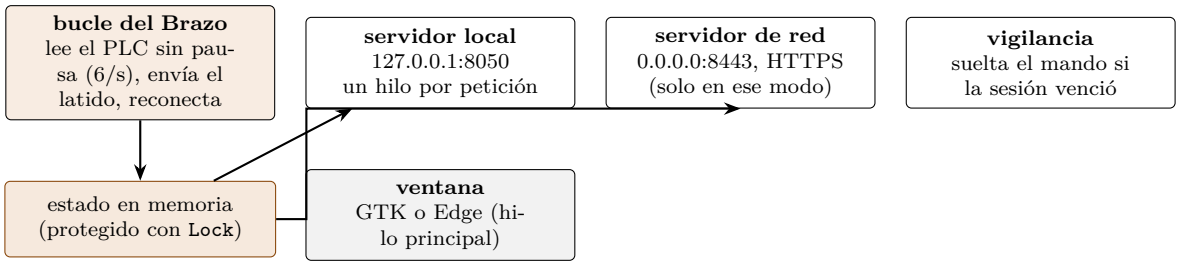


Figura 10.2: Hilos de la app. Las páginas nunca esperan al PLC: leen el último estado guardado.

10.3.3. Perfiles: la app se adapta al programa del PLC

Perfil	Se reconoce por	Lee	Órdenes
gemelo	VB1141 = `16#47`	40 bytes desde VB1120: posición contada, estado, error, E/S	homing, mover, garra, detener, velocidad
cargado	VD340/VD344 = 30000/25000	E/S, SMB66–85 (PTO), VB200, VB300 y VW320: estima la posición	iniciar, detener, emergencia, rearmar, garra
monitor	cualquier otro	E/S y PTO; estima la posición	ninguna

Listing 10.1: Lectura del estado con el perfil *gemelo* (`brazo.py`, fragmento).

```
def leer(self):
    v = self.cli.leer('V', 1120, 40)                    # una sola petición PPI
    estado, ack, flags, err = v[0], v[1], v[2], v[3]
    nz, nx, ny = struct.unpack('>iii', v[4:16])         # posición contada (big-endian)
    ib0, ib1, qb0 = v[18], v[19], v[20]
    sz, sx, sy = struct.unpack('>iii', v[24:36])        # tramo en curso
    # ... mientras un tramo está en marcha, se interpola su avance con el tiempo y el ciclo,
    # porque el PLC suma el tramo a la posición recién al terminarlo
```

10.3.4. El estimador (programa original)

Con el programa original el PLC no cuenta la posición. La app la estima en cada lectura:

1. si un PTO está ocupado, suma $f\,\Delta t$ pulsos, con $f = 10^6$/ciclo leído de SMW68/SMW78;
2. reparte: PTO0 → base; PTO1 → X si K1 está cerrado e Y si K2 lo está; el signo sale de la DIR;
3. los tramos largos (homing, secuencias) se ajustan al número exacto de pulsos al terminar;
4. el jog del panel se integra con un factor de trabajo (0,93, SUPUESTO) que se *autocalibra* cuando un carro va de un final de carrera al otro;
5. los finales de carrera y el inductivo corrigen la estimación al activarse.

Prueba automática (`pruebas_estimador.py`, con el PLC simulado imitando al programa original): error máximo de 1,35 mm en X/Y.

10.3.5. El PLC simulado

`PLCSimulado` imita al programa `brazo_gemelo` con la misma interfaz que el cliente PPI (`leer`, `escribir`): avanza los ejes a la frecuencia pedida, respeta finales de carrera, homing, tramo común/resto, errores y watchdog. Lo usan el modo *Simulación* y las pruebas.

10.3.6. Modos y menú

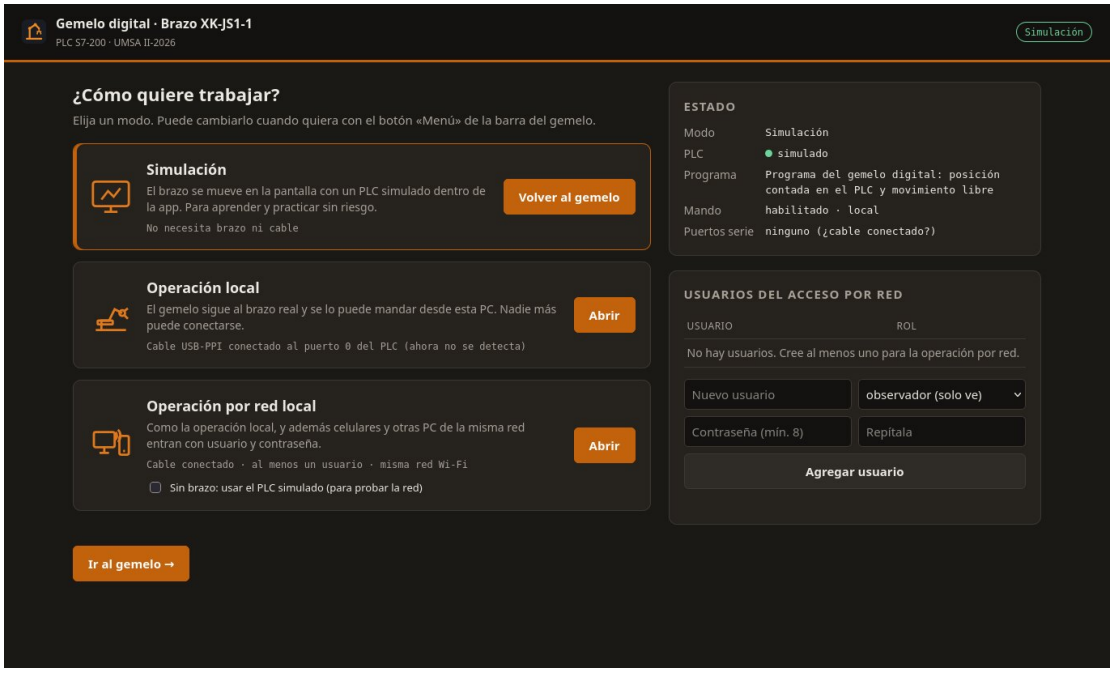


Figura 10.3: Menú de la app.

Listing 10.2: Cambio de modo (`gemelo_brazo.py`).

```
def aplicar_modo(modo, red_simulado=False):
    if modo == 'red' and not ACCESO.hay_usuarios():
        raise ValueError('para la operación por red cree antes al menos un usuario')
    simulado = modo == 'simulacion' or (modo == 'red' and red_simulado)
    with CAMBIO:
```

```
        if BRAZO is None or BRAZO.simulado != simulado:
            if BRAZO:
                BRAZO.cerrar()                  # detiene el brazo y suelta el puerto
            BRAZO = Brazo(Config.puerto, Config.direccion, simulado, REGISTRO)
        if modo == 'red':
            _activar_red()                      # servidor HTTPS en 0.0.0.0:8443
        else:
            _desactivar_red()
```

10.3.7. API HTTP

Ruta	Quién	Qué hace
`GET /api/estado`	sesión	posición, E/S, mando, conexión (6 veces por segundo)
`GET /api/info`	sesión	programa reconocido, órdenes admitidas, transmisión
`GET /api/sesion`	sesión	usuario, rol, conectados
`POST /api/mando`	operador	tomar o soltar el mando
`POST /api/orden/<nombre>`	operador con mando	homing, mover, garra, detener, velocidad, …
`POST /api/calibrar_base`	operador	calcula los pulsos por grado
`GET /menu, GET /api/app`	solo esta PC	menú y su estado
`POST /api/app/modo`	solo esta PC	cambiar de modo
`POST /api/app/usuarios`	solo esta PC	agregar, cambiar clave o rol, quitar
`GET/POST /login, GET /salir`	todos	ingreso y salida (acceso por red)

10.3.8. Seguridad y control de acceso por roles

- **Dos puertas**: la local (127.0.0.1) no sale a la red y no pide contraseña; la de red (8443) exige HTTPS, usuario y rol.
- **Roles** (RBAC): *observador* solo lee; *operador* puede tomar el mando. Un solo mando a la vez; cualquier operador puede dar paro.
- **Filtros de cada orden**: sesión válida, origen de la propia página (contra sitios ajenos), rol, mando, selector en automático, homing previo; y luego los límites del PLC.
- **Contraseñas**: PBKDF2-SHA256, 200 000 iteraciones, sal aleatoria por usuario; `usuarios.json` no se sube al repositorio.
- **Registro**: cada ingreso, orden aceptada u orden rechazada queda en `registro_gemelo.log`.

10.4. La página publicada

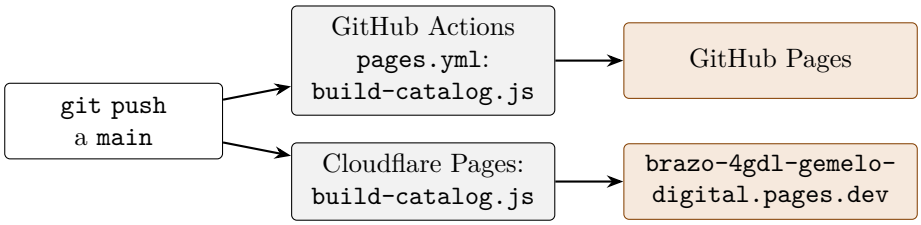


Figura 10.4: Publicación automática de la página en cada `push` a `main`.

`scripts/build-catalog.js` recorre `global/` y `semestres/` y genera `web/data/catalogo.js`, el índice de documentos, programas y fotos que muestra la página. Los títulos y descripciones se corrigen en `config/archivos.json`. La sección *Modelo 3D* de la página es la misma que usa la app (`web/js/modelo3d.js`, `cinematica.js`, `conexion-plc.js` y three.js 0.160).

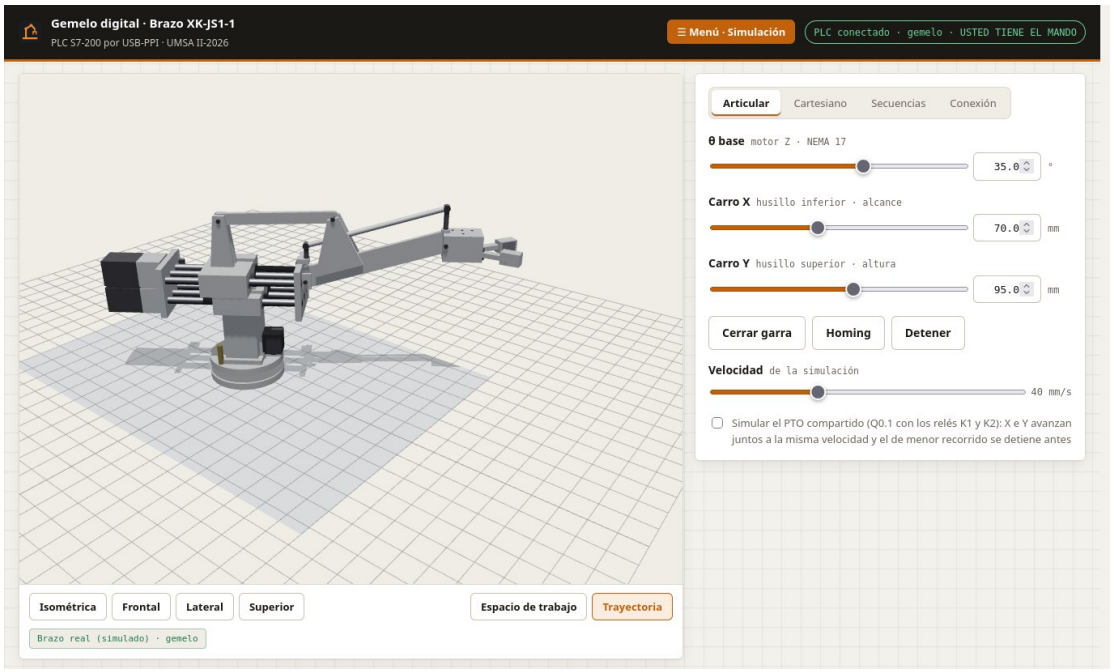


Figura 10.5: Página del gemelo dentro de la app (modo simulación).

Capítulo 11

Base de datos

11.1. Requerimientos

Del levantamiento del informe 1 (ETN-1000) y de lo que la app ya produce, la base debe guardar:

- la **telemetría** de las variables (PV, MV) con marca de tiempo: posiciones de los ejes, frecuencias;
- los **cambios de estado** de las señales digitales (finales de carrera, relés, garra);
- los **ciclos de operación**: cuándo y en qué modo se usó la planta y con qué programa;
- las **órdenes** (acciones de control): quién, cuándo, qué, con qué destinos, aceptada o rechazada y por qué;
- las **alarmas**: watchdog, paro, falla de homing, fuera de rango...;
- los **usuarios, roles e ingresos** (registro de sesiones).

11.2. Del modelo preliminar al modelo normalizado

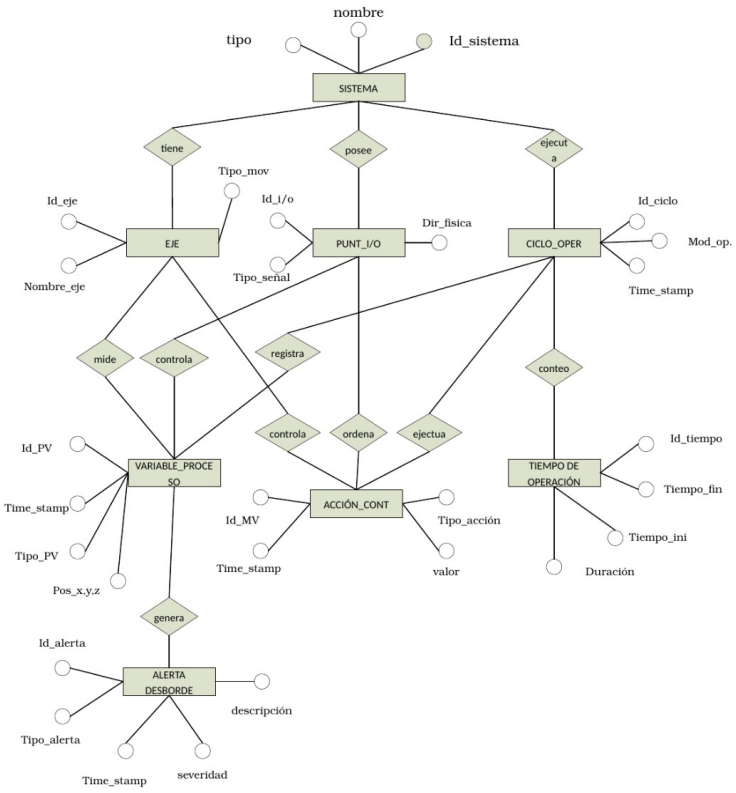


Figura 11.1: Modelo E-R preliminar del equipo (`base-de-datos/modelo/`).

Cambios para llevarlo a 3FN y a la planta real:

Preliminar	**Normalizado**
VARIABLE_PROCESO con *Pos_x.y.z*	1FN: un valor por fila; la variable (tag, unidad, rango) está en `variable` y cada medición en `muestra`
ACCION_CONT con *valor*	la orden en `orden`; sus destinos, uno por eje, en `orden_parametro` (1FN)
TIEMPO DE OPERACIÓN con *Duración*	3FN: la duración depende de inicio y fin; no se guarda, se calcula en la vista `v_ciclo`
ALERTA DESBORDE	catálogo `alarma` (código, severidad) + `evento_alarma` (cuándo se activó)
PUNT_I/O	`punto_io` con dirección del PLC, tag ISA y eje
(no existía)	`usuario`, `rol`, `sesion`: control de acceso y registro de ingresos (guías 3 y Fase 3)
(no existía)	`programa_plc`: qué programa estaba cargado en cada ciclo

11.3. Modelo lógico

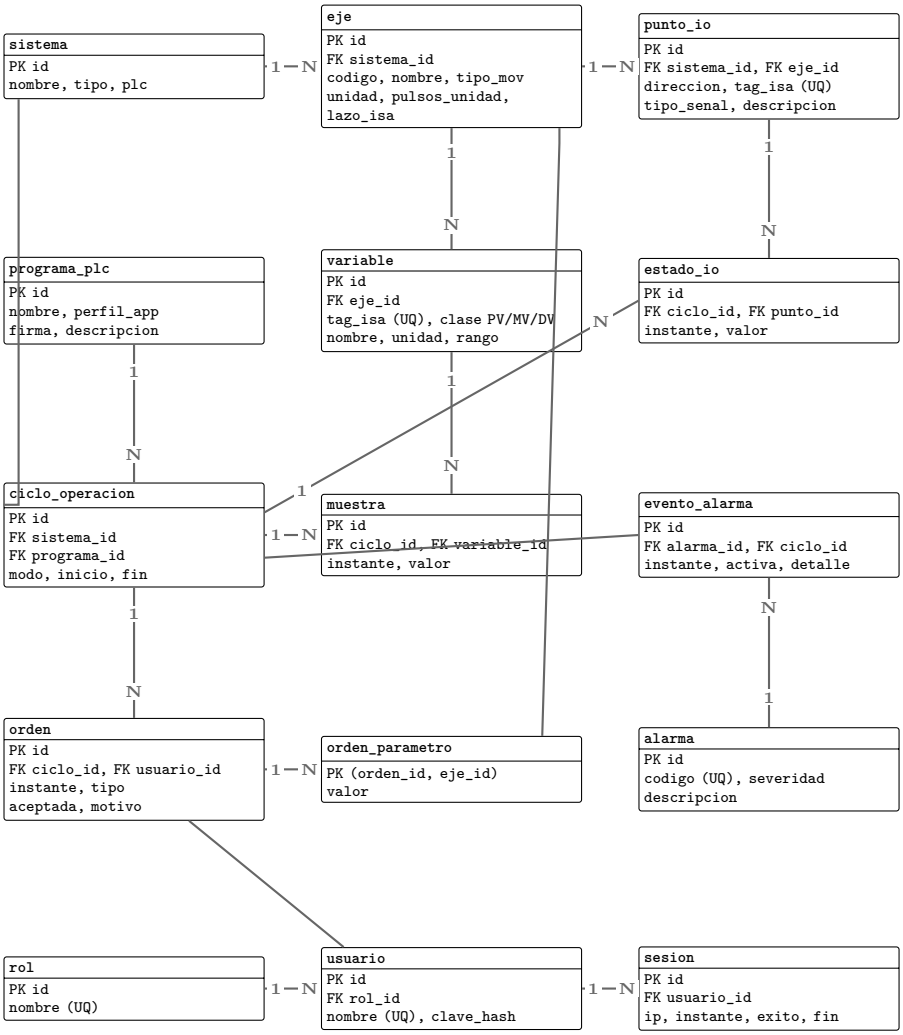


Figura 11.2: Modelo lógico normalizado (PostgreSQL). PK: clave primaria; FK: clave foránea; UQ: único; 1–N: cardinalidad.

11.4. Por qué está en tercera forma normal

- **1FN** (valores atómicos, sin grupos repetidos): la posición (X, Y, Z) no va en una celda; cada variable es una fila de `variable` y cada medición una fila de `muestra`. Los destinos de una orden van en `orden_parametro`, una fila por eje.
- **2FN** (sin dependencias parciales): todas las tablas tienen clave simple salvo `orden_parametro`, cuyo único atributo `valor` depende de la clave completa (orden y eje).
- **3FN** (sin dependencias transitivas): la unidad y el rango dependen de la variable, no de la muestra; el rol depende del usuario, no de la sesión; la duración de un ciclo depende de inicio y fin y por eso se calcula.

11.5. El script SQL

El script `base-de-datos/scripts/brazo_4gdl.sql` crea las 15 tablas con sus restricciones, un trigger, dos vistas y carga los datos fijos de la planta: sistema, 4 ejes, 22 señales del PLC con sus tags ISA, 5 variables, 3 programas, 8 alarmas y 2 roles. El listado completo está en el apéndice C.

Listing 11.1: Trigger que rechaza muestras fuera de rango (lo pide la guía 1 para la Fase 2).

```
CREATE FUNCTION validar_muestra() RETURNS trigger AS $$
DECLARE
    r variable%ROWTYPE;
BEGIN
    SELECT * INTO r FROM variable WHERE id = NEW.variable_id;
    IF NEW.valor < r.rango_min OR NEW.valor > r.rango_max THEN
        RAISE EXCEPTION 'muestra␣fuera␣de␣rango␣para␣%␣(%):␣%␣no␣está␣en␣[%,␣%]',
            r.tag_isa, r.unidad, NEW.valor, r.rango_min, r.rango_max;
    END IF;
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;
CREATE TRIGGER muestra_en_rango BEFORE INSERT OR UPDATE ON muestra
    FOR EACH ROW EXECUTE FUNCTION validar_muestra();
```

11.6. De la app a las tablas

Dato que ya produce la app	Tabla
cambio de modo en el menú	`ciclo_operacion` (inicio/fin, modo, programa reconocido)
`/api/estado`: `theta_deg`, `sX_mm`, `sY_mm`	`muestra` con las variables ZI-101, ZI-102, ZI-103
`/api/estado`: entradas y salidas	`estado_io` (solo los cambios)
línea `orden mover {...} (usuario)` del registro	`orden` + `orden_parametro`
línea `RECHAZADO ...`	`orden` con `aceptada = false` y su motivo
error del PLC (VB1123), pérdida de comunicación	`evento_alarma`
`usuarios.json`	`usuario` y `rol` (mismo hash PBKDF2)
líneas `ingreso de ...` / `ingreso fallido`	`sesion`

Listing 11.2: Consultas de ejemplo.

```
-- Posición del carro X en el último ciclo, cada segundo
SELECT date_trunc('second', m.instante) AS t, avg(m.valor) AS x_mm
FROM muestra m JOIN variable v ON v.id = m.variable_id
WHERE v.tag_isa = 'ZI-102' AND m.ciclo_id = (SELECT max(id) FROM ciclo_operacion)
GROUP BY 1 ORDER BY 1;

```

```sql
-- Órdenes rechazadas por usuario y motivo
SELECT u.nombre, o.motivo, count(*) FROM orden o LEFT JOIN usuario u ON u.id = o.usuario_id
WHERE NOT o.aceptada GROUP BY 1, 2 ORDER BY 3 DESC;
```

Pendiente. El script no se ejecutó todavía: la PC donde se escribió no tiene PostgreSQL. Falta: (1) crear la base y correr el script en la PC del laboratorio; (2) agregar a la app un módulo que inserte las muestras (por ejemplo, una por segundo y por eje), los cambios de E/S, las órdenes y los ingresos, con la biblioteca `psycopg`; (3) un tablero que consulte el historial.

Capítulo 12

Estructura del proyecto y control de versiones

12.1. Repositorios

- **Brazo-de-4-grados-de-libertad** (GitHub, `GabAndGom`): repositorio original del equipo, con el material de los semestres anteriores (2026_I) y del actual.
- **Brazo-4GDL-gemelo-digital** (GitHub, `GabAndGom`): derivado del anterior con todo su historial, agrega el gemelo digital. Publica la página en https://brazo-4gdl-gemelo-digital.pages.dev.

Regla del equipo: en el repositorio original no se modifica el material de la gestión I/2026; el trabajo nuevo va en `semestres/2026_II/`.

12.2. Árbol del repositorio

```
Brazo-4GDL-gemelo-digital/
|-- index.html, web/            pagina publicada: estilos, JS, imagenes
|   `-- js/  cinematica.js  modelo3d.js  conexion-plc.js  app.js
|-- scripts/build-catalog.js    genera el catalogo de archivos de la pagina
|-- config/                     titulos, semestres, videos (JSON)
|-- .github/workflows/pages.yml publicacion en GitHub Pages
|-- global/                     material que no depende del semestre
|   |-- componentes/            imagenes de cada componente
|   |-- datasheets/             PLC, drivers, sensores, reles
|   |-- fotos-planta/           fotos del brazo y del tablero
|   |-- manuales/               manual del fabricante, referencia ladder, MAP_SERV
|   |-- modelo-3d/              brazo_xk-js1-1.glb y su generador
|   |-- normativas/             ISA-5.1, IEC 60617
|   |-- plantillas/             plantilla LaTeX de informes
|   `-- programas-plc/          programa del fabricante, libreria MAP_SERV
`-- semestres/
    |-- 2026_I/                 gestion anterior (no se modifica)
    `-- 2026_II/
        |-- guias/              guias de laboratorio 1, 2 y 3
        |-- informes/           informe 1 (auditoria y requerimientos)
        |-- diagramas/          diagrama de bloques fenomenologico
        |-- planos/electrico/, planos/pid/   planos .dwg (historico)
        |-- base-de-datos/      mockups, modelo E-R, scripts SQL, interfaz
        |-- modelo-3d/          levantamiento, cinematica, informe y figuras
        |-- programas-plc/      programa original de secuencias (.awl, .mwp)
        |-- gemelo-digital/
        |   |-- app/            app unificada (Linux y Windows)
        |   |-- plc/            brazo_gemelo.awl y .mwp
        |   `-- manual/         manuales, arquitectura, guia y este manual
        |-- hmi/, pruebas/, evidencias/   (README con lo que va en cada una)
```

Carpeta pedida en la guía 3	Equivalente en el repositorio
`/documentacion/planos_isa/`	`semestres/2026_II/planos/` (eléctrico y P&ID)
`/documentacion/manuales_inversa/`	`semestres/2026_II/informes/`, `modelo-3d/levantamiento/`, `global/datasheets/`
`/database/scripts/`	`semestres/2026_II/base-de-datos/scripts/`

12.3. Archivos fuera del repositorio

Carpeta (en `ETN 1034/`)	Contenido
`gemelo_brazo_app/` y `.zip`	app exportada lista para usar, con los datos propios de la PC: usuarios, certificado, calibración, registro
`respaldo_vm/`	copias de los programas encontrados en la máquina virtual (programa cargado, «BRAZO MIGUEL ROBADO», programas de semestres anteriores)
máquina virtual «WINDOWS XP»	STEP 7-Micro/WIN; carpeta compartida `\\vboxsvr\MIS_COSAS`

Los datos propios de cada PC (`usuarios.json`, `tls_*.pem`, `registro_gemelo.log`) están en `.gitignore`: nunca se suben.

12.4. Flujo de trabajo con Git

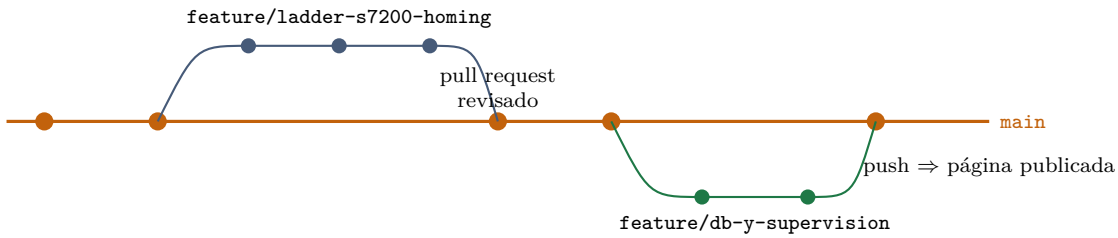


Figura 12.1: Ramas de trabajo que se integran a `main` con revisión.

```
git checkout -b feature/planos-v2-kaizen         # rama aislada para un componente
git add semestres/2026_II/planos/electrico/brazo_electrico_v2.dwg
git commit -m "Plano electrico v2.0: DIR+ de X e Y y garra corregidos"
git push origin feature/planos-v2-kaizen           # luego: Pull Request en GitHub
git log --oneline --graph                          # historial para el informe
```

Dato del historial	Valor (10/11/2026)
commits en `main`	49 y los de este manual
integrantes con commits	5 (Gabriel Gómez, José Lenin Mamani, Román Machicado, Eduardo Chura, . . .)
ramas de trabajo	`feature/ladder-s7200-homing`, `feature/db-y-supervision`, `feature/planos-v2-kaizen`, `feat`
primer commit	28/09/2026

12.5. Convenciones

- **Programas del PLC**: un archivo por versión con el número en el nombre (`brazo_secuencias_original_v1.a`
  exportar siempre también el AWL para que el código se pueda leer y comparar.
- **Mensajes de commit**: en castellano, en presente, diciendo qué cambia y por qué.
- **Fuentes y compilados**: los documentos se guardan en LaTeX con su PDF; los archivos auxiliares de compilación están en `.gitignore`.
- **Medidas**: cada dato lleva su origen (leída, estimada, SUPUESTO).

Capítulo 13

Variables medidas y resultados

13.1. Datos medidos o verificados en la planta

Magnitud	Valor	Cómo y cuándo
Eléctricas (informe 1, agosto de 2026, multímetro)		
entradas de la botonera	21,6 V accionado / 23,1 V reposo	medición pasiva en bornes
salida de la garra Q0.7	23,2 V / 0,2 V	ídem
rieles del PLC	5,1–23,7 V	ídem
Configuración (26/10/2026)		
DIP de los tres drivers	OFF OFF ON OFF ON ON ON OFF	foto del tablero
corriente de X e Y	2,37 A pico / 1,69 A RMS	tabla DM542
resolución	1000 pulsos/vuelta	tabla DM542/JMC
recorrido de los carros	170 mm (34 000 pulsos)	dato del equipo, coincide con la geometría
Mapeo de señales (26/10/2026, monitor PPI y accionamiento manual)		
finales de carrera	I0.1–I0.4	cada sensor accionado a mano
inductivo de la base	I0.0	ídem
sentido de X	Q0.3 = 0 adelante	jog con I1.2
Comunicación		
programa original	PPI dirección 6, 19,2 kbit/s	búsqueda automática, 26/10/2026
con `brazo_gemelo`	PPI dirección 2, 9,6 kbit/s	27/10/2026
lectura E/S + VB1000–1011 a 19,2 kbit/s	145 ms	cliente PPI, solo lectura
lectura de 40 bytes a 9,6 kbit/s	568 ms → 155 ms	antes y después del silencio entre tramas
Calibración		
base	190,3 pulsos/grado	giro medido con transportador, 26/10/2026 (`calibracion.json`)

13.2. Pruebas realizadas

Prueba	Resultado
Cinemática: directa seguida de inversa en toda la grilla de carreras (`pruebas_cinematica.js`)	devuelve el mismo q
Modelo frente a fotos del manual y de la planta	diferencia menor a 5 mm en los pivotes B y C

Prueba	Resultado
Estimador con el programa original (`pruebas_estimador.py`)	error máximo 1,35 mm en X/Y; autocalibración del jog
Compilación de `brazo_gemelo` en Micro/WIN	0 errores; cargado en el PLC
App con el brazo real (programa del gemelo)	conexión estable a 9,6 kbit/s, sin cortes
Cliente PPI contra un PLC falso (pseudoterminal)	lecturas, escrituras y protección de zonas correctas
Acceso por red: roles y filtros	observador rechazado al mover y al tomar el mando; operador sin mando rechazado; mover sin homing rechazado
App unificada en modo simulado	menú, alta de usuarios, cambio entre los tres modos, acceso remoto; el menú no es accesible desde la red
Certificado HTTPS generado en Python	validado con OpenSSL; conexión TLS correcta

13.3. Validación del gemelo

El hito de la Fase 2 pide gemelos validados en simulación. En el brazo:

- la geometría del modelo reproduce las fotos con menos de 5 mm de diferencia;
- el PLC simulado reproduce el comportamiento de `brazo_gemelo` (mismos pasos, errores y tiempos) y permite probar toda la app sin hardware;
- con el programa del gemelo cargado, la posición del modelo es la contada por el PLC (exacta al pulso mientras no se pierdan pasos).

Pendiente. Medir con regla o calibre el error de posición de la garra en varios puntos del espacio de trabajo y verificar el paso del husillo (SUPUESTO de 5 mm), las cotas estimadas del mecanismo y el sentido de la base.

Capítulo 14

Cumplimiento de los objetivos de las guías

Objetivo	Estado	Evidencia o lo que falta
Laboratorio 1: auditoría		
Clasificar variables PV, MV, DV	**cumplido**	capítulo 2, corregido con la planta en marcha
Ficha técnica del hardware	**cumplido**	capítulos 2 y 4
Ruptura de causalidad, lazo abierto	**cumplido**	sección del capítulo 2
Arquitectura de 3 capas	**cumplido**	capítulo 10
Laboratorio 2: planos e interfaces		
Plano eléctrico versión 2.0 en AutoCAD Electrical	**pendiente**	discrepancias documentadas (sección 2.8) y esquema funcional (capítulo 3); falta dibujar el `.dwg` v2.0
P&ID con tags ISA-5.1	**parcial**	tags propuestos (capítulo 2); falta el plano
Mockups HMI y dashboard	**parcial**	mockups del equipo en `base-de-datos/mockups/`; la app y la página son la interfaz real
Laboratorio 3: Git y modelo de datos		
Repositorio estructurado, ramas y pull requests	**cumplido**	capítulo 12
Modelo E-R normalizado en 3FN	**cumplido**	capítulo 11
Scripts en `/database/scripts/`	**parcial**	`brazo_4gdl.sql` escrito; falta ejecutarlo en PostgreSQL
Fase 2: modelado y backend		
Modelado y linealización del brazo	**cumplido**	capítulo 8: jacobiano y linealización con errores medidos
Redes de Petri	**cumplido**	sección 7.7
Triggers contra datos fuera de rango	**parcial**	trigger escrito; falta probarlo
Gemelo validado en simulación	**cumplido**	capítulo 13
Fase 3: implementación		
Programa del PLC con PTO y comunicación	**cumplido**	capítulo 7
Comunicación PLC–PC por tramas binarias	**cumplido**	capítulo 9 (PPI en lugar de Modbus, justificado)
Middleware en Python	**cumplido**	capítulo 10
Control de acceso por roles	**cumplido**	sección 10.3.8
Lazo cerrado en el hardware	**pendiente**	sin encoders: el brazo sigue en lazo abierto con supervisión discreta
Telemetría en PostgreSQL y dashboard	**pendiente**	diseño listo; falta conectar la app a la base

14.1. Siguientes pasos

1. Dibujar los planos versión 2.0 con las discrepancias corregidas.
2. Instalar PostgreSQL, ejecutar el script y agregar a la app el registro de telemetría.
3. Implementar rampas en el PTO y subir la corriente de X e Y.

4. Medir el error de posición de la garra y verificar los SUPUESTOS.
5. Agregar encoders (o el contador rápido HSC del PLC) para cerrar el lazo de posición.

Capítulo A

Programa original de secuencias (AWL completo)

Archivo `semestres/2026_II/programas-plc/brazo_secuencias_original_v1.awl`.

```
ORGANIZATION_BLOCK MAIN:OB1
TITLE=COMENTARIOS DEL PROGRAMA
BEGIN
Network 1 // Título de segmento
// Comentario de segmento
LD     SM0.1
MOVB   0, VB104
LD     V104.7
EU
LD     V104.7
ED
OLD
=      M5.3
LDN    M5.3
TON    T61, +50
LD     T61
MOVB   0, VB104
LD     I0.7
O      V104.0
AN     V104.2
AN     V104.6
=      M4.0
LD     M4.0
EU
=      M5.0
LD     V104.3
ON     V104.4
A      I0.5
=      M4.3
LD     I0.6
O      V104.5
=      M4.5
LD     V104.2
O      V104.6
=      M4.6
LD     M4.6
S      M4.7, 1
LD     V100.3
EU
=      M5.1
LD     I0.2
EU
=      M5.5
LD     I0.4
EU
=      M5.6
Network 2
LD     M4.6
LPS
MOVB   16#0, SMB67
PLS    0
MOVB   16#0, SMB77
PLS    1
R      Q0.0, 7
MOVW   +0, VW320
MOVW   +0, VW322
LRD
MOVB   16#0, VB301
R      M1.0, 1
R      M0.5, 1
S      M0.0, 1
LPP
R      M0.3, 1
R      V302.0, 1
LD     M4.7
AN     M4.6
LD     M5.0
O      M5.1
ALD
R      M4.7, 1
R      M5.0, 1
Network 3
LD     SM0.1
R      Q0.0, 8
R      M0.0, 32
MOVB   16#0, VB100
MOVB   16#0, VB104
MOVB   16#0, VB200
MOVB   16#0, VB300
MOVB   16#0, VB301
```

```
MOVD   +0, VD304
MOVD   +0, VD308
MOVW   +0, VW320
MOVW   +0, VW322
S      M0.0, 1
R      M4.6, 2
R      M5.0, 8
MOVD   +30000, VD340
MOVD   +25000, VD344
MOVD   +45000, VD348
MOVD   +20000, VD330
MOVD   +14000, VD334
Network 4
LD     M4.5
EU
O      V300.2
O      I1.0
O      V104.1
A      M4.3
R      Q0.0, 7
MOVB   16#0, SMB67
MOVB   16#0, SMB77
PLS    0
PLS    1
MOVW   +0, VW320
MOVW   +0, VW322
MOVB   16#0, VB301
R      V300.2, 1
R      M1.0, 1
S      M0.0, 1
R      M0.3, 1
R      M0.5, 1
R      M5.0, 1
R      M5.2, 1
R      M0.4, 1
Network 5
LDN    M4.3
EU
MOVB   16#0, SMB67
PLS    0
MOVB   16#0, SMB77
PLS    1
R      Q0.0, 7
MOVW   +0, VW320
MOVW   +0, VW322
R      M0.3, 1
R      M0.5, 1
MOVB   16#0, VB301
R      M1.0, 1
LDN    M4.3
AN     M4.6
AN     M4.7
CALL   SBR20
Network 6
LD     M4.3
EU
MOVB   16#0, SMB67
PLS    0
MOVB   16#0, SMB77
PLS    1
R      Q0.0, 7
MOVW   +0, VW320
MOVW   +0, VW322
S      M0.0, 1
R      M0.5, 1
LD     M4.3
AN     M4.6
AN     M4.7
CALL   SBR21
Network 7
LD     V100.0
EU
A      M4.3
AN     M4.6
AN     M4.7
S      M1.0, 1
LD     V100.2
EU
S      V300.2, 1
Network 8
LD     M4.3
AW=    VW320, +0
AW=    VW322, +0
AN     M4.6
AN     M4.7
AN     M5.2
=      V200.0
LDW>   VW320, +0
OW>    VW322, +0
=      V200.1
LD     V301.5
=      V200.2
LD     M5.2
=      V200.3
LD     SM0.0
```

```
R      V200.4, 2
LD     V301.6
=      V200.6
LD     M0.3
=      V200.7
LD     SM0.0
MOVB   VB321, VB201
MOVB   IB0, VB202
MOVB   VB323, VB203
Network 9
LD     I0.2
AN     Q0.3
LD     I0.1
A      Q0.3
OLD
A      M4.3
A      Q0.5
AN     SM76.7
MOVB   16#0, SMB77
PLS    1
S      M1.1, 1
Network 10
LD     SM0.0
MOVD   VD210, VD304
MOVD   VD220, VD308
END_ORGANIZATION_BLOCK
SUBROUTINE_BLOCK SBR_HOMING:SBRO
TITLE=COMENTARIOS DE LA SUBRUTINA
BEGIN
Network 1 // Título de segmento
// Comentario de segmento
LD     M0.0
A      M0.5
AN     M5.2
AW=    VW322, +0
S      V301.0, 1
R      V301.1, 1
MOVW   +10, VW322
Network 2
LDW=   VW322, +10
R      Q0.6, 1
S      Q0.5, 1
S      Q0.3, 1
TON    T34, +5
LD     T34
MOVW   +11, VW322
Network 3
LDW=   VW322, +11
LPS
EU
MOVB   16#85, SMB77
MOVW   +250, SMW78
MOVD   VD340, SMD82
PLS    1
LPP
LDW=   VW322, +11
A      I0.1
LPS
MOVB   16#0, SMB77
PLS    1
LRD
R      Q0.5, 1
LPP
MOVW   +20, VW322
Network 4
LDW=   VW322, +20
LPS
EU
S      Q0.2, 1
MOVB   16#85, SMB67
MOVW   +400, SMW68
MOVD   VD348, SMD72
PLS    0
LPP
LDW=   VW322, +20
A      I0.0
LPS
MOVB   16#0, SMB67
PLS    0
LPP
MOVW   +30, VW322
Network 5
LDW=   VW322, +30
R      Q0.5, 1
S      Q0.6, 1
S      Q0.4, 1
TON    T33, +5
LD     T33
MOVW   +31, VW322
Network 6
LDW=   VW322, +31
LPS
EU
MOVB   16#85, SMB77
MOVW   +250, SMW78
```

```
MOVD   VD344, SMD82
PLS    1
LPP
LDW=   VW322, +31
A      I0.3
LPS
MOVB   16#0, SMB77
PLS    1
LRD
R      Q0.6, 1
LPP
MOVW   +40, VW322
Network 7
LDW=   VW322, +40
R      Q0.7, 1
TON    T37, +4
LD     T37
MOVW   +50, VW322
Network 8
LDW=   VW322, +50
MOVD   +0, VD210
MOVD   +0, VD220
MOVD   +0, VD304
MOVD   +0, VD308
R      M0.0, 1
S      M0.3, 1
R      M0.4, 1
R      M0.5, 1
R      V301.0, 1
S      V301.1, 1
MOVW   +0, VW322
Network 9
LDW=   VW322, +11
TON    T58, +120
LDW=   VW322, +20
TON    T59, +250
LDW=   VW322, +31
TON    T60, +120
LDW=   VW322, +11
A      T58
LDW=   VW322, +20
A      T59
OLD
LDW=   VW322, +31
A      T60
OLD
LDW=   VW322, +11
A      SM76.7
AN     I0.1
LDW>   T58, +3
ALD
OLD
LDW=   VW322, +20
A      SM66.7
AN     I0.0
LDW>   T59, +3
ALD
OLD
LDW=   VW322, +31
A      SM76.7
AN     I0.3
LDW>   T60, +3
ALD
OLD
LDW=   VW322, +11
A      M5.5
OLD
LDW=   VW322, +31
A      M5.6
OLD
LPS
MOVB   16#0, SMB67
PLS    0
MOVB   16#0, SMB77
PLS    1
LRD
R      Q0.5, 2
MOVW   +0, VW322
R      M0.5, 1
LPP
S      M5.2, 1
R      V301.0, 1
END_SUBROUTINE_BLOCK
SUBROUTINE_BLOCK SBR_MANUAL:SBR20
TITLE=
BEGIN
Network 1
LD     M5.0
LPS
AN     Q0.7
S      M10.0, 1
LRD
A      Q0.7
R      Q0.7, 1
LPP
```

```
A      M10.0
S      Q0.7, 1
R      M10.0, 1
Network 2
LD     I1.0
AN     I1.1
A      SM66.7
R      Q0.2, 1
MOVB   16#85, SMB67
MOVW   +250, SMW68
MOVD   +100, SMD72
PLS    0
Network 3
LD     I1.1
AN     I1.0
A      SM66.7
S      Q0.2, 1
MOVB   16#85, SMB67
MOVW   +250, SMW68
MOVD   +100, SMD72
PLS    0
Network 4
LDN    I1.0
AN     I1.1
EU
MOVB   16#0, SMB67
PLS    0
Network 5
LD     I1.2
AN     I1.3
AN     I0.2
AN     Q0.6
LPS
S      Q0.5, 1
R      Q0.3, 1
TON    T35, +3
LPP
LD     T35
A      I1.2
AN     I0.2
A      SM76.7
MOVB   16#85, SMB77
MOVW   +250, SMW78
MOVD   +100, SMD82
PLS    1
Network 6
LD     I1.3
AN     I1.2
AN     I0.1
AN     Q0.6
LPS
S      Q0.5, 1
S      Q0.3, 1
TON    T36, +3
LPP
LD     T36
A      I1.3
AN     I0.1
A      SM76.7
MOVB   16#85, SMB77
MOVW   +250, SMW78
MOVD   +100, SMD82
PLS    1
Network 7
LD     I1.4
AN     I1.5
AN     I0.4
AN     Q0.5
LPS
S      Q0.6, 1
R      Q0.4, 1
TON    T97, +3
LPP
LD     T97
A      I1.4
AN     I0.4
A      SM76.7
MOVB   16#85, SMB77
MOVW   +250, SMW78
MOVD   +100, SMD82
PLS    1
Network 8
LD     I1.5
AN     I1.4
AN     I0.3
AN     Q0.5
LPS
S      Q0.6, 1
S      Q0.4, 1
TON    T98, +3
LPP
LD     T98
A      I1.5
AN     I0.3
A      SM76.7
```

```
MOVB   16#85, SMB77
MOVW   +250, SMW78
MOVD   +100, SMD82
PLS    1
Network 9
LDN    I1.2
AN     I1.3
AN     I1.4
AN     I1.5
LPS
EU
MOVB   16#0, SMB77
PLS    1
LPP
TON    T99, +3
LD     T99
R      Q0.5, 1
R      Q0.6, 1
END_SUBROUTINE_BLOCK
SUBROUTINE_BLOCK SBR_AUTO:SBR21
TITLE=
BEGIN
Network 1
LDN    M0.3
R      V302.0, 1
R      V301.6, 1
MOVW   +0, VW320
LD     M5.0
O      M1.0
AN     M0.3
AN     M5.2
S      M0.0, 1
S      M0.5, 1
R      M5.0, 1
LDN    M0.3
CALL   SBR0
CRET
Network 2
LD     M5.0
O      M1.0
A      M0.3
AN     V302.0
AW=    VW320, +0
S      V301.2, 1
R      V301.1, 1
R      V301.5, 1
R      V301.6, 1
MOVW   +10, VW320
Network 3
LDW=   VW320, +10
S      Q0.6, 1
R      Q0.5, 1
R      Q0.4, 1
TON    T38, +2
LD     T38
MOVW   +11, VW320
Network 4
LDW=   VW320, +11
MOVB   16#85, SMB77
MOVW   +250, SMW78
MOVD   +12000, SMD82
PLS    1
MOVW   +12, VW320
Network 5
LDW=   VW320, +12
TON    T39, +1
LDW=   VW320, +12
A      T39
A      SM76.7
R      Q0.6, 1
MOVW   +20, VW320
Network 6
LDW=   VW320, +20
R      Q0.2, 1
MOVB   16#85, SMB67
MOVW   +250, SMW68
MOVD   +17000, SMD72
PLS    0
MOVW   +21, VW320
Network 7
LDW=   VW320, +21
TON    T62, +1
LDW=   VW320, +21
A      T62
A      SM66.7
MOVW   +30, VW320
Network 8
LDW=   VW320, +30
S      Q0.5, 1
R      Q0.6, 1
R      Q0.3, 1
TON    T40, +2
LD     T40
MOVW   +31, VW320
Network 9
```

```
LDW=   VW320, +31
MOVB   16#85, SMB77
MOVW   +250, SMW78
MOVD   VD330, SMD82
PLS    1
MOVW   +32, VW320
Network 10
LDW=   VW320, +32
TON    T41, +1
LDW=   VW320, +32
A      T41
A      SM76.7
R      Q0.5, 1
MOVW   +40, VW320
Network 11
LDW=   VW320, +40
R      Q0.7, 1
R      V301.2, 1
R      V301.6, 1
S      V301.3, 1
S      V301.1, 1
S      V302.0, 1
MOVW   +0, VW320
Network 12
LD     M5.0
O      M1.0
A      M0.3
A      V302.0
AW=    VW320, +0
S      V301.4, 1
R      V301.1, 1
R      V301.6, 1
R      M1.0, 1
R      V301.5, 1
MOVW   +100, VW320
Network 13
LDW=   VW320, +100
S      Q0.7, 1
TON    T42, +4
LD     T42
MOVW   +110, VW320
Network 14
LDW=   VW320, +110
S      Q0.6, 1
R      Q0.5, 1
R      Q0.4, 1
TON    T43, +2
LD     T43
MOVW   +111, VW320
Network 15
LDW=   VW320, +111
MOVB   16#85, SMB77
MOVW   +250, SMW78
MOVD   +5000, SMD82
PLS    1
MOVW   +112, VW320
Network 16
LDW=   VW320, +112
TON    T44, +1
LDW=   VW320, +112
A      T44
A      SM76.7
R      Q0.6, 1
S      V301.6, 1
MOVW   +120, VW320
Network 17
LDW=   VW320, +120
S      Q0.5, 1
R      Q0.6, 1
S      Q0.3, 1
TON    T45, +2
LD     T45
MOVW   +121, VW320
Network 18
LDW=   VW320, +121
MOVB   16#85, SMB77
MOVW   +250, SMW78
MOVD   VD334, SMD82
PLS    1
MOVW   +122, VW320
Network 19
LDW=   VW320, +122
TON    T46, +1
LDW=   VW320, +122
A      T46
A      SM76.7
R      Q0.5, 1
MOVW   +130, VW320
Network 20
LDW=   VW320, +130
R      Q0.2, 1
MOVB   16#85, SMB67
MOVW   +250, SMW68
MOVD   +18000, SMD72
PLS    0
```

```
MOVW   +131, VW320
Network 21
LDW=   VW320, +131
TON    T47, +1
LDW=   VW320, +131
A      T47
A      SM66.7
MOVW   +140, VW320
Network 22
LDW=   VW320, +140
S      Q0.6, 1
R      Q0.5, 1
S      Q0.4, 1
TON    T48, +2
LD     T48
MOVW   +141, VW320
Network 23
LDW=   VW320, +141
MOVB   16#85, SMB77
MOVW   +250, SMW78
MOVD   +14000, SMD82
PLS    1
MOVW   +142, VW320
Network 24
LDW=   VW320, +142
TON    T49, +1
LDW=   VW320, +142
A      T49
A      SM76.7
R      Q0.6, 1
MOVW   +150, VW320
Network 25
LDW=   VW320, +150
R      Q0.7, 1
TON    T50, +4
LD     T50
MOVW   +160, VW320
Network 26
LDW=   VW320, +160
S      Q0.6, 1
R      Q0.5, 1
R      Q0.4, 1
TON    T51, +2
LD     T51
MOVW   +161, VW320
Network 27
LDW=   VW320, +161
MOVB   16#85, SMB77
MOVW   +250, SMW78
MOVD   +14000, SMD82
PLS    1
MOVW   +162, VW320
Network 28
LDW=   VW320, +162
TON    T52, +1
LDW=   VW320, +162
A      T52
A      SM76.7
R      Q0.6, 1
MOVW   +170, VW320
Network 29
LDW=   VW320, +170
S      Q0.2, 1
MOVB   16#85, SMB67
MOVW   +250, SMW68
MOVD   +18000, SMD72
PLS    0
MOVW   +171, VW320
Network 30
LDW=   VW320, +171
TON    T53, +1
LDW=   VW320, +171
A      T53
A      SM66.7
MOVW   +180, VW320
Network 31
LDW=   VW320, +180
S      Q0.5, 1
R      Q0.6, 1
R      Q0.3, 1
TON    T54, +2
LD     T54
MOVW   +181, VW320
Network 32
LDW=   VW320, +181
MOVB   16#85, SMB77
MOVW   +250, SMW78
MOVD   VD334, SMD82
PLS    1
MOVW   +182, VW320
Network 33
LDW=   VW320, +182
TON    T55, +1
LDW=   VW320, +182
A      T55
```

```
A      SM76.7
R      Q0.5, 1
MOVW   +190, VW320
Network 34
LDW=   VW320, +190
S      Q0.6, 1
R      Q0.5, 1
S      Q0.4, 1
TON    T56, +2
LD     T56
MOVW   +191, VW320
Network 35
LDW=   VW320, +191
MOVB   16#85, SMB77
MOVW   +250, SMW78
MOVD   +5000, SMD82
PLS    1
MOVW   +192, VW320
Network 36
LDW=   VW320, +192
TON    T57, +1
LDW=   VW320, +192
A      T57
A      SM76.7
R      Q0.6, 1
MOVW   +200, VW320
Network 37
LDW=   VW320, +200
R      Q0.7, 1
R      V301.4, 1
S      V301.5, 1
S      V301.1, 1
S      V302.0, 1
MOVW   +0, VW320
END_SUBROUTINE_BLOCK
```

Capítulo B

Programa `brazo_gemelo` (AWL completo)

Archivo `semestres/2026_II/gemelo-digital/plc/brazo_gemelo.awl`.

```
ORGANIZATION_BLOCK MAIN:OB1
TITLE=Brazo XK-JS1-1 - programa para el gemelo digital (v1)
// Basado en "BRAZO MIGUEL ROBADO": mismo modo manual (selector I0.5 = 0) y mismo homing.
// Agrega: cuenta de pulsos por eje, finales de carrera que cortan el PTO y mando remoto
// desde la PC por PPI (selector I0.5 = 1), con watchdog.
//
// Mando (lo escribe la PC):      Estado (lo escribe el PLC):
//  VB1100 orden 1 homing 2 mover  VB1120 estado 0 reposo 1 homing 2 moviendo
//         3 paro 4 cerrar garra    VB1121 eco de la secuencia
//         5 abrir garra            VB1122 .0 referenciado .1 remoto .2 PTO0 ocupado
//  VB1101 secuencia (+1 por orden)        .3 PTO1 ocupado .4 posicion incierta .5 watchdog
//  VW1102 latido (la PC lo cambia) VB1123 error 1 ocupado 2 fuera de rango 3 watchdog
//  VD1104 pulsos base (con signo)         4 sin referencia 5 orden desconocida 6 paro
//  VD1108 pulsos X (con signo)     VD1124 / VD1128 / VD1132 posicion base / X / Y [pulsos]
//  VD1112 pulsos Y (con signo)     VW1136 paso   VB1138 IB0  VB1139 IB1  VB1140 QB0
//  VW1116 ciclo X/Y [us]           VB1141 firma 16#47
//  VW1118 ciclo base [us]          VD1144 / VD1148 / VD1152 tramo en curso base / X / Y
//                                  VW1156 / VW1158 ciclo en curso X/Y / base [us]
// Sentidos: X+ con Q0.3 = 0 (adelante), Y+ con Q0.4 = 0 (arriba), base+ con Q0.2 = 0 (SUPUESTO).
// Recorridos: X e Y 0..34000 pulsos (170 mm a 0.005 mm/pulso); base +-50000 pulsos (SUPUESTO).
BEGIN
Network 1 // Primer ciclo: salidas y PTO apagados, memoria de mando y estado en cero
LD     SM0.1
R      Q0.0, 8
MOVB   16#0, SMB67
MOVB   16#0, SMB77
FILL   +0, VW1100, 55
MOVW   +250, VW1116
MOVW   +400, VW1118

Network 2 // Firma del programa (la app la usa para reconocerlo)
LD     SM0.0
MOVB   16#47, VB1141

Network 3 // Paro: pulsador I0.6 o cambio del selector manual/remoto I0.5
LD     I0.6
EU
LD     I0.5
EU
OLD
LD     I0.5
ED
OLD
MOVB   6, VB1123
CALL   SBR0

Network 4 // Final X atras (I0.1) con X yendo hacia atras: corta Q0.1 y X = 0
LD     I0.1
A      Q0.5
A      Q0.3
AN     SM76.7
LPS
MOVB   16#0, SMB77
PLS    1
R      V1200.1, 1
MOVD   +0, VD1128
LPP
A      Q0.6
S      V1200.2, 1

Network 5 // Final X adelante (I0.2) con X yendo hacia adelante: corta Q0.1 y X = 34000
LD     I0.2
A      Q0.5
AN     Q0.3
AN     SM76.7
LPS
MOVB   16#0, SMB77
PLS    1
R      V1200.1, 1
MOVD   +34000, VD1128
LPP
A      Q0.6
S      V1200.2, 1

Network 6 // Final Y abajo (I0.3) con Y bajando: corta Q0.1 e Y = 0
LD     I0.3
A      Q0.6
A      Q0.4
AN     SM76.7
LPS
MOVB   16#0, SMB77
PLS    1
R      V1200.1, 1
```

```
MOVD   +0, VD1132
LPP
A      Q0.5
S      V1200.2, 1

Network 7 // Final Y arriba (I0.4) con Y subiendo: corta Q0.1 e Y = 34000
LD     I0.4
A      Q0.6
AN     Q0.4
AN     SM76.7
LPS
MOVB   16#0, SMB77
PLS    1
R      V1200.1, 1
MOVD   +34000, VD1132
LPP
A      Q0.5
S      V1200.2, 1

Network 8 // Fin de un tramo de la base: suma sus pulsos a la posicion
LD     SM66.7
EU
A      V1200.0
+D     VD1144, VD1124
R      V1200.0, 1

Network 9 // Fin de un tramo de Q0.1: suma los pulsos a X y/o Y segun los reles
LD     SM76.7
EU
A      V1200.1
+D     VD1148, VD1128
+D     VD1152, VD1132
R      V1200.1, 1

Network 10 // Modo manual (panel), igual que el programa de Miguel
LDN    I0.5
CALL   SBR3

Network 11 // Modo remoto (ordenes de la PC)
LD     I0.5
CALL   SBR4

Network 12 // Copia de entradas y salidas para la PC
LD     SM0.0
MOVB   IB0, VB1138
MOVB   IB1, VB1139
MOVB   QB0, VB1140

Network 13 // Banderas de estado
LD     V1200.3
=      V1122.0
LD     I0.5
=      V1122.1
LDN    SM66.7
=      V1122.2
LDN    SM76.7
=      V1122.3
LD     V1200.2
=      V1122.4
LD     V1200.5
=      V1122.5
END_ORGANIZATION_BLOCK
SUBROUTINE_BLOCK PARO:SBR0
TITLE=Paro: corta los dos PTO, abre los reles y vuelve al reposo
BEGIN
Network 1 // Corta los trenes de pulsos
LD     SM0.0
MOVB   16#0, SMB67
PLS    0
MOVB   16#0, SMB77
PLS    1

Network 2 // Si habia un tramo en curso la posicion ya no es exacta: pide homing
LD     V1200.0
O      V1200.1
S      V1200.2, 1
R      V1200.3, 1

Network 3 // Reposo
LD     SM0.0
R      V1200.0, 2
R      V1200.6, 2
R      V1201.0, 1
R      Q0.5, 2
MOVW   +0, VW1136
MOVB   0, VB1120
END_SUBROUTINE_BLOCK
SUBROUTINE_BLOCK PULSO_BASE:SBR1
TITLE=Inicia un tramo en Q0.0: pulsos en VD1202, ciclo en VW1198 (us)
BEGIN
Network 1 // Arranca el PTO0 y guarda el tramo con su signo (Q0.2 = 1 resta)
LD     SM0.0
MOVB   16#85, SMB67
MOVW   VW1198, SMW68
MOVD   VD1202, SMD72
```

```
PLS    0
MOVW   VW1198, VW1158
MOVD   VD1202, VD1144
S      V1200.0, 1

Network 2
LD     Q0.2
MOVD   +0, VD1144
-D     VD1202, VD1144
END_SUBROUTINE_BLOCK
SUBROUTINE_BLOCK PULSO_XY:SBR2
TITLE=Inicia un tramo en Q0.1: pulsos en VD1202, ciclo en VW1196 (us)
BEGIN
Network 1 // Arranca el PTO1; los pulsos van a X si K1 esta cerrado y a Y si K2 esta cerrado
LD     SM0.0
MOVB   16#85, SMB77
MOVW   VW1196, SMW78
MOVD   VD1202, SMD82
PLS    1
MOVW   VW1196, VW1156
MOVD   +0, VD1148
MOVD   +0, VD1152
S      V1200.1, 1

Network 2 // X adelante
LD     Q0.5
AN     Q0.3
MOVD   VD1202, VD1148

Network 3 // X atras
LD     Q0.5
A      Q0.3
-D     VD1202, VD1148

Network 4 // Y arriba
LD     Q0.6
AN     Q0.4
MOVD   VD1202, VD1152

Network 5 // Y abajo
LD     Q0.6
A      Q0.4
-D     VD1202, VD1152
END_SUBROUTINE_BLOCK
SUBROUTINE_BLOCK MANUAL:SBR3
TITLE=Modo manual con los pulsadores del panel (como el programa de Miguel)
// Diferencias con el original: cada paquete de 100 pulsos se termina (no se corta al soltar)
// para que la cuenta sea exacta, y la direccion se fija antes de dar pulsos.
BEGIN
Network 1 // I0.7: abre o cierra la garra
LD     I0.7
EU
LPS
AN     Q0.7
S      M10.0, 1
LRD
A      Q0.7
R      Q0.7, 1
LPP
A      M10.0
S      Q0.7, 1
R      M10.0, 1

Network 2 // Base con I1.0: direccion Q0.2 = 0
LD     I1.0
AN     I1.1
A      SM66.7
R      Q0.2, 1

Network 3
LD     I1.0
AN     I1.1
TON    T106, +1

Network 4 // Paquete de 100 pulsos a 2.5 kHz (la base arranca sin rampa: mas suave que a 4 kHz)
LD     T106
AN     Q0.2
A      SM66.7
MOVD   +100, VD1202
MOVW   +400, VW1198
CALL   SBR1

Network 5 // Base con I1.1: direccion Q0.2 = 1
LD     I1.1
AN     I1.0
A      SM66.7
S      Q0.2, 1

Network 6
LD     I1.1
AN     I1.0
TON    T107, +1

Network 7
LD     T107
```

```
A      Q0.2
A      SM66.7
MOVD   +100, VD1202
MOVW   +400, VW1198
CALL   SBR1

Network 8 // X adelante con I1.2: cierra K1, Q0.3 = 0
LD     I1.2
AN     I1.3
AN     Q0.6
A      SM76.7
S      Q0.5, 1
R      Q0.3, 1

Network 9
LD     I1.2
AN     I1.3
TON    T101, +2

Network 10
LD     T101
A      Q0.5
AN     Q0.3
AN     Q0.6
AN     I0.2
A      SM76.7
MOVD   +100, VD1202
MOVW   +250, VW1196
CALL   SBR2

Network 11 // X atras con I1.3: cierra K1, Q0.3 = 1
LD     I1.3
AN     I1.2
AN     Q0.6
A      SM76.7
S      Q0.5, 1
S      Q0.3, 1

Network 12
LD     I1.3
AN     I1.2
TON    T102, +2

Network 13
LD     T102
A      Q0.5
A      Q0.3
AN     Q0.6
AN     I0.1
A      SM76.7
MOVD   +100, VD1202
MOVW   +250, VW1196
CALL   SBR2

Network 14 // Y arriba con I1.4: cierra K2, Q0.4 = 0
LD     I1.4
AN     I1.5
AN     Q0.5
A      SM76.7
S      Q0.6, 1
R      Q0.4, 1

Network 15
LD     I1.4
AN     I1.5
TON    T103, +2

Network 16
LD     T103
A      Q0.6
AN     Q0.4
AN     Q0.5
AN     I0.4
A      SM76.7
MOVD   +100, VD1202
MOVW   +250, VW1196
CALL   SBR2

Network 17 // Y abajo con I1.5: cierra K2, Q0.4 = 1
LD     I1.5
AN     I1.4
AN     Q0.5
A      SM76.7
S      Q0.6, 1
S      Q0.4, 1

Network 18
LD     I1.5
AN     I1.4
TON    T104, +2

Network 19
LD     T104
A      Q0.6
A      Q0.4
```

```
AN      Q0.5
AN      I0.3
A       SM76.7
MOVD    +100, VD1202
MOVW    +250, VW1196
CALL    SBR2

Network 20 // Sin pulsadores de X/Y y con Q0.1 quieto: abre los reles despues de 300 ms
LDN     I1.2
AN      I1.3
AN      I1.4
AN      I1.5
A       SM76.7
TON     T105, +3

Network 21
LD      T105
R       Q0.5, 2
END_SUBROUTINE_BLOCK
SUBROUTINE_BLOCK REMOTO:SBR4
TITLE=Modo remoto: ordenes de la PC (VB1100..VB1119) y maquina de pasos (VW1136)
BEGIN
Network 1 // Limites de velocidad: ciclo X/Y entre 200 y 2000 us, base entre 250 y 4000 us
LDW<    VW1116, +200
MOVW    +200, VW1116

Network 2
LDW>    VW1116, +2000
MOVW    +2000, VW1116

Network 3
LDW<    VW1118, +250
MOVW    +250, VW1118

Network 4
LDW>    VW1118, +4000
MOVW    +4000, VW1118

Network 5 // Watchdog: con una orden de la PC en curso, el latido VW1102 debe cambiar cada 2 s
LDW=    VW1102, VW1160
A       V1200.6
AW<>    VW1136, +0
TON     T110, +20

Network 6
LDW<>   VW1102, VW1160
MOVW    VW1102, VW1160

Network 7
LD      T110
S       V1200.5, 1
MOVB    3, VB1123
CALL    SBR0

Network 8 // Orden nueva: la secuencia VB1101 cambio
LDB<>   VB1101, VB1121
MOVB    VB1101, VB1121
MOVB    VB1100, VB1162
MOVB    0, VB1123
R       V1200.5, 1
S       V1200.7, 1

Network 9 // Orden 3: paro (siempre se acepta)
LD      V1200.7
AB=     VB1162, 3
R       V1200.7, 1
MOVB    6, VB1123
CALL    SBR0

Network 10 // Otra orden con un movimiento en curso: se rechaza (error 1)
LD      V1200.7
AW<>    VW1136, +0
MOVB    1, VB1123
R       V1200.7, 1

Network 11 // Orden 4: cerrar garra
LD      V1200.7
AB=     VB1162, 4
S       Q0.7, 1
R       V1200.7, 1

Network 12 // Orden 5: abrir garra
LD      V1200.7
AB=     VB1162, 5
R       Q0.7, 1
R       V1200.7, 1

Network 13 // Orden 1: homing
LD      V1200.7
AB=     VB1162, 1
S       V1200.6, 1
MOVW    +100, VW1136
MOVB    1, VB1120
R       V1200.7, 1

```

```
Network 14 // I0.7 del panel en reposo: homing sin la PC
LD     I0.7
EU
AW=    VW1136, +0
R      V1200.6, 1
MOVW   +100, VW1136
MOVB   1, VB1120

Network 15 // Orden 2 sin homing previo: se rechaza (error 4)
LD     V1200.7
AB=    VB1162, 2
AN     V1200.3
MOVB   4, VB1123
R      V1200.7, 1

Network 16 // Orden 2: posiciones de destino
LD     V1200.7
AB=    VB1162, 2
MOVD   VD1128, VD1164
+D     VD1108, VD1164
MOVD   VD1132, VD1168
+D     VD1112, VD1168
MOVD   VD1124, VD1172
+D     VD1104, VD1172

Network 17 // Destino fuera del recorrido: se rechaza (error 2)
LD     V1200.7
AB=    VB1162, 2
LDD<   VD1164, +0
OD>    VD1164, +34000
OD<    VD1168, +0
OD>    VD1168, +34000
OD<    VD1172, -50000
OD>    VD1172, +50000
ALD
MOVB   2, VB1123
R      V1200.7, 1

Network 18 // Orden 2 aceptada
LD     V1200.7
AB=    VB1162, 2
S      V1200.6, 1
MOVW   +10, VW1136
MOVB   2, VB1120
R      V1200.7, 1

Network 19 // Orden desconocida (error 5)
LD     V1200.7
MOVB   5, VB1123
R      V1200.7, 1

Network 20 // Paso 10: valores absolutos de los tramos
LDW=   VW1136, +10
MOVD   VD1108, VD1176
MOVD   VD1112, VD1180
MOVD   VD1104, VD1184

Network 21
LDW=   VW1136, +10
AD<    VD1108, +0
MOVD   +0, VD1176
-D     VD1108, VD1176

Network 22
LDW=   VW1136, +10
AD<    VD1112, +0
MOVD   +0, VD1180
-D     VD1112, VD1180

Network 23
LDW=   VW1136, +10
AD<    VD1104, +0
MOVD   +0, VD1184
-D     VD1104, VD1184

Network 24 // Tramo comun = el menor de X e Y; resto = el mayor menos el comun
LDW=   VW1136, +10
MOVD   VD1176, VD1188
MOVD   VD1176, VD1192

Network 25
LDW=   VW1136, +10
AD<    VD1180, VD1188
MOVD   VD1180, VD1188

Network 26
LDW=   VW1136, +10
AD>    VD1180, VD1192
MOVD   VD1180, VD1192

Network 27
LDW=   VW1136, +10
-D     VD1188, VD1192

Network 28 // Reles y direcciones
```

```
LDW=   VW1136, +10
AD>    VD1176, +0
S      Q0.5, 1

Network 29
LDW=   VW1136, +10
AD>    VD1180, +0
S      Q0.6, 1

Network 30
LDW=   VW1136, +10
AD<    VD1108, +0
S      Q0.3, 1

Network 31
LDW=   VW1136, +10
AD>=   VD1108, +0
R      Q0.3, 1

Network 32
LDW=   VW1136, +10
AD<    VD1112, +0
S      Q0.4, 1

Network 33
LDW=   VW1136, +10
AD>=   VD1112, +0
R      Q0.4, 1

Network 34
LDW=   VW1136, +10
AD<    VD1104, +0
S      Q0.2, 1

Network 35
LDW=   VW1136, +10
AD>=   VD1104, +0
R      Q0.2, 1

Network 36 // 300 ms para que cierren los reles
LDW=   VW1136, +10
TON    T111, +3

Network 37
LDW=   VW1136, +10
A      T111
MOVW   +11, VW1136

Network 38 // Paso 11: la base arranca en paralelo
LDW=   VW1136, +11
AD>    VD1184, +0
A      SM66.7
MOVD   VD1184, VD1202
MOVW   VW1118, VW1198
CALL   SBR1

Network 39 // Paso 11: tramo comun de X e Y (K1 y K2 cerrados)
LDW=   VW1136, +11
AD>    VD1188, +0
A      SM76.7
MOVD   VD1188, VD1202
MOVW   VW1116, VW1196
S      V1201.0, 1
CALL   SBR2

Network 40 // Tramo comun arrancado: paso 12
LD     V1201.0
R      V1201.0, 1
MOVW   +12, VW1136

Network 41 // Sin tramo comun: paso 13
LDW=   VW1136, +11
AD<=   VD1188, +0
MOVW   +13, VW1136

Network 42 // Paso 12: fin del tramo comun; se abre el rele del eje que ya llego
LDW=   VW1136, +12
TON    T112, +1

Network 43
LDW=   VW1136, +12
A      T112
A      SM76.7
LPS
AD<=   VD1176, VD1188
R      Q0.5, 1
LRD
AD<=   VD1180, VD1188
R      Q0.6, 1
LPP
MOVW   +13, VW1136

Network 44 // Paso 13: espera 300 ms y da el resto al eje que falta
LDW=   VW1136, +13
TON    T113, +3
```

```
654
655  Network 45
656  LDW=   VW1136, +13
657  A      T113
658  A      SM76.7
659  AD>    VD1192, +0
660  MOVD   VD1192, VD1202
661  MOVW   VW1116, VW1196
662  S      V1201.0, 1
663  CALL   SBR2
664
665  Network 46 // Resto arrancado: paso 14
666  LD     V1201.0
667  R      V1201.0, 1
668  MOVW   +14, VW1136
669
670  Network 47 // Sin resto: paso 15
671  LDW=   VW1136, +13
672  A      T113
673  AD<=   VD1192, +0
674  MOVW   +15, VW1136
675
676  Network 48 // Paso 14: fin del resto
677  LDW=   VW1136, +14
678  TON    T114, +1
679
680  Network 49
681  LDW=   VW1136, +14
682  A      T114
683  A      SM76.7
684  MOVW   +15, VW1136
685
686  Network 50 // Paso 15: espera a la base, abre los reles y vuelve al reposo
687  LDW=   VW1136, +15
688  TON    T115, +1
689
690  Network 51
691  LDW=   VW1136, +15
692  A      T115
693  A      SM66.7
694  A      SM76.7
695  R      Q0.5, 2
696  R      V1200.6, 1
697  MOVB   0, VB1120
698  MOVW   +0, VW1136
699
700  Network 52 // Homing paso 100: Y hacia abajo (K2, Q0.4 = 1)
701  LDW=   VW1136, +100
702  R      Q0.5, 1
703  S      Q0.6, 1
704  S      Q0.4, 1
705  TON    T120, +3
706
707  Network 53
708  LDW=   VW1136, +100
709  A      T120
710  A      I0.3
711  MOVW   +103, VW1136
712
713  Network 54
714  LDW=   VW1136, +100
715  A      T120
716  AN     I0.3
717  A      SM76.7
718  MOVB   16#85, SMB77
719  MOVW   +250, SMW78
720  MOVD   +500000, SMD82
721  PLS    1
722  MOVW   +102, VW1136
723
724  Network 55 // Paso 102: llega a I0.3 (la red 6 del OB1 ya corta el PTO y pone Y = 0)
725  LDW=   VW1136, +102
726  A      I0.3
727  MOVB   16#0, SMB77
728  PLS    1
729  MOVD   +0, VD1132
730  MOVW   +103, VW1136
731
732  Network 56 // Paso 103: X hacia atras (K1, Q0.3 = 1)
733  LDW=   VW1136, +103
734  R      Q0.6, 1
735  S      Q0.5, 1
736  S      Q0.3, 1
737  TON    T121, +3
738
739  Network 57
740  LDW=   VW1136, +103
741  A      T121
742  A      I0.1
743  MOVW   +106, VW1136
744
745  Network 58
746  LDW=   VW1136, +103
747  A      T121
748  AN     I0.1
```

```
A      SM76.7
MOVB   16#85, SMB77
MOVW   +250, SMW78
MOVD   +500000, SMD82
PLS    1
MOVW   +105, VW1136

Network 59 // Paso 105: llega a I0.1
LDW=   VW1136, +105
A      I0.1
MOVB   16#0, SMB77
PLS    1
MOVD   +0, VD1128
MOVW   +106, VW1136

Network 60 // Paso 106: abre la garra y gira la base (Q0.2 = 1) hacia el inductivo
LDW=   VW1136, +106
R      Q0.5, 1
R      Q0.7, 1
S      Q0.2, 1
TON    T122, +4

Network 61
LDW=   VW1136, +106
A      T122
A      I0.0
MOVW   +109, VW1136

Network 62
LDW=   VW1136, +106
A      T122
AN     I0.0
A      SM66.7
MOVB   16#85, SMB67
MOVW   +400, SMW68
MOVD   +500000, SMD72
PLS    0
MOVW   +108, VW1136

Network 63 // Paso 108: llega a I0.0
LDW=   VW1136, +108
A      I0.0
MOVB   16#0, SMB67
PLS    0
MOVW   +109, VW1136

Network 64 // Paso 109: origen en los tres ejes
LDW=   VW1136, +109
MOVD   +0, VD1124
MOVD   +0, VD1128
MOVD   +0, VD1132
R      V1200.0, 3
S      V1200.3, 1
R      V1200.6, 1
MOVB   0, VB1120
MOVW   +0, VW1136
END_SUBROUTINE_BLOCK
```

Capítulo C

Script de la base de datos

Archivo `semestres/2026_II/base-de-datos/scripts/brazo_4gdl.sql`.

```sql
-- ============================================================
-- Base de datos de telemetría del brazo XK-JS1-1 (4 GDL) - II/2026
-- PostgreSQL 13 o superior.
--
-- Parte del modelo E-R preliminar del equipo (modelo/modelo_er_preliminar.png)
-- y lo lleva a tercera forma normal (3FN):
--   * 1FN: sin grupos repetidos (los destinos de una orden van en orden_parametro,
--          una fila por eje; la posición X, Y, Z no va en una sola celda).
--   * 2FN: cada tabla tiene clave primaria simple o, si es compuesta, todos los
--          atributos dependen de la clave completa.
--   * 3FN: sin dependencias transitivas (la duración de un ciclo no se guarda:
--          se calcula en la vista v_ciclo; la unidad y el rango de una medida
--          viven en variable, no en cada muestra).
-- Los tags siguen la propuesta ISA-5.1 del manual técnico (lazos 100 a 104).
--
-- Uso:
--   createdb brazo4gdl
--   psql -d brazo4gdl -f brazo_4gdl.sql
-- ============================================================

BEGIN;

-- ------------------------------------------------------------ catálogos
CREATE TABLE sistema (
    id          SMALLSERIAL PRIMARY KEY,
    nombre      VARCHAR(60)  NOT NULL UNIQUE,
    tipo        VARCHAR(40)  NOT NULL,
    plc         VARCHAR(60)  NOT NULL
);

CREATE TABLE eje (
    id              SMALLSERIAL PRIMARY KEY,
    sistema_id      SMALLINT     NOT NULL REFERENCES sistema(id),
    codigo          CHAR(1)      NOT NULL,              -- Z (base), X, Y, G (garra)
    nombre          VARCHAR(40)  NOT NULL,
    tipo_mov        VARCHAR(12)  NOT NULL CHECK (tipo_mov IN ('rotacional', 'lineal', 'binario')),
    unidad          VARCHAR(8)   NOT NULL,
    pulsos_unidad   NUMERIC(10,4),                      -- NULL para la garra
    lazo_isa        SMALLINT     NOT NULL,
    UNIQUE (sistema_id, codigo)
);

CREATE TABLE punto_io (                                 -- señales del PLC (entidad PUNT_I/O)
    id              SMALLSERIAL PRIMARY KEY,
    sistema_id      SMALLINT     NOT NULL REFERENCES sistema(id),
    eje_id          SMALLINT     REFERENCES eje(id),    -- NULL: señal general del panel
    direccion       VARCHAR(8)   NOT NULL,              -- I0.1, Q0.5 ...
    tag_isa         VARCHAR(16)  NOT NULL UNIQUE,
    tipo_senal      VARCHAR(4)   NOT NULL CHECK (tipo_senal IN ('DI', 'DO', 'PTO')),
    descripcion     VARCHAR(120) NOT NULL,
    UNIQUE (sistema_id, direccion)
);

CREATE TABLE variable (                                 -- magnitudes continuas (entidad VARIABLE_PROCESO)
    id              SMALLSERIAL PRIMARY KEY,
    eje_id          SMALLINT     NOT NULL REFERENCES eje(id),
    tag_isa         VARCHAR(16)  NOT NULL UNIQUE,
    clase           CHAR(2)      NOT NULL CHECK (clase IN ('PV', 'MV', 'DV')),
    nombre          VARCHAR(80)  NOT NULL,
    unidad          VARCHAR(8)   NOT NULL,
    rango_min       DOUBLE PRECISION NOT NULL,
    rango_max       DOUBLE PRECISION NOT NULL,
    CHECK (rango_min < rango_max)
);

CREATE TABLE programa_plc (
    id              SMALLSERIAL PRIMARY KEY,
    nombre          VARCHAR(40)  NOT NULL UNIQUE,
    perfil_app      VARCHAR(12)  NOT NULL UNIQUE,       -- gemelo, cargado, monitor
    firma           VARCHAR(40),
    descripcion     VARCHAR(160)
);

CREATE TABLE alarma (                                   -- catálogo (entidad ALERTA)
    id              SMALLSERIAL PRIMARY KEY,
    codigo          VARCHAR(24)  NOT NULL UNIQUE,
    severidad       SMALLINT     NOT NULL CHECK (severidad BETWEEN 1 AND 3),   -- 1 aviso, 2 falla, 3 crítica
    descripcion     VARCHAR(160) NOT NULL
);

-- ------------------------------------------------------------ usuarios
CREATE TABLE rol (
    id      SMALLSERIAL PRIMARY KEY,
```

```
    nombre  VARCHAR(16) NOT NULL UNIQUE
);

CREATE TABLE usuario (
    id          SERIAL PRIMARY KEY,
    nombre      VARCHAR(32)  NOT NULL UNIQUE,
    rol_id      SMALLINT     NOT NULL REFERENCES rol(id),
    clave_hash  VARCHAR(200) NOT NULL,                  -- PBKDF2, igual que usuarios.json de la app
    creado_en   TIMESTAMPTZ  NOT NULL DEFAULT now()
);

CREATE TABLE sesion (                                   -- registro de ingresos (logs)
    id          BIGSERIAL PRIMARY KEY,
    usuario_id  INTEGER      REFERENCES usuario(id),    -- NULL: usuario inexistente
    nombre_usado VARCHAR(32) NOT NULL,
    ip          INET         NOT NULL,
    instante    TIMESTAMPTZ  NOT NULL DEFAULT now(),
    exito       BOOLEAN      NOT NULL,
    fin         TIMESTAMPTZ
);

-- ------------------------------------------------------------- operación
CREATE TABLE ciclo_operacion (                          -- cada vez que se elige un modo en la app
    id          BIGSERIAL PRIMARY KEY,
    sistema_id  SMALLINT     NOT NULL REFERENCES sistema(id),
    programa_id SMALLINT     REFERENCES programa_plc(id),
    modo        VARCHAR(12)  NOT NULL CHECK (modo IN ('simulacion', 'local', 'red')),
    inicio      TIMESTAMPTZ  NOT NULL DEFAULT now(),
    fin         TIMESTAMPTZ,
    CHECK (fin IS NULL OR fin >= inicio)
);

CREATE TABLE muestra (                                  -- telemetría de variables continuas
    id          BIGSERIAL PRIMARY KEY,
    ciclo_id    BIGINT           NOT NULL REFERENCES ciclo_operacion(id),
    variable_id SMALLINT         NOT NULL REFERENCES variable(id),
    instante    TIMESTAMPTZ      NOT NULL,
    valor       DOUBLE PRECISION NOT NULL
);
CREATE INDEX muestra_variable_instante ON muestra (variable_id, instante);

CREATE TABLE estado_io (                                -- cambios de las señales digitales
    id          BIGSERIAL PRIMARY KEY,
    ciclo_id    BIGINT      NOT NULL REFERENCES ciclo_operacion(id),
    punto_id    SMALLINT    NOT NULL REFERENCES punto_io(id),
    instante    TIMESTAMPTZ NOT NULL,
    valor       BOOLEAN     NOT NULL
);
CREATE INDEX estado_io_punto_instante ON estado_io (punto_id, instante);

CREATE TABLE orden (                                    -- acción de control (entidad ACCION_CONT)
    id          BIGSERIAL PRIMARY KEY,
    ciclo_id    BIGINT       NOT NULL REFERENCES ciclo_operacion(id),
    usuario_id  INTEGER      REFERENCES usuario(id),    -- NULL: PC del laboratorio (local)
    instante    TIMESTAMPTZ  NOT NULL DEFAULT now(),
    tipo        VARCHAR(16)  NOT NULL CHECK (tipo IN ('homing', 'mover', 'garra', 'detener', 'velocidad',
                                                      'iniciar', 'emergencia', 'rearmar', 'mando')),
    aceptada    BOOLEAN      NOT NULL,
    motivo      VARCHAR(200)                           -- motivo del rechazo
);

CREATE TABLE orden_parametro (                          -- destinos de una orden, uno por eje (1FN)
    orden_id    BIGINT           NOT NULL REFERENCES orden(id) ON DELETE CASCADE,
    eje_id      SMALLINT         NOT NULL REFERENCES eje(id),
    valor       DOUBLE PRECISION NOT NULL,
    PRIMARY KEY (orden_id, eje_id)
);

CREATE TABLE evento_alarma (
    id          BIGSERIAL PRIMARY KEY,
    alarma_id   SMALLINT     NOT NULL REFERENCES alarma(id),
    ciclo_id    BIGINT       NOT NULL REFERENCES ciclo_operacion(id),
    instante    TIMESTAMPTZ  NOT NULL DEFAULT now(),
    activa      BOOLEAN      NOT NULL,
    detalle     VARCHAR(200)
);

-- ------------------------------------------------------------- reglas
-- Trigger: una muestra fuera del rango de su variable no entra (pide la guía de laboratorio 1).
CREATE FUNCTION validar_muestra() RETURNS trigger AS $$
DECLARE
    r variable%ROWTYPE;
BEGIN
    SELECT * INTO r FROM variable WHERE id = NEW.variable_id;
    IF NEW.valor < r.rango_min OR NEW.valor > r.rango_max THEN
        RAISE EXCEPTION 'muestra␣fuera␣de␣rango␣para␣%␣(%):␣%␣no␣está␣en␣[%,␣%]',
            r.tag_isa, r.unidad, NEW.valor, r.rango_min, r.rango_max;
    END IF;
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER muestra_en_rango BEFORE INSERT OR UPDATE ON muestra
    FOR EACH ROW EXECUTE FUNCTION validar_muestra();

```

```sql
-- Vista: duración de cada ciclo (se calcula, no se guarda: 3FN)
CREATE VIEW v_ciclo AS
SELECT c.id, c.modo, p.nombre AS programa, c.inicio, c.fin,
       COALESCE(c.fin, now()) - c.inicio AS duracion
FROM ciclo_operacion c LEFT JOIN programa_plc p ON p.id = c.programa_id;

-- Vista: última posición conocida de cada eje
CREATE VIEW v_ultima_posicion AS
SELECT DISTINCT ON (m.variable_id) v.tag_isa, v.nombre, v.unidad, m.valor, m.instante
FROM muestra m JOIN variable v ON v.id = m.variable_id
WHERE v.clase = 'PV'
ORDER BY m.variable_id, m.instante DESC;

-- ------------------------------------------------------------ datos de la planta
INSERT INTO sistema (nombre, tipo, plc) VALUES
    ('Brazo XK-JS1-1', 'brazo robótico 4 GDL', 'Siemens S7-200 CPU 224 CN');

INSERT INTO eje (sistema_id, codigo, nombre, tipo_mov, unidad, pulsos_unidad, lazo_isa) VALUES
    (1, 'Z', 'base (theta)', 'rotacional', 'deg', 190.3, 101),
    (1, 'X', 'carro X (alcance)', 'lineal', 'mm', 200, 102),
    (1, 'Y', 'carro Y (altura)', 'lineal', 'mm', 200, 103),
    (1, 'G', 'garra', 'binario', '-', NULL, 104);

INSERT INTO punto_io (sistema_id, eje_id, direccion, tag_isa, tipo_senal, descripcion) VALUES
    (1, 1,    'I0.0', 'ZS-101',   'DI',  'sensor inductivo de la base (theta = 0)'),
    (1, 2,    'I0.1', 'ZSL-102',  'DI',  'final de carrera X atrás (origen)'),
    (1, 2,    'I0.2', 'ZSH-102',  'DI',  'final de carrera X adelante'),
    (1, 3,    'I0.3', 'ZSL-103',  'DI',  'final de carrera Y abajo (origen)'),
    (1, 3,    'I0.4', 'ZSH-103',  'DI',  'final de carrera Y arriba'),
    (1, NULL, 'I0.5', 'HS-100',   'DI',  'selector manual / automático'),
    (1, NULL, 'I0.6', 'HS-100P',  'DI',  'pulsador de paro'),
    (1, 4,    'I0.7', 'HS-104',   'DI',  'garra / inicio'),
    (1, 1,    'I1.0', 'HS-101A',  'DI',  'jog base +'),
    (1, 1,    'I1.1', 'HS-101B',  'DI',  'jog base -'),
    (1, 2,    'I1.2', 'HS-102A',  'DI',  'jog X adelante'),
    (1, 2,    'I1.3', 'HS-102B',  'DI',  'jog X atrás'),
    (1, 3,    'I1.4', 'HS-103A',  'DI',  'jog Y arriba'),
    (1, 3,    'I1.5', 'HS-103B',  'DI',  'jog Y abajo'),
    (1, 1,    'Q0.0', 'ZY-101',   'PTO', 'tren de pulsos de la base (PTO0)'),
    (1, NULL, 'Q0.1', 'ZY-100',   'PTO', 'tren de pulsos compartido X/Y (PTO1)'),
    (1, 1,    'Q0.2', 'ZY-101D',  'DO',  'dirección base (1 = hacia el inductivo)'),
    (1, 2,    'Q0.3', 'ZY-102D',  'DO',  'dirección X (0 = adelante)'),
    (1, 3,    'Q0.4', 'ZY-103D',  'DO',  'dirección Y (0 = arriba)'),
    (1, 2,    'Q0.5', 'ZY-102K',  'DO',  'relé K1: Q0.1 al driver X'),
    (1, 3,    'Q0.6', 'ZY-103K',  'DO',  'relé K2: Q0.1 al driver Y'),
    (1, 4,    'Q0.7', 'XY-104',   'DO',  'relé K3 del solenoide de la garra (1 = cerrada)');

INSERT INTO variable (eje_id, tag_isa, clase, nombre, unidad, rango_min, rango_max) VALUES
    (1, 'ZI-101',  'PV', 'posición de la base', 'deg', -263, 263),
    (2, 'ZI-102',  'PV', 'posición del carro X', 'mm', 0, 170),
    (3, 'ZI-103',  'PV', 'posición del carro Y', 'mm', 0, 170),
    (1, 'SY-101',  'MV', 'frecuencia del tren de la base', 'Hz', 0, 20000),
    (2, 'SY-100',  'MV', 'frecuencia del tren X/Y', 'Hz', 0, 20000);

INSERT INTO programa_plc (nombre, perfil_app, firma, descripcion) VALUES
    ('brazo_gemelo', 'gemelo', 'VB1141 = 16#47', 'posición contada en el PLC, órdenes por VB1100-VB1119'),
    ('brazo_secuencias_original_v1', 'cargado', 'VD340 = 30000, VD344 = 25000', 'homing y 2 secuencias fijas'),
    ('otro', 'monitor', NULL, 'programa no reconocido: solo lectura');

INSERT INTO alarma (codigo, severidad, descripcion) VALUES
    ('comunicacion', 2, 'se perdió la comunicación PPI con el PLC'),
    ('watchdog', 3, 'el PLC detuvo un movimiento porque la PC dejó de dar señales de vida'),
    ('paro', 2, 'paro por pulsador, selector u orden de la PC'),
    ('fuera_de_rango', 1, 'destino fuera del recorrido'),
    ('sin_homing', 1, 'orden de movimiento sin referencia'),
    ('ocupado', 1, 'orden con un movimiento en curso'),
    ('falla_homing', 3, 'homing sin sensor o con tiempo excedido'),
    ('posicion_incierta', 2, 'tramo interrumpido: hace falta un nuevo homing');

INSERT INTO rol (nombre) VALUES ('observador'), ('operador');

COMMIT;
```